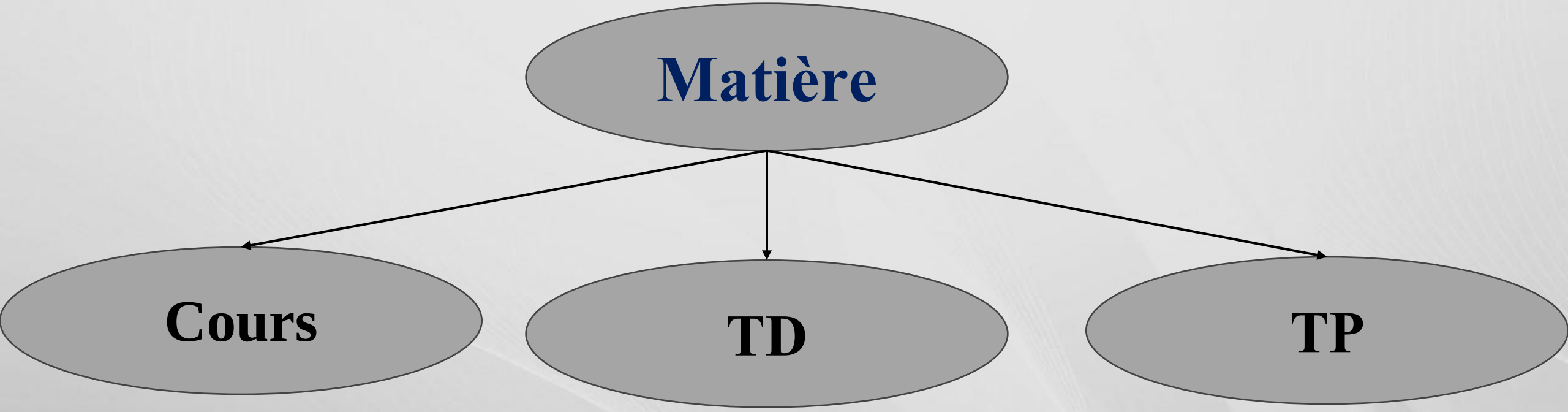
École supérieure en Sciences et Technologies de l'Informatique et du Numérique

Module: Cryptographie Avancée

Niveau: 2CS – Cyber sécurité

Présenter par : DR BOUCHOUCHA

Année Universitaire 2024-2025



Crédit : 5

Coefficient: 3

Programme

- CHAPITRE 1: Rappels sur les bases de la cryptographie
- CHAPITRE 2: La cryptanalyse
- CHAPITRE 3: La cryptographie à courbe elliptique
- CHAPITRE 4 : Preuve à divulgation nulle de connaissance
- CHPITRE 5: La cryptographie Quantique

École supérieure en Sciences et Technologies de l'Informatique et du Numérique

Module: Cryptographie Avancée
Niveau: 2CS – Cyber-Sécurité

Chapitre 1
Rappels sur les bases de la cryptographie

Présenter par : DR BOUCHOUCHA

Année Universitaire 2024-2025

PLAN

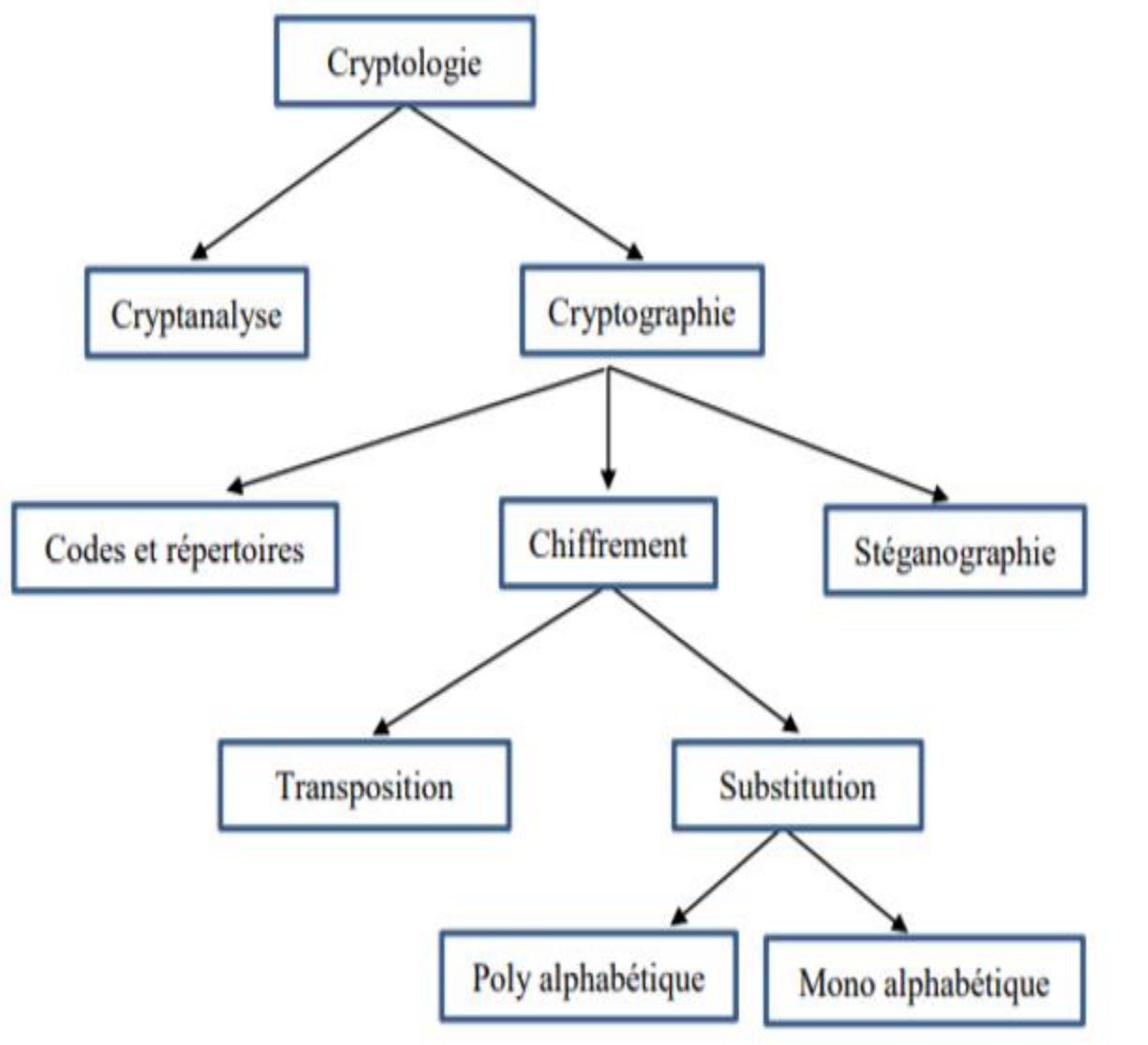
1. Introduction
2. Les objectifs de la sécurité
3. Vocabulaire et définitions
4. Les crypto-systémes
5. Les fonctions à sens uniques
6. Les fonctions de Hachages
7. La signature numérique
8. Le certificat électronique
9. Codes d'Authentification des messages
10. Les générateurs pseudos aléatoires

Introduction

La sécurité est une fonction incontournable des réseaux de communication. Elle consiste à éviter que des curieux puissent lire ou modifier les messages destinés à d'autre, ou des individus qui essaient d'utiliser des services en ligne auxquels ils ne sont pas autorisés à accéder.

Pour cela un ensemble de moyens sont mis en œuvre pour minimiser la vulnérabilité d'un système contre des menaces accidentelles ou intentionnelles.

La sécurité a pour but d'assurer que les ressources matérielles et logicielles d'une organisation soient uniquement utilisées dans le cadre prévu.



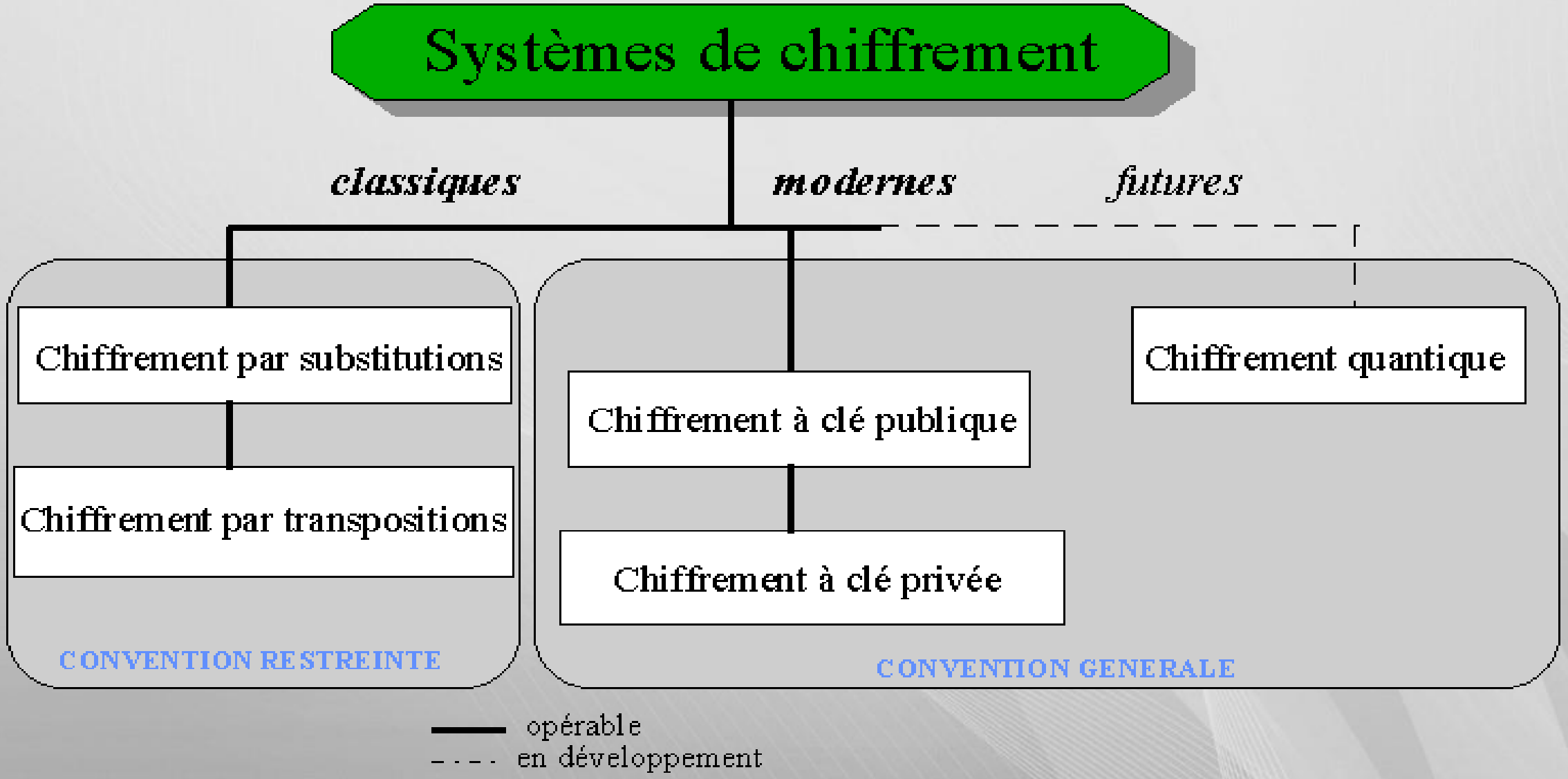


fig 1.1 Principales techniques en cryptographie

Protection des données

Au repos

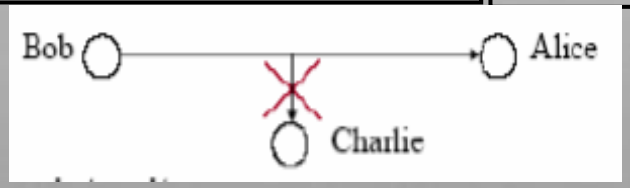
- contrôle d'accès
  - identification
  - autorisation
  - audit
- disponibilité

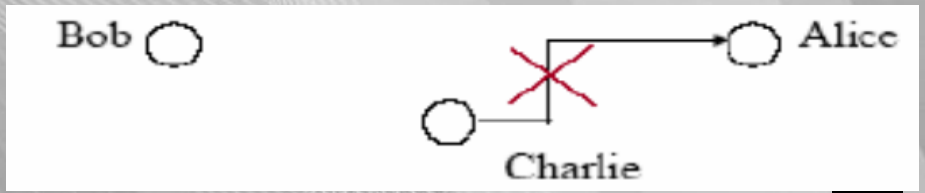
En transit

- confidentialité
- intégrité
- authentification
- non répudiation

2. Les objectifs de la sécurité

confidentialité	Authentification	Non répudiation de l'information	Intégrité	Disponibilité
Elle permet d'assurer qu'une communication reste privée entre un émetteur et un destinataire.	Elle consiste à vérifier l'identité d'un utilisateur. Il permet un contrôle d'accès aux ressources uniquement aux personnes autorisées.	Elle permet de garantir qu'aucun des correspondants ne pourra nier la transaction.	Elle permet d'assurer qu'aucune modification n'aurait eu lieu entre l'émetteur et le destinataire au cours de la transmission	L'information doit être utilisable à la demande





3. Vocabulaire et définitions

❖ **Cryptologie**

La cryptologie est la science des messages secrets. Elle est la réunion de deux disciplines qui s'alimentent l'une à l'autre à savoir, la cryptographie et la cryptanalyse.

❖ **Cryptographie**

Le mot « Cryptographie » est composé des mots grecs :

- CRYPTO = caché
- GRAPHY = écrire

C'est donc l'art de l'écriture secrète. C'est une science permettant de préserver la confidentialité des échanges.

❖ **Cryptanalyse**

La cryptanalyse est l'étude des techniques mathématiques visant à tenter de déjouer les techniques cryptographiques et, plus généralement, les services de sécurité de l'information.

❖ **Crypto système**

Un terme général désignant un ensemble de primitives cryptographiques utilisées pour fournir des services de sécurité des informations. Le plus souvent, le terme est utilisé en conjonction avec des primitives assurant la confidentialité, c'est-à-dire le chiffrement.

❖ **Cryptogramme**

Un cryptogramme est un message écrit à l’aide d’un système de chiffrement.

❖ **Chiffre:**

Ensemble de procédés et de symboles (lettres, nombres, signes, etc.) utilisées pour remplacer les lettres du message à chiffrer. On distingue généralement **les chiffres à transposition et ceux à substitution**.

❖ **Chiffrement (cryptage) :**

Opération qui consiste à transformer un texte clair en cryptogramme. On parle de chiffrement, car, à la Renaissance, on utilisait principalement des chiffres arabes comme caractères de l’écriture secrète. Claire (ou message clair) Version intelligible d’un message et compréhensible par tous.

- **Déchiffrement:**
  Opération inverse du chiffrement. Elle consiste à obtenir la version originale d'un message qui a été précédemment chiffré en connaissant la méthode de chiffrement et les clés (contrairement au décryptage).

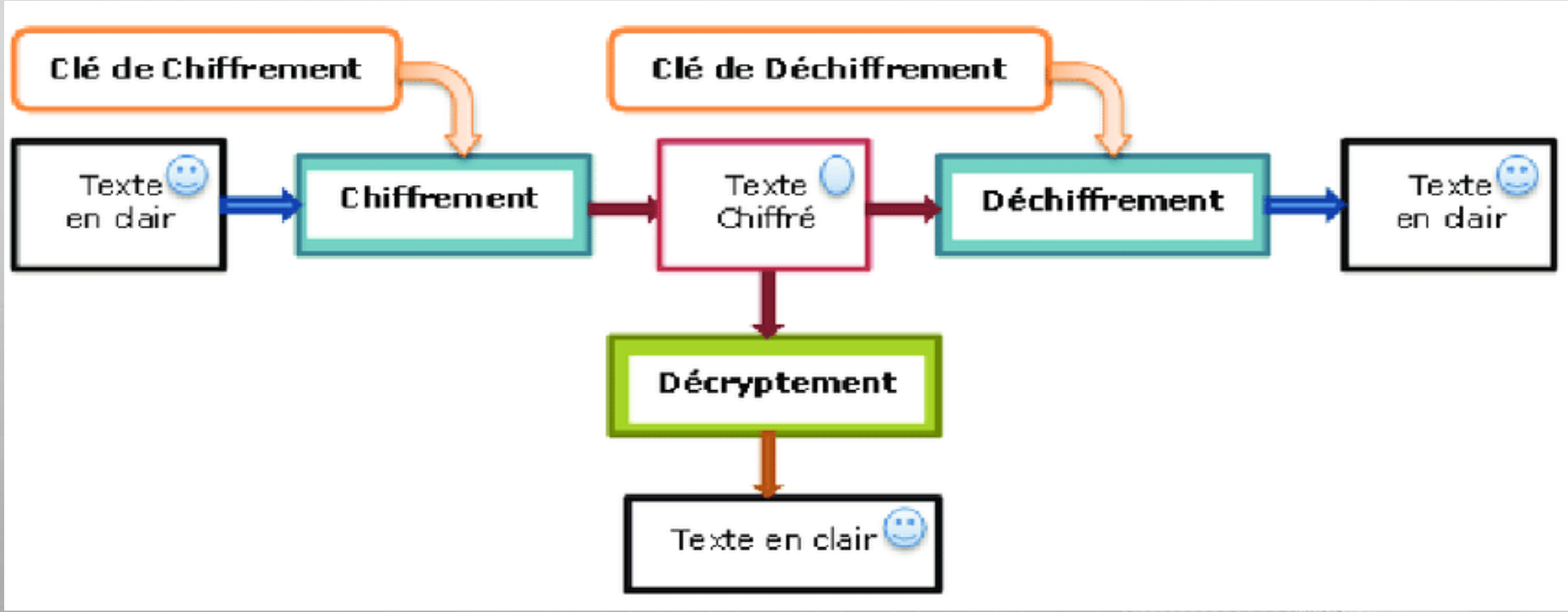
- **Décryptage:**
  Opération qui consiste à retrouver le texte en clair sans disposer des clés théoriquement nécessaires. Il ne faut pas confondre déchiffrement et décryptage.

- **Digramme:**
  Un groupe de deux lettres.

- **Casser un code:** C'est de trouver la clé du code ou le moyen d'accéder à ce qu'il protégeait.

❖ **Clé:**

Dans un système de chiffrement, elle correspond à un nombre, un mot, une phrase, etc. qui permet, grâce à l'algorithme de chiffrement, de chiffrer ou de déchiffrer un message.



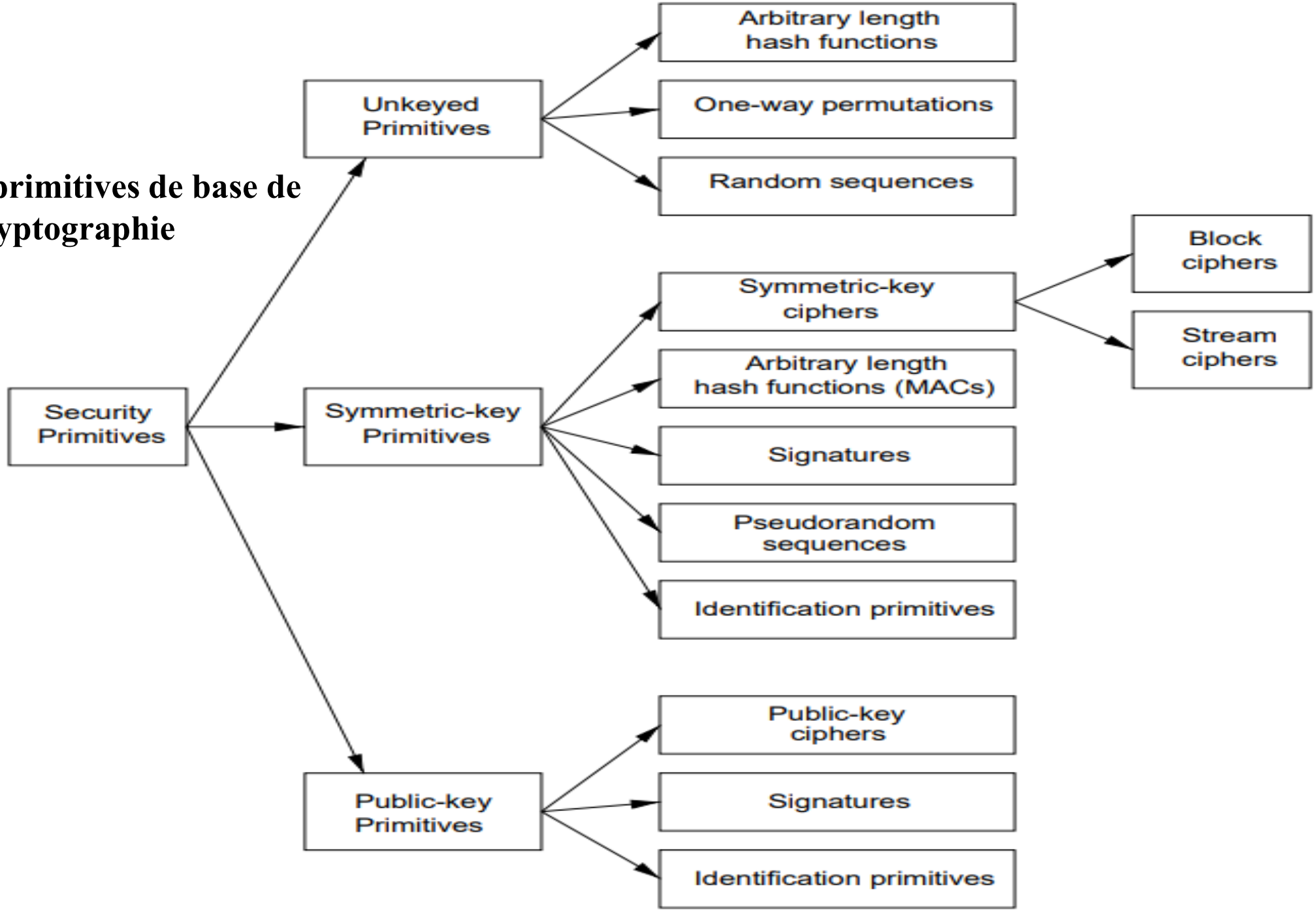
Principes de Kerkchoffs

En 1883 dans un article paru dans le Journal des sciences militaires, Auguste Kerckhoffs posa les principes de la cryptographie classique qui sont valables même à l'ère de la cryptographie moderne.

• Un crypto système sera d'autant plus résistant et sûr qu'il aura été conçu, choisi et implémenté avec la plus grande transparence et soumis ainsi à l'analyse de l'ensemble de la communauté cryptographique.

• Si un algorithme est supposé être secret, il se trouvera toujours quelqu'un soit pour vendre l'algorithme, soit pour le percer à jour, soit pour en découvrir une faiblesse ignorée de ses concepteurs. A ce moment là c'est tout le crypto système qui est à changer et pas seulement la clé.

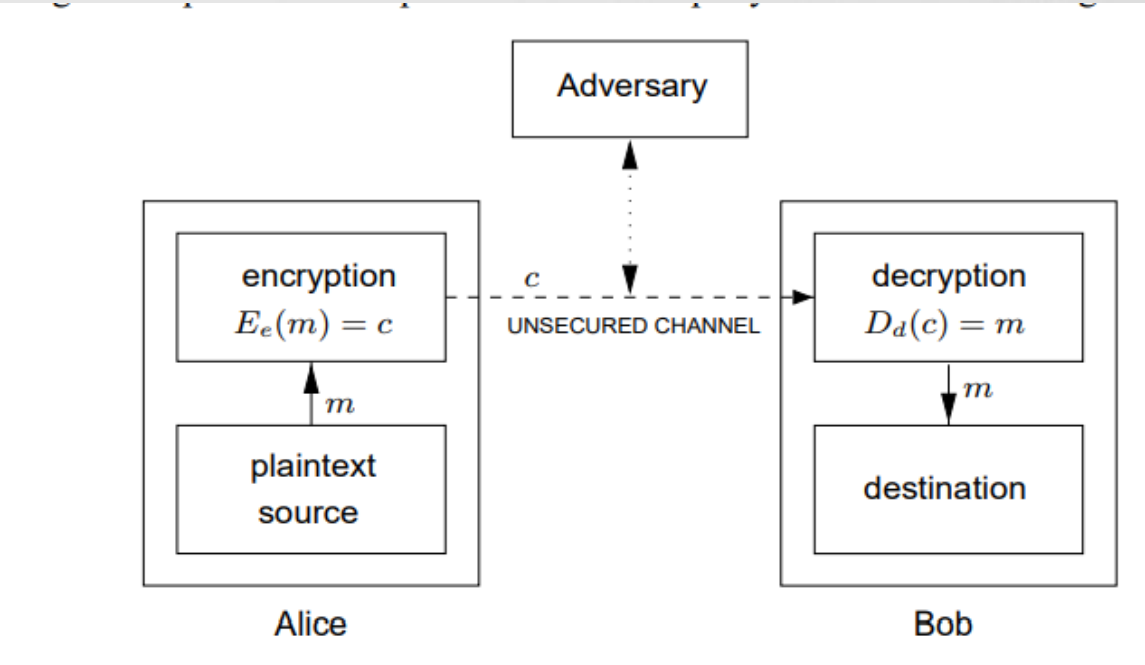
Les primitives de base de la cryptographie



Concepts cryptographiques de base

D'une manière formelle, un système cryptographique est quintuple ***(P, C, K, E, D)*** d'ou :

1. ***P*** est un ensemble fini de blocs de textes clairs possibles.
2. ***C*** est un ensemble fini de blocs de textes chiffrés possibles.
3. ***K*** est un ensemble fini de clefs possibles.



4. Pour tout $k \in K$, il y a **une règle de chiffrement** $\mathbf{e_k} \in \mathbf{E}$ et ***une règle de déchiffrement*** correspondante $d_k \in D$:

Chaque $\mathbf{e_k} : \mathbf{P} \rightarrow \mathbf{C}$ et $d_k : \mathbf{D} \rightarrow \mathbf{P}$ sont des fonctions satisfaisant la propriété de base de la cryptographie telle que : $d_k(e_k(\mathbf{M})) = \mathbf{M}$ pour tout texte clair $\mathbf{M} \in \mathbf{P}$. La principale propriété est la quatrième.

Les crypto-systémes classique

Les méthodes de chiffrement classique en deux familles:

1. **le chiffrement par substitution**
2. **le chiffrement par transposition.**

1. Chiffrement par substituions

Elle consiste à remplacer chaque lettre par une autre lettre selon une règle convenue. Dans le chiffrement par substitutions, chaque lettre est remplacée par une autre, mais garde sa place d'origine. Il existe trois types de chiffrement par substitution :

a) **Chiffre de substitution mono-alphabétique:** Chaque lettre du message d'origine est toujours remplacée par une même autre lettre.

b) **Chiffre de substitution poly-alphabétique:** une même lettre du message d'origine peut être remplacée par plusieurs lettres différentes.

c) **Chiffre de substitution polygrammique** : Les lettres ne sont pas remplacées une par une, mais par blocs de plusieurs (deux ou trois généralement).

1.1. Chiffrement par substituions mono-alphabétique: Chiffrement de César :

La substitution est définie par un décalage de lettres. L'algorithme de chiffrement consiste à décaler les lettres de l'alphabet, selon la clé de chiffrement **k** qui représente le nombre de lettres à décaler.

Description:

Soit **P = C = K = Z_{26}**. Pour $0 \leq K \leq 25$, on définit la fonction de chiffrement par :

$\mathbf{e_K(x) = x + k \bmod 26}$, tandis que, $\mathbf{d_K(y) = y - k \bmod 26}$ pour le déchiffrement

A	B	C	D	E	F	G	H	I	J	K	L	M
0	1	2	3	4	5	6	7	8	9	10	11	12

N	O	P	Q	R	S	T	U	V	W	X	Y	Z
13	14	15	16	17	18	19	20	21	22	23	24	25

1.2.Chiffrement par substituions mono-alphabétique: Chiffrement affine:

Le chiffre affine est une variante du chiffre de César, très pratique à mettre en œuvre sur un ordinateur car il se réduit à des calculs sur des nombres entiers.

Définition:

Soit $\boldsymbol{P = C = Z/Z26}$ et soit $\boldsymbol{K = (a, b) \in Z/Z26 \times Z/Z26}$:le pgcd(a, 26) = 1.

Pour **K = (a, b) ∈ K**, on définit **le chiffrement : ek(x) = ax + b mod 26**,

Et le déchiffrement : $\boldsymbol{dk(y) = a^{-1}(y - b) \bmod 26}$. ***Tel que, (x, y) ∈ Z /Z26***)

2.1. Chiffrement par substituions poly-alphabétique: Chiffrement de Vigenère

Le chiffre de Vigenère est une amélioration du chiffre de César. Sa force réside dans l'utilisation non pas d'un, mais de 26 alphabets décalés pour chiffrer un message. On peut résumer ces décalages.

Ce chiffre utilise une clé qui définit le décalage pour chaque lettre du message (A : décalage de 0 pas, B : 1 pas, C : 2 pas, .. , Z : 25 pas). La grande force du chiffre de Vigenère est que la même lettre sera chiffrée de différentes manières. **Si la clé est aussi longue que le texte clair, et moyennant quelques précautions d'utilisation, le système est appelé masque jetable.**

Claire	C	R	Y	P	T	O	G	R	A	P	H	I	E
Clé	P	A	S	S	E	P	A	S	S	E	P	A	S
Décalage	15	0	18	18	4	15	0	18	18	4	15	0	18
Chiffré	R	R	Q	H	X	D	G	J	S	T	W	I	W

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

2.2. Chiffrement par substituions poly-alphabétique: Chiffrement de de Playfair

Chiffrement de Playfair est un chiffre polygrammique. Dans le chiffrement de Playfair, on dispose les 25 lettres de l'alphabet (**W exclu, car inutile, on utilise V à la place**) dans une grille de 5x5. La **variante anglaise consiste à garder le W et à fusionner I et J.**

On remplit la matrice par les lettres du mot clé. Si une lettre se répète, on l'écrit qu'une seule fois. Puis on complète par les lettres restantes de l'alphabet.

Méthode de chiffrement

On chiffre le texte par groupes de deux lettres (des digrammes) en appliquant les règles suivantes :

1. Si les deux lettres sont sur les coins d'un rectangle, alors les lettres chiffrées sont sur les deux autres coins.

B	Y	D	G	Z
J	S	F	U	P
L	A	R	K	X
C	O	I	V	E
Q	N	M	H	T

2. Si deux lettres sont sur la même ligne, on prend les deux lettres qui les suivent immédiatement à leur droite.

B	Y	D	G	Z
J	S	F	U	P
L	A	R	K	X
C	O	I	V	E
Q	N	M	H	T

3. Si deux lettres sont sur la même colonne, on prend les deux lettres qui les suivent immédiatement en dessous.

B	Y	D	G	Z
J	S	F	U	P
L	A	R	K	X
C	O	I	V	E
Q	N	M	H	T

2.3. Chiffrement par substituions poly-alphabétique: Chiffrement de Hill

Hill, utilise les propriétés de l'arithmétique **modulaire et des matrices**. Il consiste à chiffrer le message en substituant les lettres du message, non plus lettre à lettre, mais par groupe de lettres. Il permet ainsi de rendre plus difficile le cassage du code par observation des fréquences.

Méthode de chiffrement

Les lettres sont remplacées par leur rang suivant l'alphabet. On choisit une clé K sous forme d'une **matrice de 2X2** telle que **PGCD(det(K), 26) = 1 .** Chaque paire de lettres $\mathbf{L_k}$ **et** $\mathbf{L_{k+1}}$ du message en clair sont remplacées **par** $\mathbf{C_k}$ **et** $\mathbf{C_{k+1}}$:

Le chiffrement d'un message M par $K = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$ est donné par :

$$\begin{pmatrix} C_j \\ C_{j+1} \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} M_j \\ M_{j+1} \end{pmatrix} \quad \text{mod } 26$$

Le décryptage est effectué en utilisant $K^{-1} = \frac{1}{ad-bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$

3. Chiffrement par transposition

Le chiffrement par transposition est une méthode de cryptographie qui consiste à permuter ou réarranger les lettres ou symboles du texte clair pour obtenir un texte chiffré. Contrairement aux méthodes de substitution qui remplacent les lettres par d'autres lettres ou symboles, la transposition conserve les caractères du texte clair, mais leur ordre est modifié selon une règle ou un modèle spécifique.

Types de chiffrement par transposition :

1. **Transposition simple** : les lettres sont réorganisées dans un tableau et lues par colonne.
2. **Chiffrement par grille** : On utilise une grille pour réorganiser les lettres du message. Le texte est écrit dans la grille selon un certain ordre puis lu d'une manière spécifique pour obtenir le texte chiffré.
3. **Chiffrement par permutation complexe** : Il peut y avoir des permutations multiples et des variations dans la façon dont les caractères sont réarrangés, rendant le système plus difficile à déchiffrer.

Les méthodes de chiffrement par transposition, bien que simples, sont vulnérables à des attaques de force brute si la méthode de permutation est connue ou si l'attaque par analyse de fréquence est utilisée. Pour rendre le chiffrement plus sécurisé, ces techniques sont souvent combinées avec des techniques de substitution.

Transposition simple par colonnes

Dans ce type de chiffrement , on va disposer les lettres du message horizontalement sur la matrice de langueur n, puis on collecte les lettres verticalement.

Pour un message de langueur m alors on aura deux cas de figure :

- **m mod n = 0** : dans ce cas, toutes les colonnes ont la même hauteur **[m/n]**.
- **m mod n = i**, tel que **a > 0** : dans ce cas, la hauteur des i premières colonnes est **[m/n]+1**.

La hauteur des **n − i** colonnes restantes est **[m/n].**

Transposition complexe par colonnes

- Une transposition complexe par colonnes s'effectue à partir d'une clé (**mot ou expression**) de la longueur souhaitée.
- On numérote les lettres dans l'ordre alphabétique.
- Si une même lettre apparaît plusieurs fois, elle est numérotée successivement de la gauche vers la droite.

Chiffrement

La clé du message donne à la fois le nombre de colonnes du tableau, et la l'ordre de la récolte des lettres. De la même manière que la transposition simple, on dispose horizontalement les lettres du message sur la matrice de langueur équivalant à la longueur de la clé. On collecte verticalement les lettres suivant l'ordre croissant des lettres de la clé par rapport à l'alphabet.

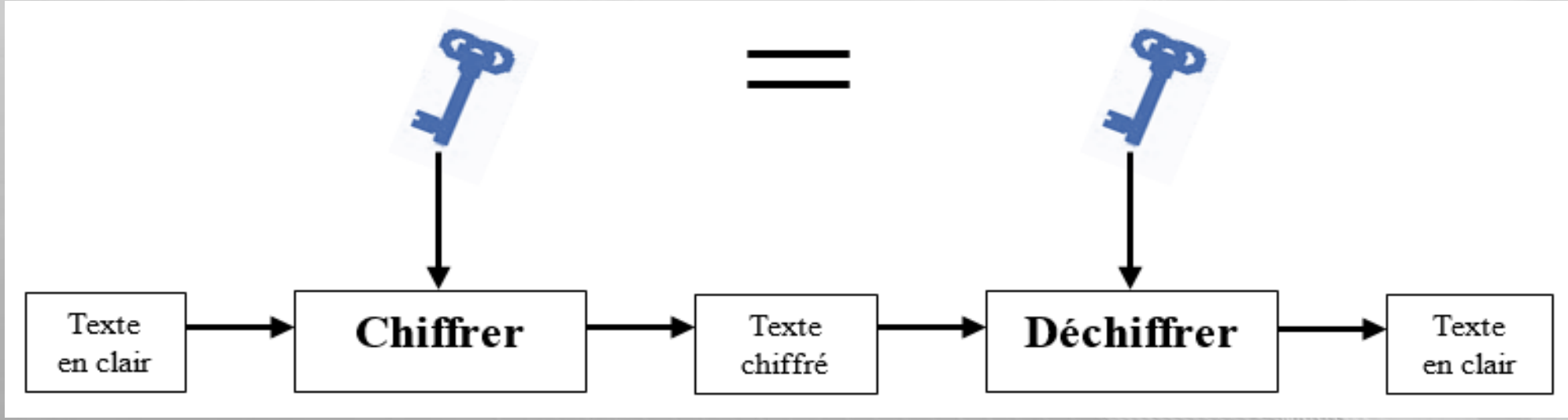
4. Les crypto-systémes

Les systèmes de chiffrement informatique peuvent généralement être classées dans l'une des catégories suivantes :

- chiffrement symétrique,
- chiffrement asymétrique ou à clé publique.

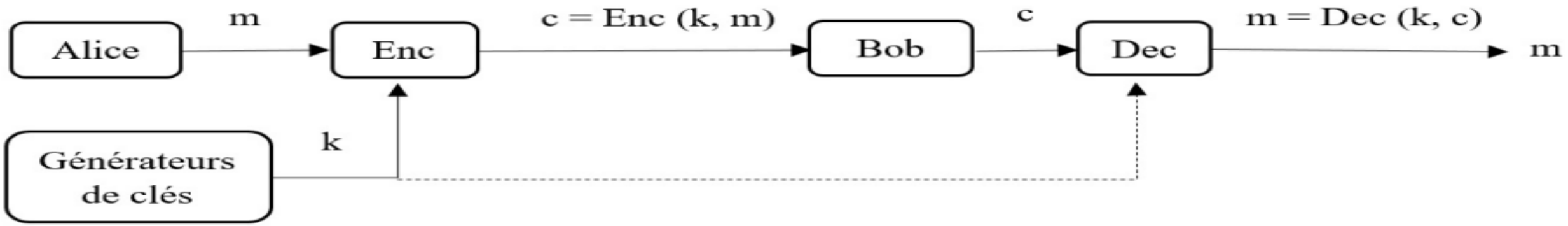
4.1. Chiffrement symétrique

Les chiffrements symétriques aussi appelés chiffrement à clé privé (secrète) sont les héritiers des méthodes cryptographiques anciennes (comme **les substitutions, les transpositions**). Dans ce type de chiffrement, l'expéditeur et le destinataire utilisent une instance différente d'une même clé pour chiffrer et déchiffrer les messages. Le chiffrement symétrique s'appuie grandement sur le fait que **les clés doivent être gardées secrètes**.



Un système de chiffrement symétrique est la donnée de trois algorithmes polynomiaux **E = (Gen, Enc, Dec)** définis comme suit :

- Génération des clés : un algorithme probabiliste génère une clé secrète k à partir d'un paramètre de sécurité.
- Chiffrement : un algorithme probabiliste (**Enc)** prend en entrée un message m et la clé secrète k, et produit un texte chiffré **c**.
- Déchiffrement : un algorithme déterministe (**Dec)** prend en entrée le texte chiffré c et la clé secrète k, et retourne le message clair m.

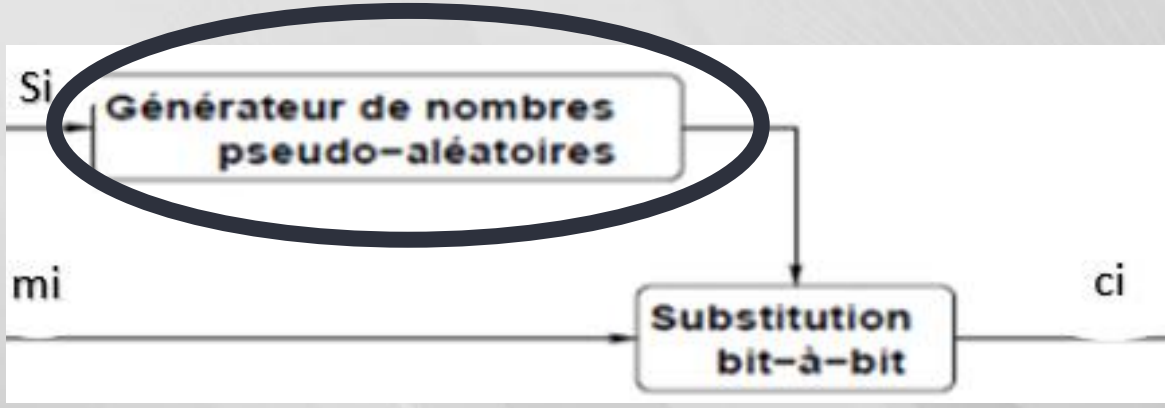


L’un des **avantages** du chiffrement symétrique est **la rapidité du chiffrement** des messages par rapport au chiffrement asymétrique. Néanmoins, l’échange des clés secrètes, qui doit se faire par un canal sécurisé, qui est souvent le point faible de ces méthodes de chiffrement.

Enfin, **le chiffrement symétrique assure uniquement la confidentialité des données** transmises ou stockées, elle ne peut pas être employée pour confirmer intégrité ni leur authenticité.

4.1.1 Chiffrement symétrique par flot

Le chiffrement à flot consiste à utiliser le chiffrement de **Vernam** (ou encore **masque jetable**) avec un générateur pseudo-aléatoire.



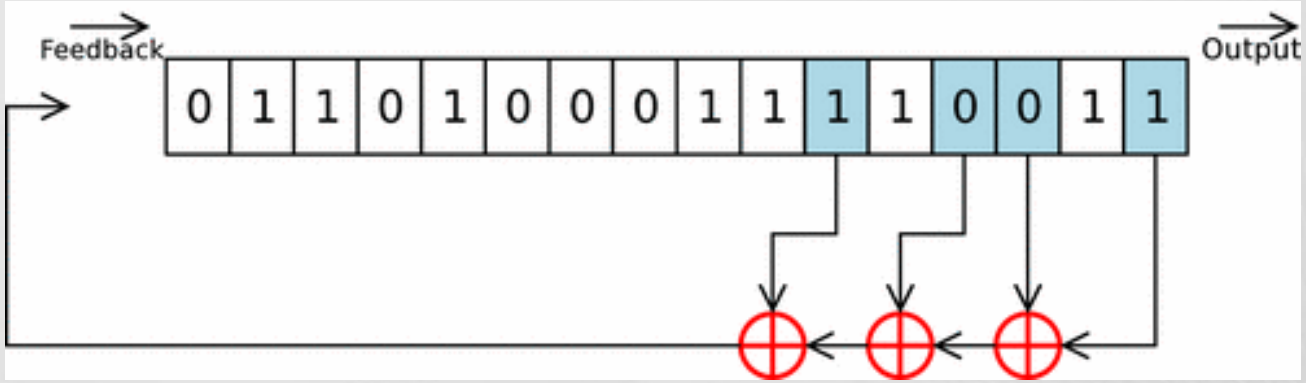
On pose:

- ***si*** **:** la séquence pseudo-aléatoire,
- ***mi*** **:** les bits du messages
- ***ci*** **:** les bits chiffrés associés
- **le chiffrement :** $ci = si \oplus mi$
- **le déchiffrement:** $mi = ci \oplus si = mi \oplus si \oplus si$.

- L'intérêt du chiffrement à flot par rapport au chiffrement par blocs est qu'il demande en général moins de ressources pour être implémenté puisqu'on a juste besoin d'un générateur pseudo-aléatoire. Par contrele chiffrement par bloc nécessite de faire un nombre de tours importants. Cela en fait un chiffrement intéressant pour des environnements contraints comme le téléphone portable où un chiffrement de ce type (le chiffrement A5/1) est utilisé pour la norme GSM.

- En général le générateur est connu, seule l’initialisation du générateur est inconnue.
- L’initialisation est le secret partager par les deux partis et la clé secrète du système.
- L’intérêt du chiffrement à flot est qu’il repose sur un générateur pseudo-aléatoire Toute la difficulté du chiffrement à flot consiste à trouver des générateurs pseudo-aléatoires qui soient à la fois sûr et rapide.
- un générateur pseudo-aléatoire doit donc être au moins plus rapide ou plus léger à implémenter que les générateurs dérivant des chiffrements par blocs.
- La famille des registres à décalage linéaire est une exemple de famille avec de bonne propriétés statistiques et très rapide. Elle ne peut cependant être utilisée telle quelle car on peut l’attaquer facilement.

Registres à décalage linéaires



Un registre à décalage de longueur L est constitué

de L cases mémoires numérotées de 0 à L-1,

chacune pouvant contenir 0 ou 1 (1 bit) et ayant une sortie et une entrée,

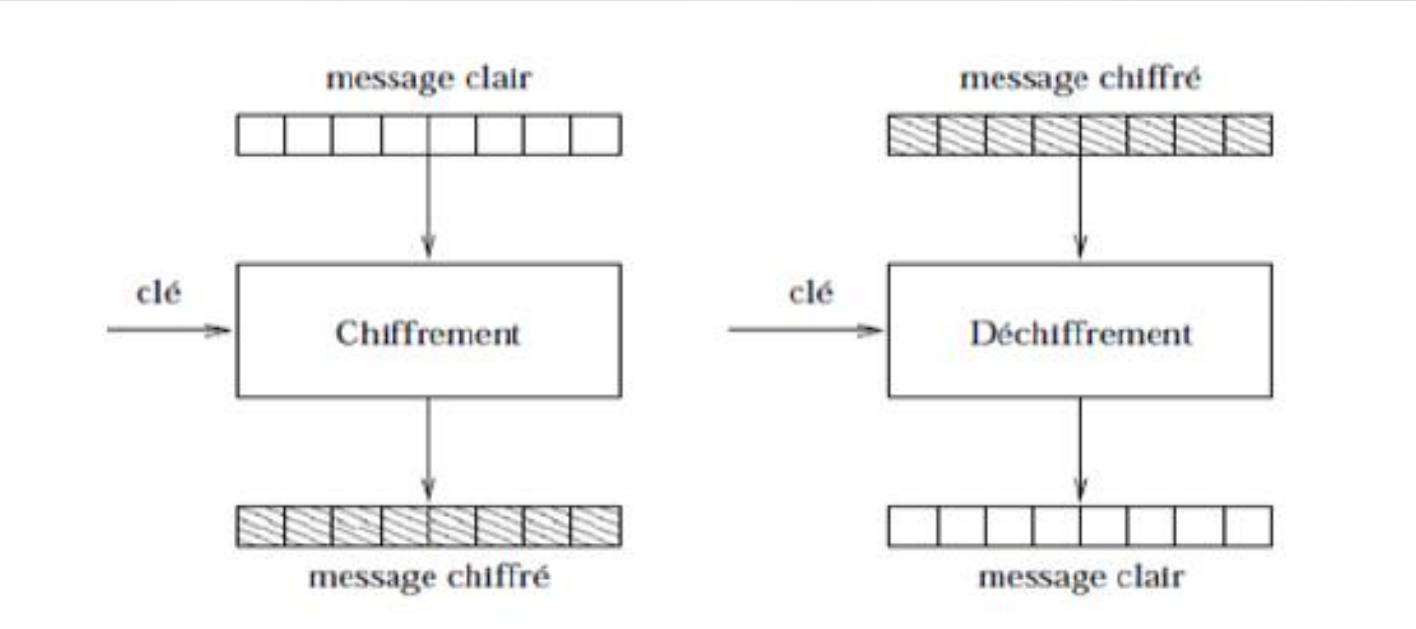
ainsi une horloge contrôlante des mouvement des données.

A chaque top d'horloge :

- un bit de rétroaction est calculé par combinaison de bits des cases 0 à L − 1.
- le contenu de la case 0 sort du registre pour former la séquence de sortie.
- le contenu de la case i passe dans la case i − 1 pour $1 \leqslant i \leqslant L - 1$.
- la case L − 1 est remplacé par le bit de rétroaction

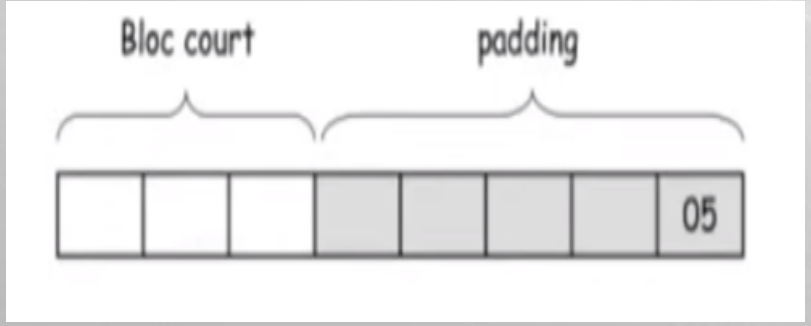
4.1.2 Chiffrement symétrique par bloc

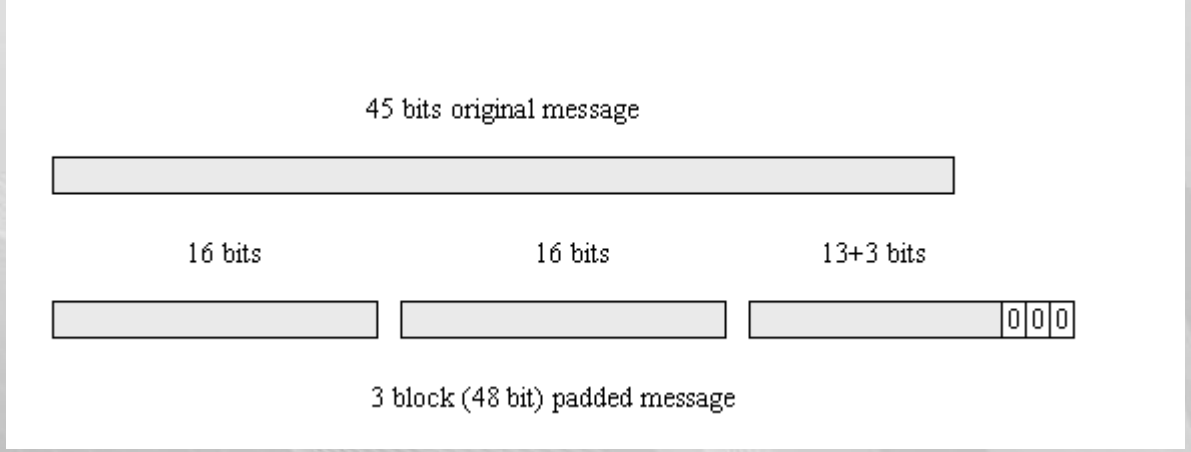
Un algorithme de chiffrement par bloc consiste à décomposer le texte clair en blocs de taille identique (64 bits, 128 bits) et chiffre les blocs les uns après les autres avec une clé secrète. Les algorithmes de chiffrement par bloc les plus connu sont le DES et l'AES.



1. Les codes par blocs

Le chiffrement par blocs fonctionne différemment. Au lieu de prendre chaque bit un par un, les **messages sont découpés en blocs** (**la taille des blocs dépend de la clé**). Ensuite, chaque bloc est additionné à la clé et un traitement de type permutation, opération XOR ou autre est appliqué à chaque bloc.





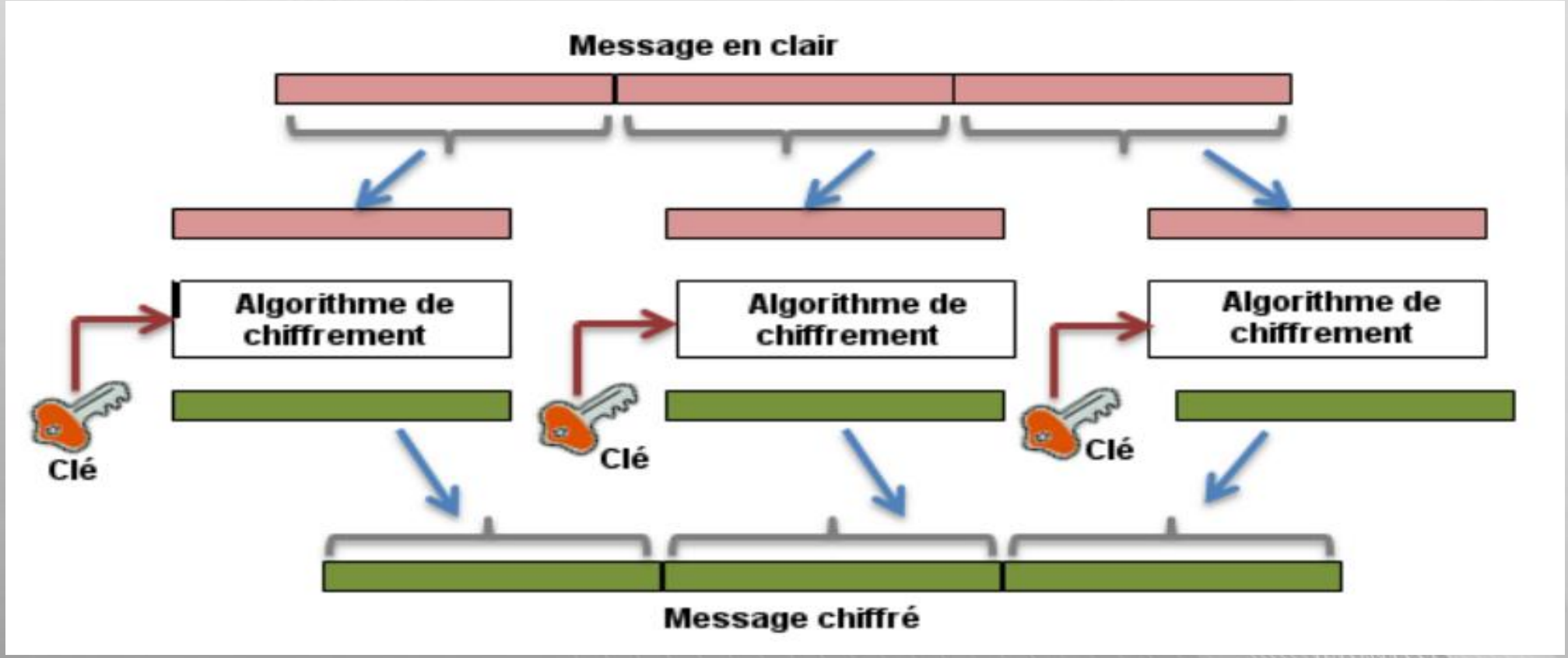
Condition :

La clé doit être suffisamment grande pour un bon algorithme.

Il existe plusieurs modes de chiffrement par bloc :

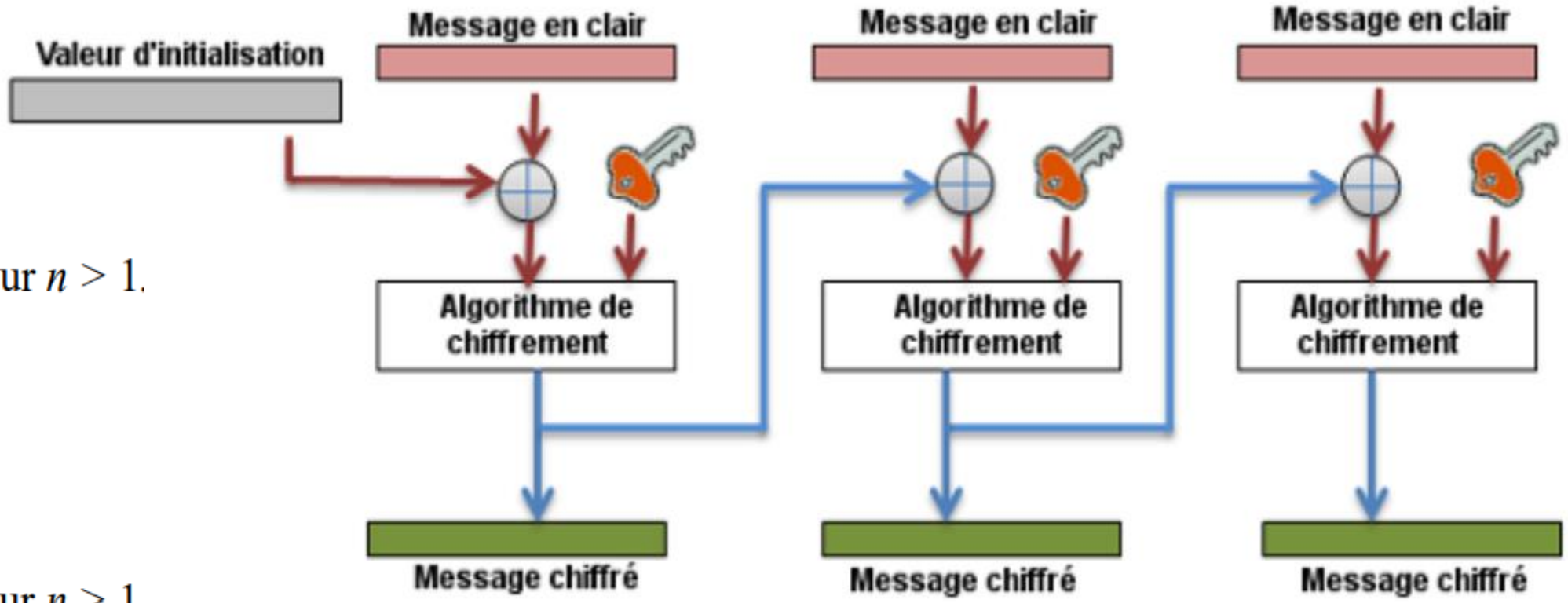
1.1. Le mode Dictionnaire (Electronic Codebook) : ECB

Dans ce mode, le message M est découpé en blocs Mi de taille fixe. Chaque bloc est alors chiffré séparément par un algorithme de chiffrement, paramétrée par une clé ki . Ainsi un bloc de message donné sera toujours codé de la même manière. Ce mode de chiffrement est le plus simple mais il est très vulnérable aux attaques.



1.2. Le mode Enchaînement des blocs (Cipher Block Chaining) : CBC

Pour remédier aux risques de sécurité constatés dans le mode ECB, le mode CBC a été introduit pour qu'un bloc ne soit pas codé de la même manière. Où le premier bloc doit être modifier par un XOR avec une valeur initiale aléatoire ajouter au début de l'opération. Puis,Chaque bloc est d'abord modifié par XOR avec le bloc chiffré précédemment avant d'être lui-même chiffré. Le mode CBC est le mode de chiffrement le plus utilisé.



$C_1 = E\,(M_1 \oplus VI)$

$C_n = E\,(M_n \oplus C_{n-1})$, pour $n > 1$.

Le déchiffrement:

$M_1 = D\,(C_1) \oplus VI$

$M_N = D\,(C_N) \oplus C_{n-1}$, pour $n > 1$.

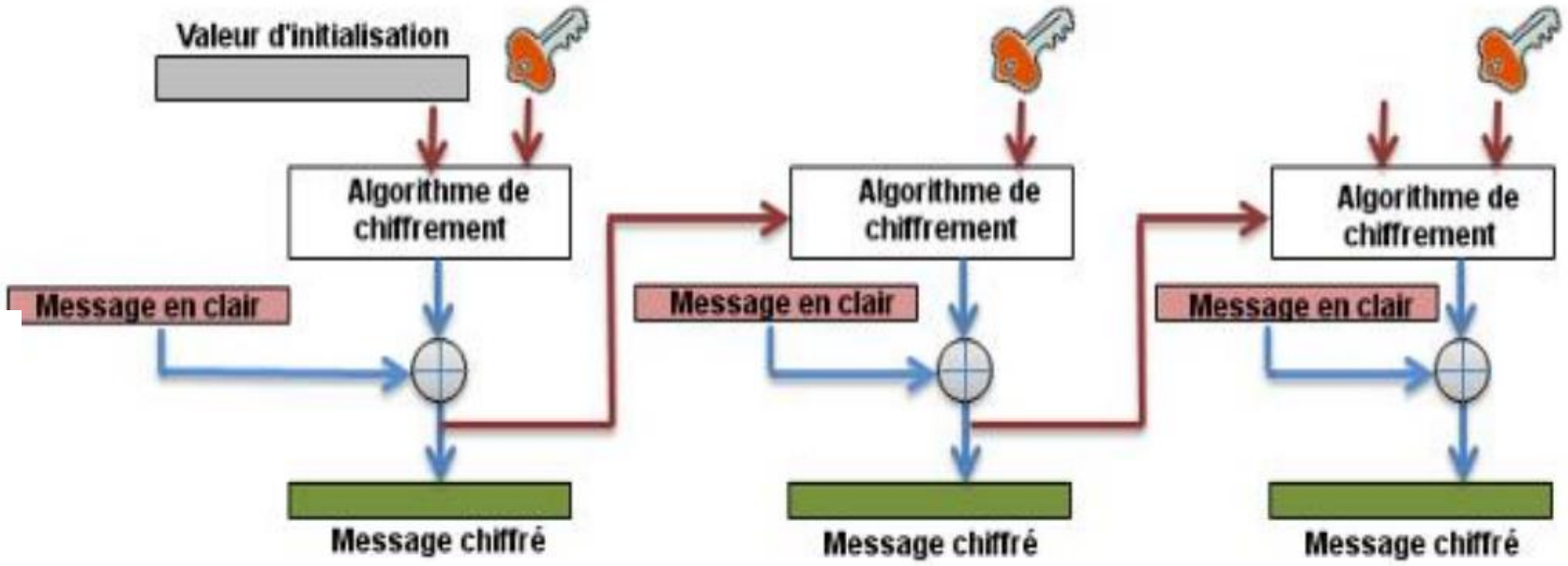
1.3. Le mode Chiffrement à rétroaction (Cipher Feedback): CFB

Chaque chiffrement de bloc dépend du résultat du bloc précédent en manipulant un registre de décalage de la taille d'un bloc de texte en clair. Ce registre de décalage est rempli **par un vecteur d'initialisation** et chiffré avec l'algorithme de chiffrement.
Ce mode est donc moins sûr que le CBC.

$I_1 = VI$

$I_n = (I_{n-1} \ll k) \mid C_{n-1}$, pour $n>1$

$C_n = M_n \oplus MSBk\,(E(I_n))$, pour $n \geq 1$. Tel que MSBk sont les k bits les plus significatifs



Et pour le déchiffrement :

$I_1 = VI$

$I_n = (I_{n-1} \ll k) \mid C_{n-1}$, pour $n>1$

$M_n = C_n \oplus MSBk\,(E(I_n))$, pour $n \geq 1$.

1.4. Le mode Chiffrement à rétroaction de sortie (Output Feedback): OFB

Il manipule également un registre de décalage qui est chiffré par un algorithme de chiffrement puis modifié par un XOR avec le message en clair dont le résultat sera utilisé pour le chiffrement du bloc suivant.

• Pour chiffrer un texte en mode OFB à k bits, on effectue les opérations suivantes :

$I_1 = VI$

$I_n = (I_{n-1} \ll k) \mid MSBk\,(E\,(I_{n-1}))$, pour $n > 1$

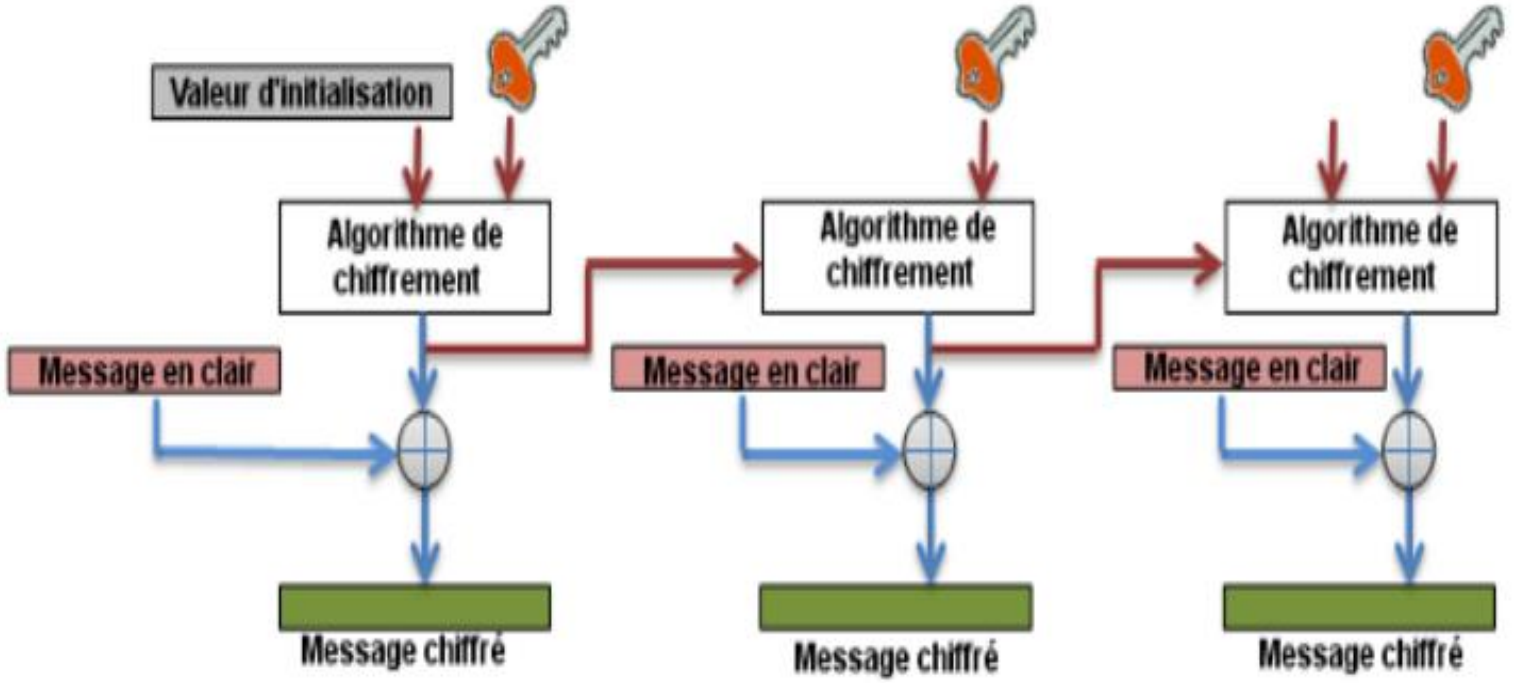
$C_n = M_n \oplus MSBk\,(E(I_n))$, pour $n \geq 1$.

• Pour déchiffrer :

$I_1 = VI$

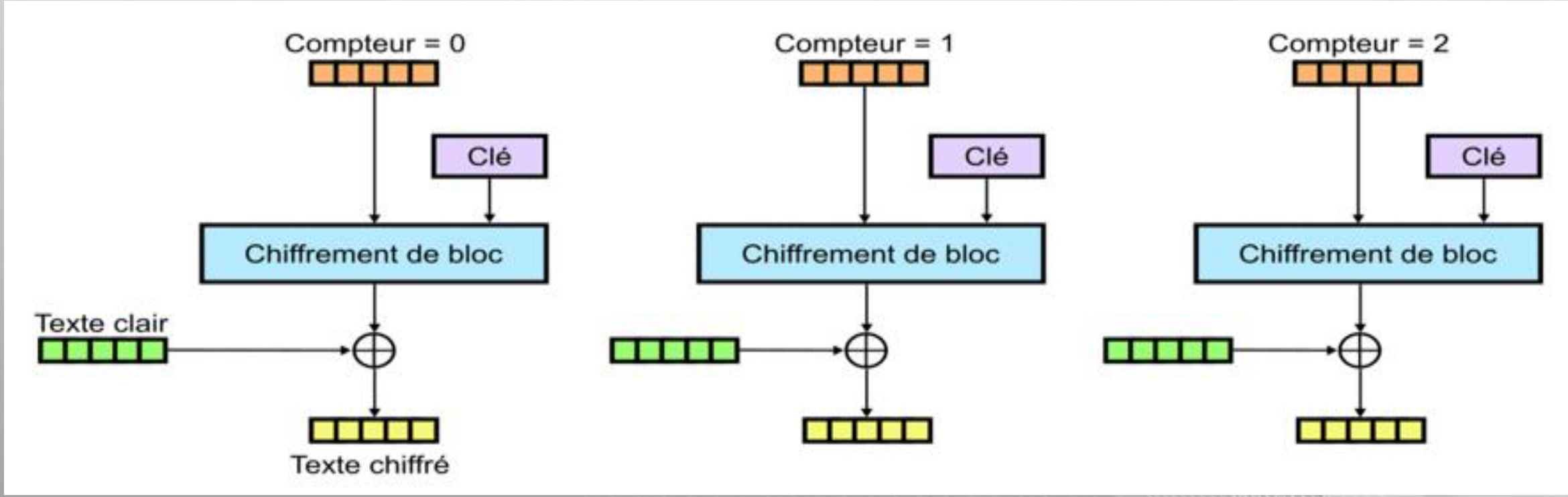
$I_n = (I_{n-1} \ll k) \mid MSBk\,(E\,(I_{n-1}))$, pour $n > 1$

$C_n = M_n \oplus MSBk\,(E(I_n))$, pour $n \geq 1$.



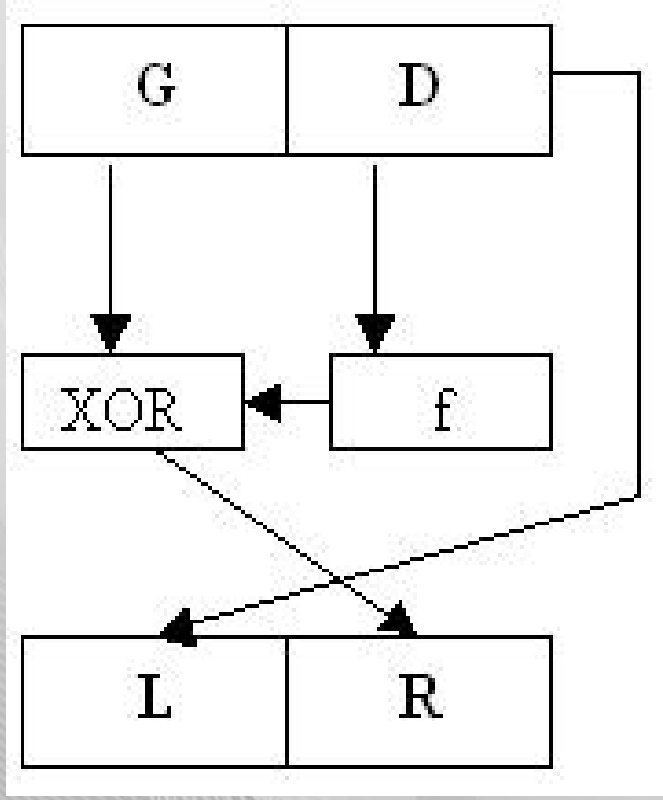
1.5. Le mode Chiffrement basé sur un compteur(Counter): CTR

Ce mode imite le fonctionnement d'un chiffrement de flux, en utilisant un **compteur** qui s'incrémente à chaque bloc. Le compteur et la clé sont chiffrés avec la fonction de chiffrement de bloc, ce qui donne un flux pseudo-aléatoire découpé en blocs. Puis une opération XOR est appliqué entre le bloc du flux pseudo-aléatoire et le bloc en clair, ce qui donne le bloc chiffré.



2. Chiffrement de Feistel

Chiffrement de Feistel ; C'est la base pratiquement de plusieurs algorithmes modernes à clé secrète (en particulier DES), il est proposé par Horst Feistel (IBM) en 1973**. Il est basé sur un chiffrement itératif par blocs opérant sur des blocs de 2n bits.**



Principe de chiffrement :

Un bloc de texte en clair est découpé en deux, puis une transformation de ronde (fonction f) est appliquée pour la partie droite, et le résultat est combiné avec la partie gauche par le ou-exclusif. Les deux moitiés sont alors inversées pour l'application de la ronde suivante. Le déchiffrement est structurellement identique au chiffrement.

2.1. L'algorithme DES

DES est l’algorithme de chiffrement moderne le plus populaire qui est apparu en 1970 et son premier standard est publié en 1977.

L'algorithme **DES transforme un bloc de 64 bits en un autre bloc de 64 bits**. Il manipule des **clés individuelles de 56 bits**, représentées par 64 bits (**les bits 8, 16, 24, 32, 40, 48, 56 ne sont pas utilisées pour la clé mais participent à un code correcteur qui permet de vérifier que la clé n’a pas été altérée et** le contrôle de parité). Ce système de chiffrement symétrique fait partie de la famille des chiffrements itératifs par blocs, plus particulièrement il s'agit d'un schéma de Feistel.

on peut dire que DES fonctionne en trois étapes :

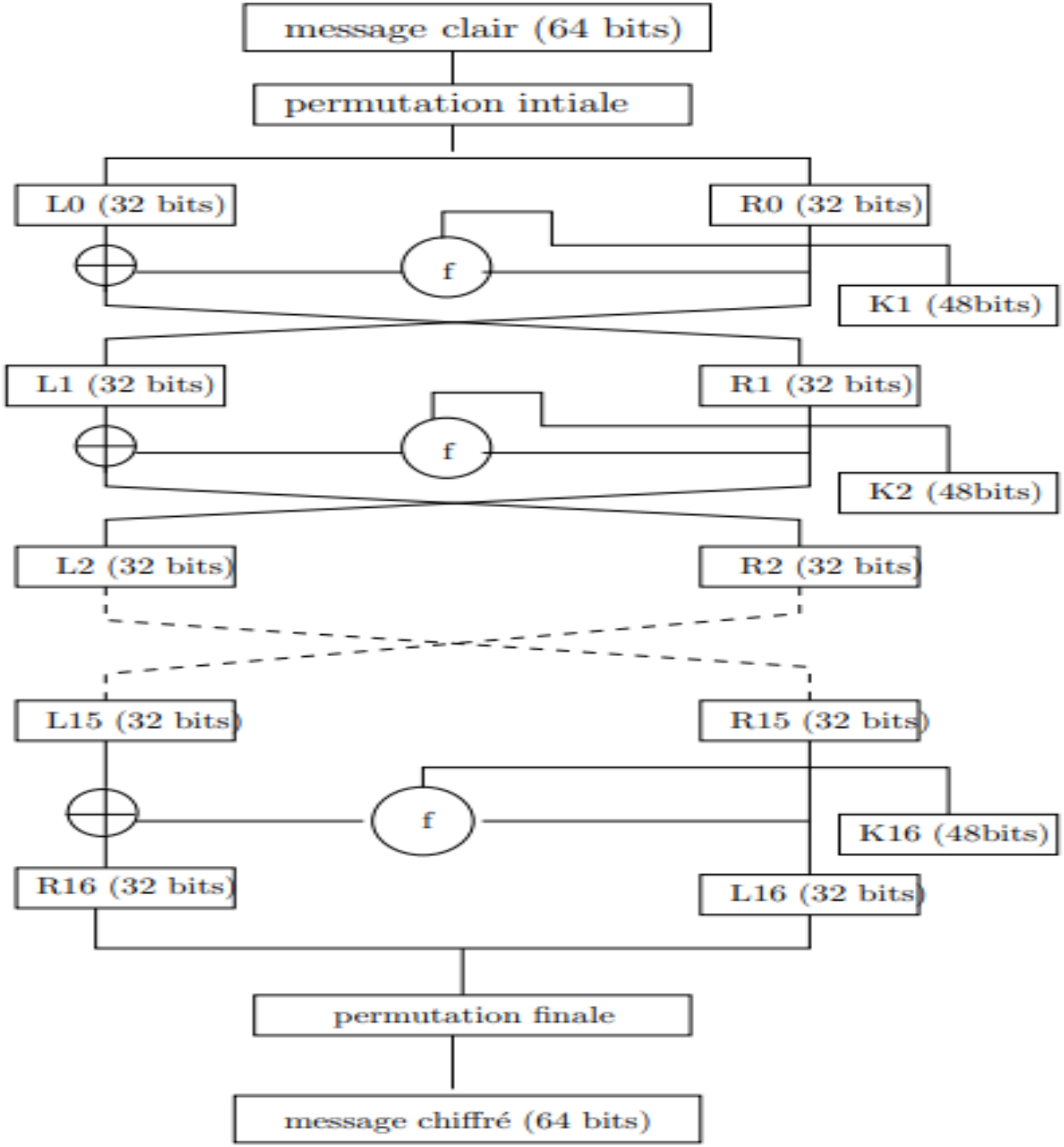
Etape 1:
Permutation initiale et fixe d'un bloc (sans aucune incidence sur le niveau de sécurité)

Etape 2:

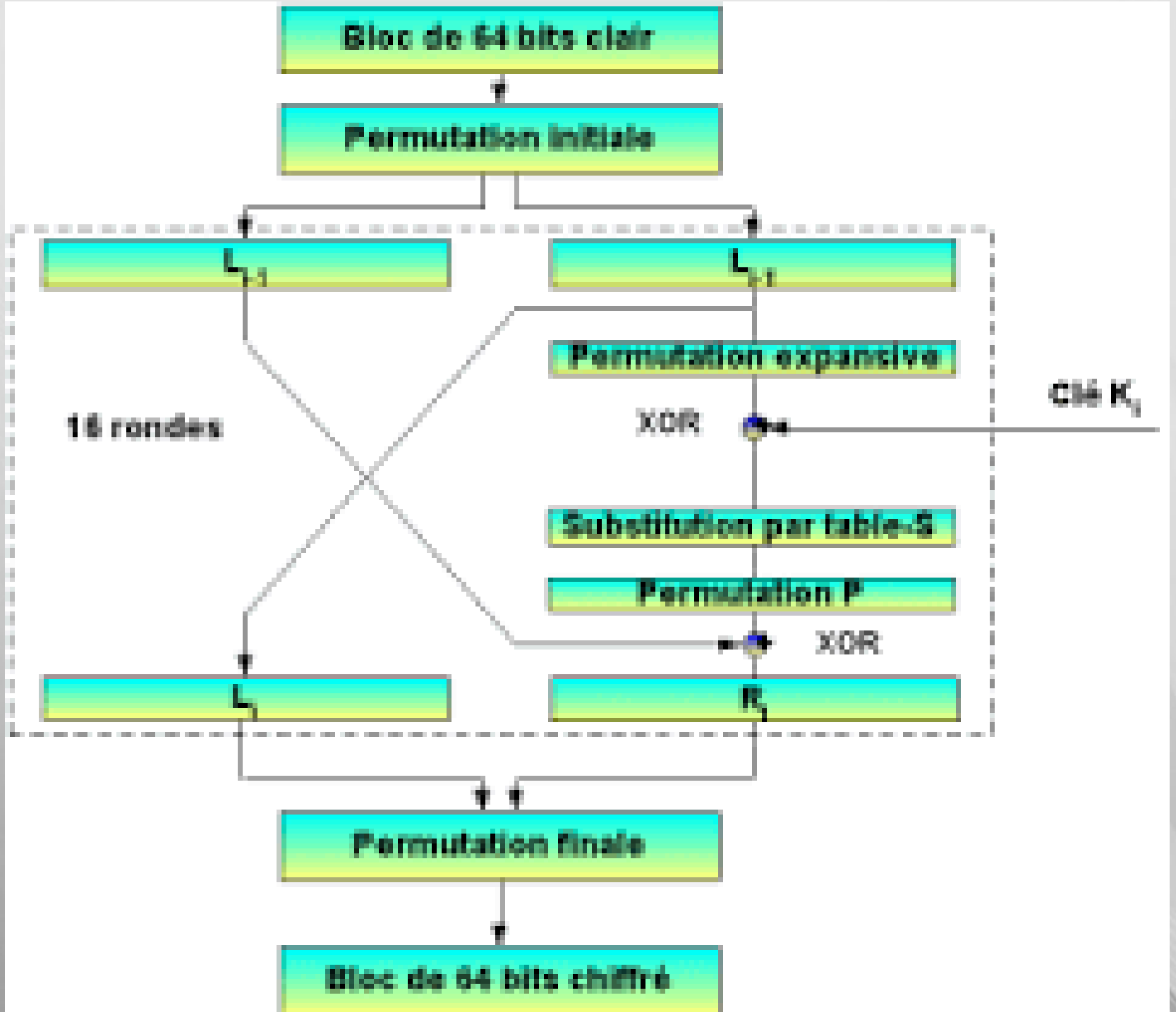
Le résultat est soumis à 16 itérations d'une transformation, ces itérations dépendent à chaque tour d'une autre clé partielle de 48 bits. Cette clé de tour intermédiaire est calculée à partir de la clé initiale de l'utilisateur (grâce à un réseau de tables de substitution et d'opérateurs XOR).
Lors de chaque tour, le bloc de 64 bits est découpé en deux blocs de 32 bits, et ces blocs sont échangés l'un avec l'autre selon un schéma de Feistel.
Le bloc de 32 bits ayant le poids le plus fort (celui qui s'étend du bit 32 au bit 64) subira une transformation.

Etape 3:

Le résultat du dernier tour est transformé par la fonction inverse de la permutation initiale.



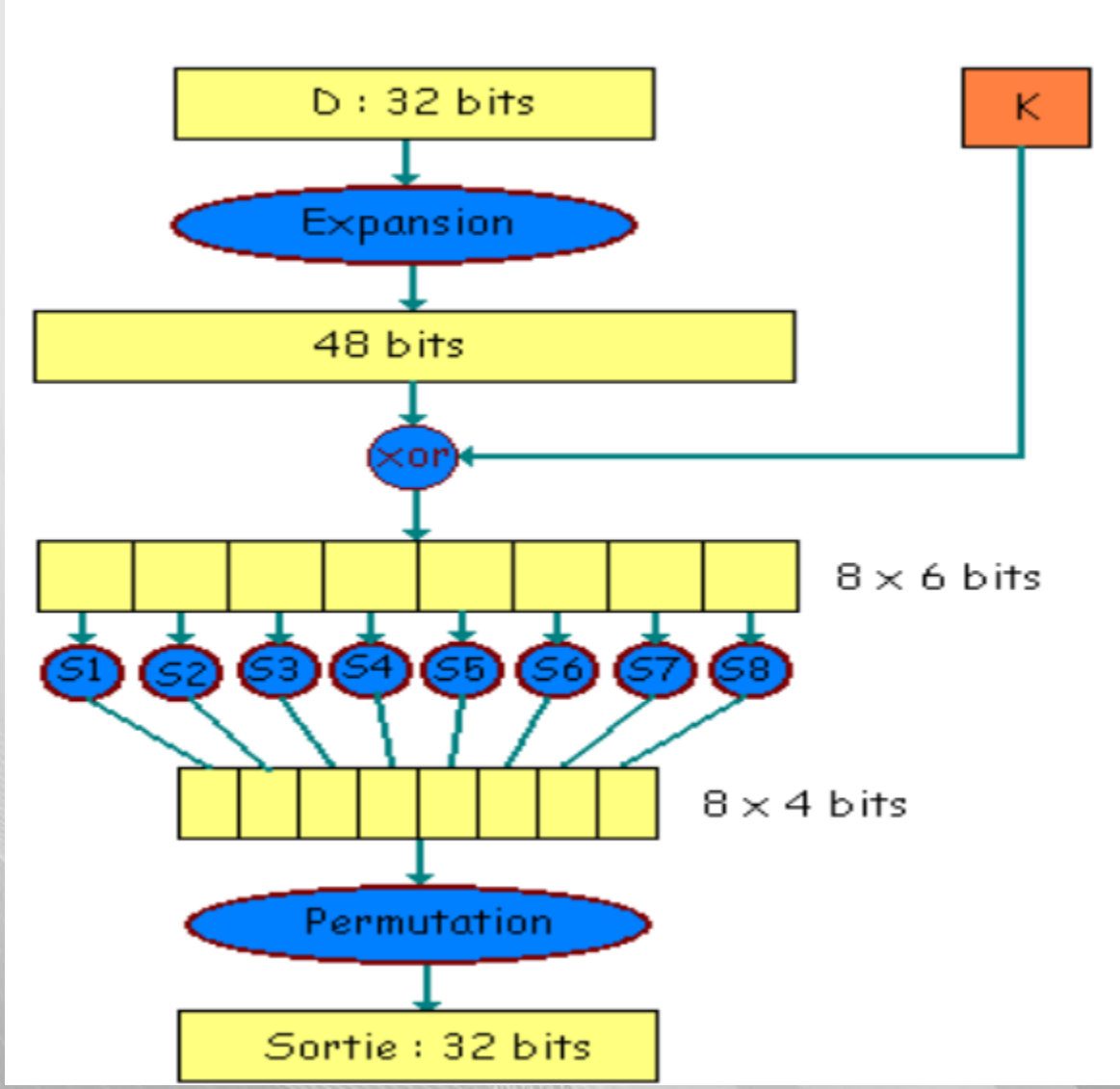
	Matrice PI						
58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7


Bloc de 64 bits clair
Permutation initiale
L_{i-1}
L_{i-1}
Permutation expansive
16 rondes
XOR
Clé K_i
Substitution par table-S
Permutation P
XOR
L_i
R_i
Permutation finale
Bloc de 64 bits chiffré

Les blocs Gn et Dn sont soumis à un ensemble de transformations itératives **appelés rondes ou itérations appelées aussi tours**

Les 32 bits du bloc D sont étendus à 48 bits grâce à une table (matrice) appelé table d'expansion (notée E), **dans laquelle les 32 bits sont mélangés et 16 d'entre eux sont dupliqués.**

E						
	32	1	2	3	4	5
	4	5	6	7	8	9
	8	9	10	11	12	13
	12	13	14	15	16	17
	16	17	18	19	20	21
	20	21	22	23	24	25
	24	25	26	27	28	29
	28	29	30	31	32	1



La matrice Sbox: boîtes de substitution ou fonctions de compression

S_2

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
1	3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5
2	0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
3	13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9

S_3

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8
1	13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1
2	13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7
3	1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12

S_4

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15
1	13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9
2	10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4
3	3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14

S_5

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9
1	14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6
2	4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
3	11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3

S_6

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11
1	10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
2	9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6
3	4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13

S_7

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1
1	13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6
2	1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2
3	6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12

S_8

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
1	1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
1	7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
1	2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11

Le bloc de 32 bits obtenu est enfin soumis à une permutation **P**

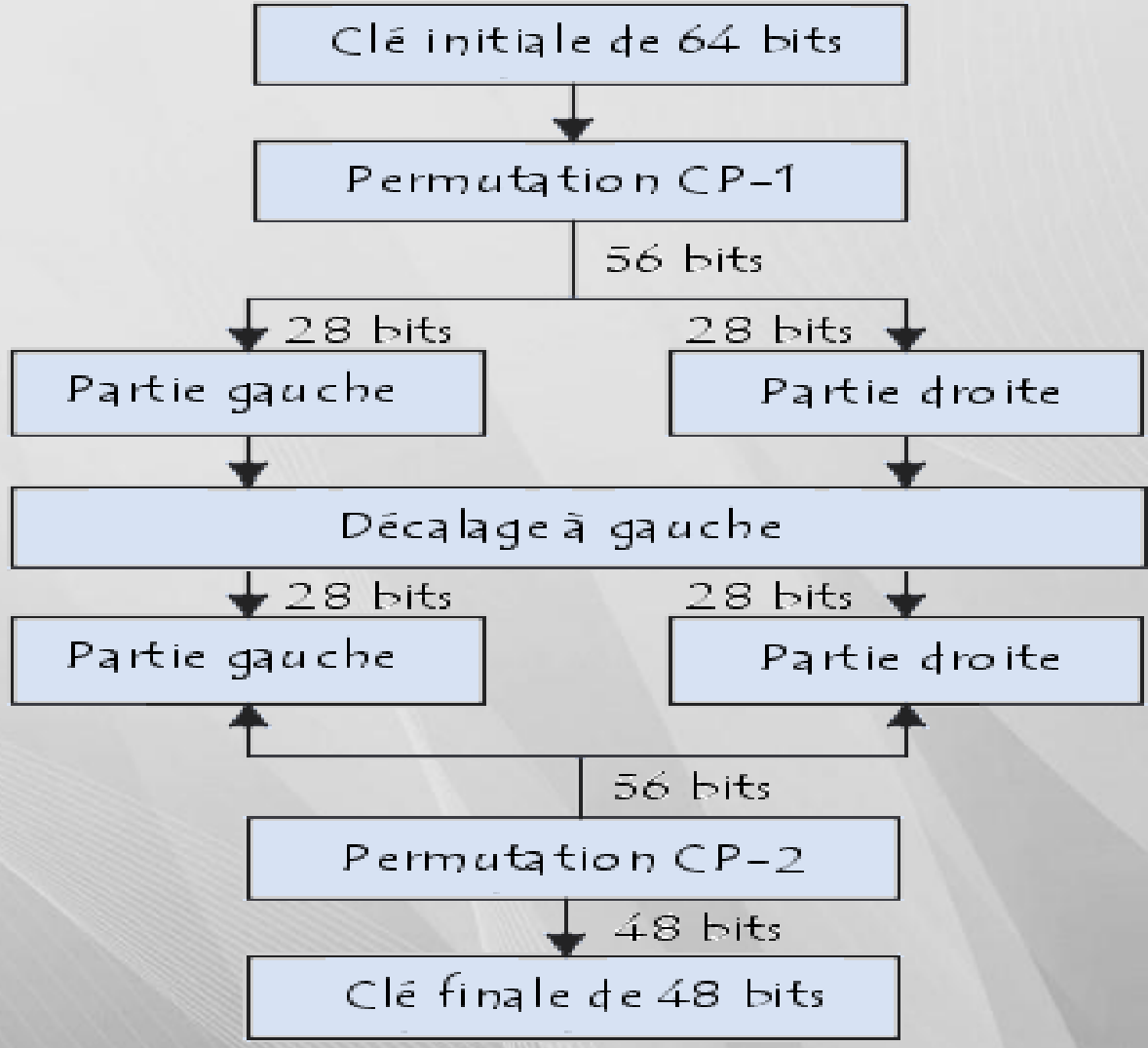
P	16	7	20	21	29	12	28	17
	1	15	23	26	5	18	31	10
	2	8	24	14	32	27	3	9
	19	13	30	6	22	11	4	25

Construction des sous-clés:

Toute la sécurité de l'algorithme DES repose sur la complexité des clés de chiffrement.

Dans un premier temps les bits de parité de la clé sont éliminés afin d'obtenir une clé d'une longueur utile de 56 bits.

comment obtenir à partir d'une clé de 64 bits 8 clés diversifiées de 48 bits chacune servant dans l'algorithme du DES ?

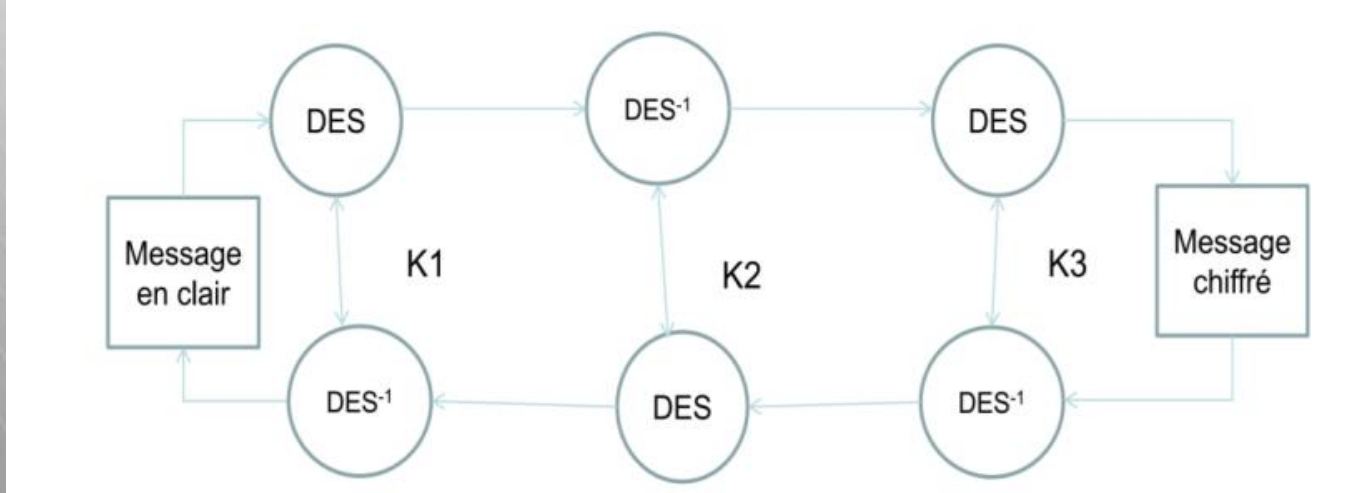


CP-1	57	49	41	33	25	17	9	1	58	50	42	34	26	18
	10	2	59	51	43	35	27	19	11	3	60	52	44	36
	63	55	47	39	31	23	15	7	62	54	46	38	30	22
	14	6	61	53	45	37	29	21	13	5	28	20	12	4

CP-2	14	17	11	24	1	5	3	28	15	6	21	10
	23	19	12	4	26	8	16	7	27	20	13	2
	41	52	31	37	47	55	30	40	51	45	33	48
	44	49	39	56	34	53	46	42	50	36	29	32

Afin de prolonger la durée de vie de DES en attendant AES, le DES est remplacé par le 3DES qui utilise deux clés de 56 bits et effectue le chiffrement en trois phases:

- La première phase consiste à ***chiffrer*** les données avec la première clé ***K1*** ,
- la seconde phase ***déchiffre*** avec la seconde clé ***K2 ,***
- la troisième phase ***chiffre*** avec la première clé ***K3***.



2.2. L'algorithme AES (Rijndael)

L'AES est un système de chiffrement symétrique largement utilisé. Ce système est recommandé par plusieurs organisations de sécurité comme standard du chiffrement symétrique. En outre, AES est considéré parmi les **systèmes cryptographiques post-quantiques**.

L'AES opère sur des blocs de 128 bits (16 octets), il les transforme en blocs chiffrés de 128 bits au moyen d'une clé de taille 128 bits (16 octets), 192 bits (24 octets) ou 256 bits (32 octets).

	$\|k\| = 128$	$\|k\| = 192$	$\|k\| = 256$
blocs 128	10	12	14
blocs 192	12	12	14
blocs 256	14	14	14

L'état ou le bloc en clair est représenté par une matrice. **Le nombre de ligne de cette matrice égale toujours à 4** mais le nombre de colonne égale se change selon la taille du bloc **: 4 colonnes correspond à un bloc de 128 bits, 6 colonnes à un bloc de 192 bits et 8 colonnes à un bloc de 256 bits.**

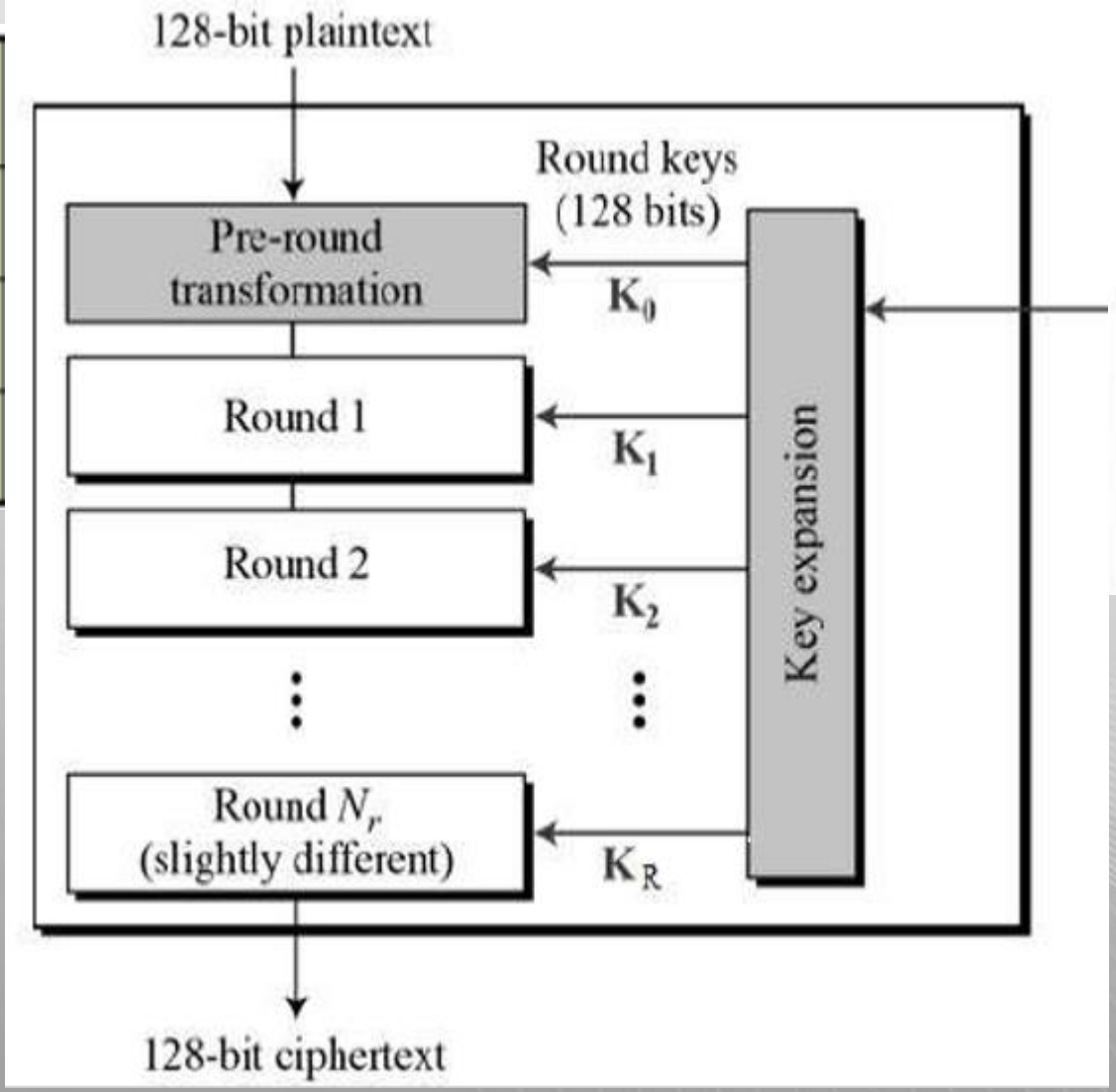
$a_{0,0}$	$a_{0,1}$	$a_{0,2}$	$a_{0,3}$	$a_{0,4}$	$a_{0,5}$	$a_{0,6}$	$a_{0,7}$
$a_{1,0}$	$a_{1,1}$	$a_{1,2}$	$a_{1,3}$	$a_{1,4}$	$a_{1,5}$	$a_{1,6}$	$a_{1,7}$
$a_{2,0}$	$a_{2,1}$	$a_{2,2}$	$a_{2,3}$	$a_{2,4}$	$a_{2,5}$	$a_{2,6}$	$a_{2,7}$
$a_{3,0}$	$a_{3,1}$	$a_{3,2}$	$a_{3,3}$	$a_{3,4}$	$a_{3,5}$	$a_{3,6}$	$a_{3,7}$

$k_{0,0}$	$k_{0,1}$	$k_{0,2}$	$k_{0,3}$	$k_{0,4}$	$k_{0,5}$	$k_{0,6}$	$k_{0,7}$
$k_{1,0}$	$k_{1,1}$	$k_{1,2}$	$k_{1,3}$	$k_{1,4}$	$k_{1,5}$	$k_{1,6}$	$k_{1,7}$
$k_{2,0}$	$k_{2,1}$	$k_{2,2}$	$k_{2,3}$	$k_{2,4}$	$k_{2,5}$	$k_{2,6}$	$k_{2,7}$
$k_{3,0}$	$k_{3,1}$	$k_{3,2}$	$k_{3,3}$	$k_{3,4}$	$k_{3,5}$	$k_{3,6}$	$k_{3,7}$

- Il repose sur le réseau de substitution-permutation

AES traite les 128 bits de texte en clair ou de bloc de texte original comme **16 octets** de blocs. Ces 16 octets de texte en clair sont disposés en quatre colonnes et quatre lignes pour être traités comme une matrice de 4X4

$a_{0,0}$	$a_{0,1}$	$a_{0,2}$	$a_{0,3}$
$a_{1,0}$	$a_{1,1}$	$a_{1,2}$	$a_{1,3}$
$a_{2,0}$	$a_{2,1}$	$a_{2,2}$	$a_{2,3}$
$a_{3,0}$	$a_{3,1}$	$a_{3,2}$	$a_{3,3}$



Cipher key
(128, 192, or 256 bits)

R	Key size
10	128
12	192
14	256

Dans le reste de la description, nous supposons que la taille du bloc et la taille de la clé sont égalent à **128 (16 octets) donc le chiffrement du bloc nécessite 10 rondes et 10 sous-clés**

1. **La matrice en entrée est d'abord modifiée par un ou exclusif avec la matrice principale de la clé**.

2. Les 16 octets en sortie vont passer par quatre transformations de ronde :

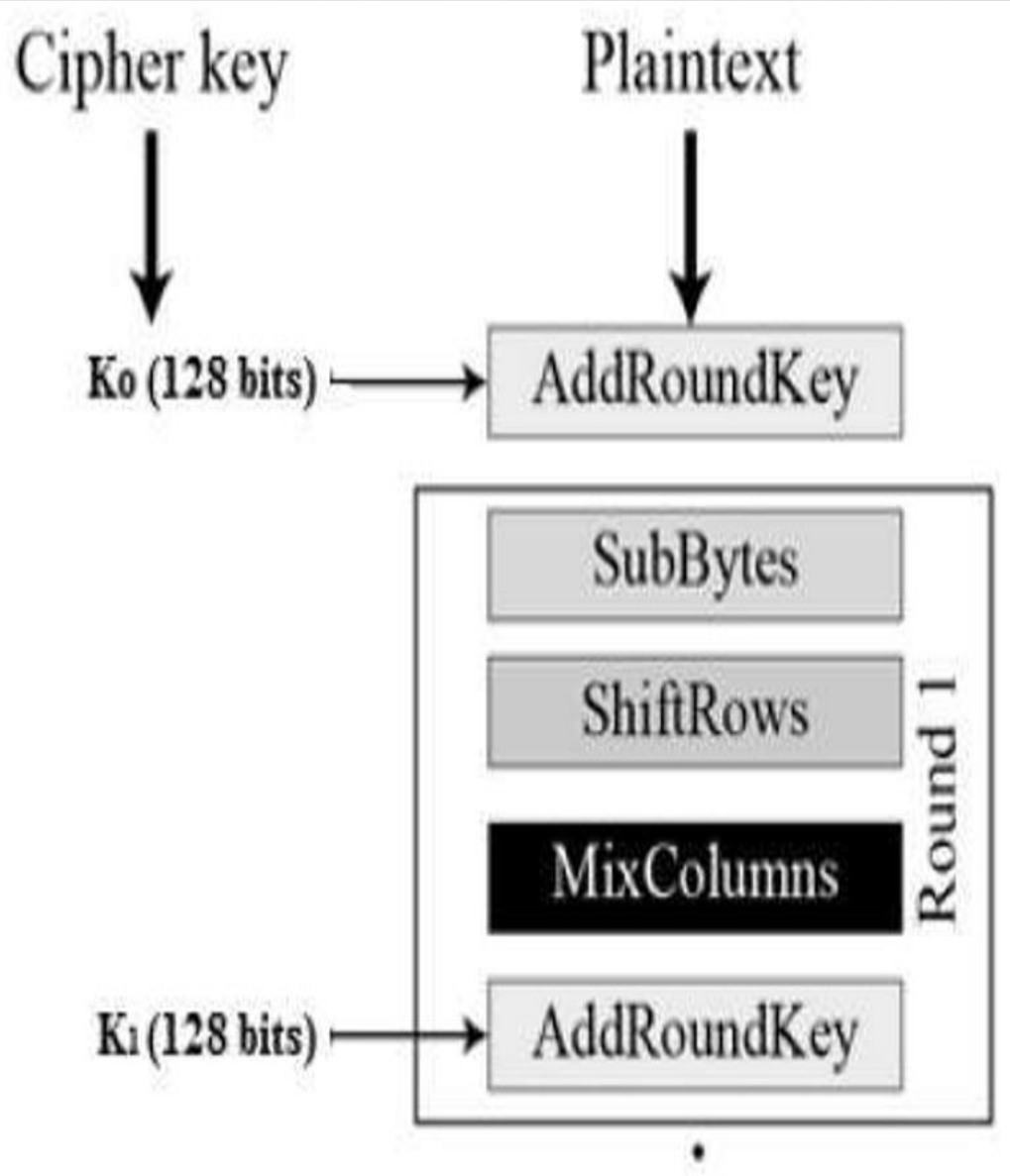
   1. SubBytes (substitution): passage dans une S-box

   2. ShiftRows (rotation): décalage des lignes.

   3. MixColumns (mélange): mélange des colonnes (sauf dernier tour)

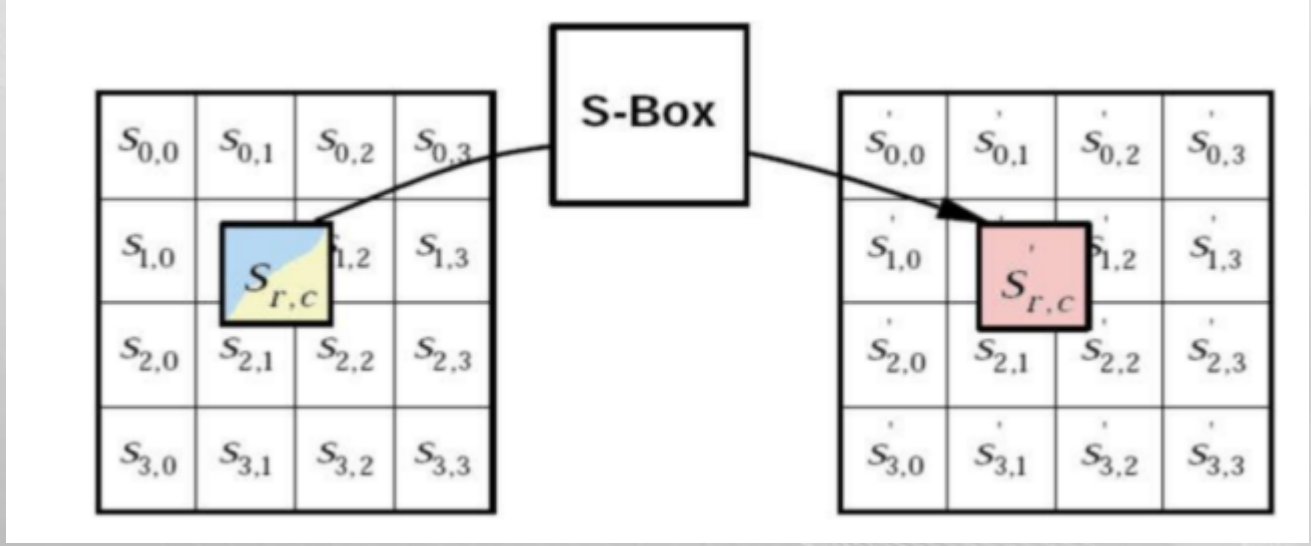
   4. AddRoundKey (Ajout de la sous-clé): XOR avec la sous-clé de ronde.

La dernière ronde (dans ce cas la dixième ronde) ne subit pas la transformation de mélange (MixColumns).



Transformation SubBytes():

Il s’agit d’une transformation non linéaire appliquée indépendamment à chacun des octets de l’état en utilisant une table de substitution (Sbox).



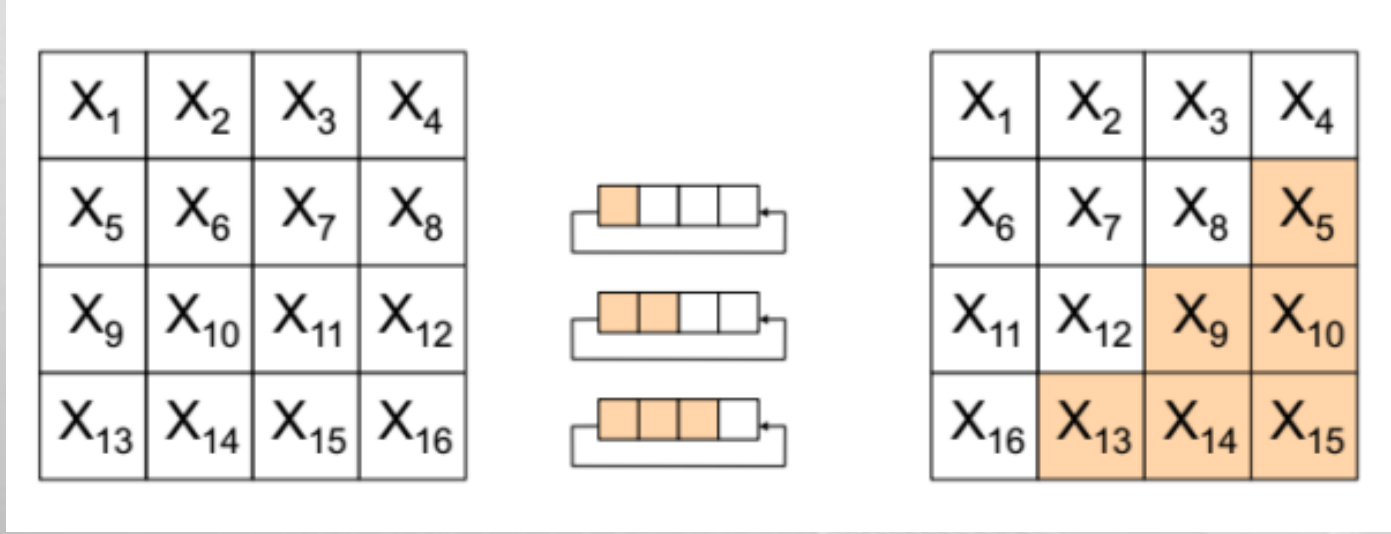
	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
e	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

pour s1,1 = {53}
s'1,1 = SubBytes(s1,1) = {ed}

	0	1	2	3
0	63	7c	77	7b
1	ca	82	c9	7d
2	b7	fd	93	26
3	04	c7	23	c3
4	09	83	2c	1a
5	53	d1	00	ed

Transformation ShiftRows():

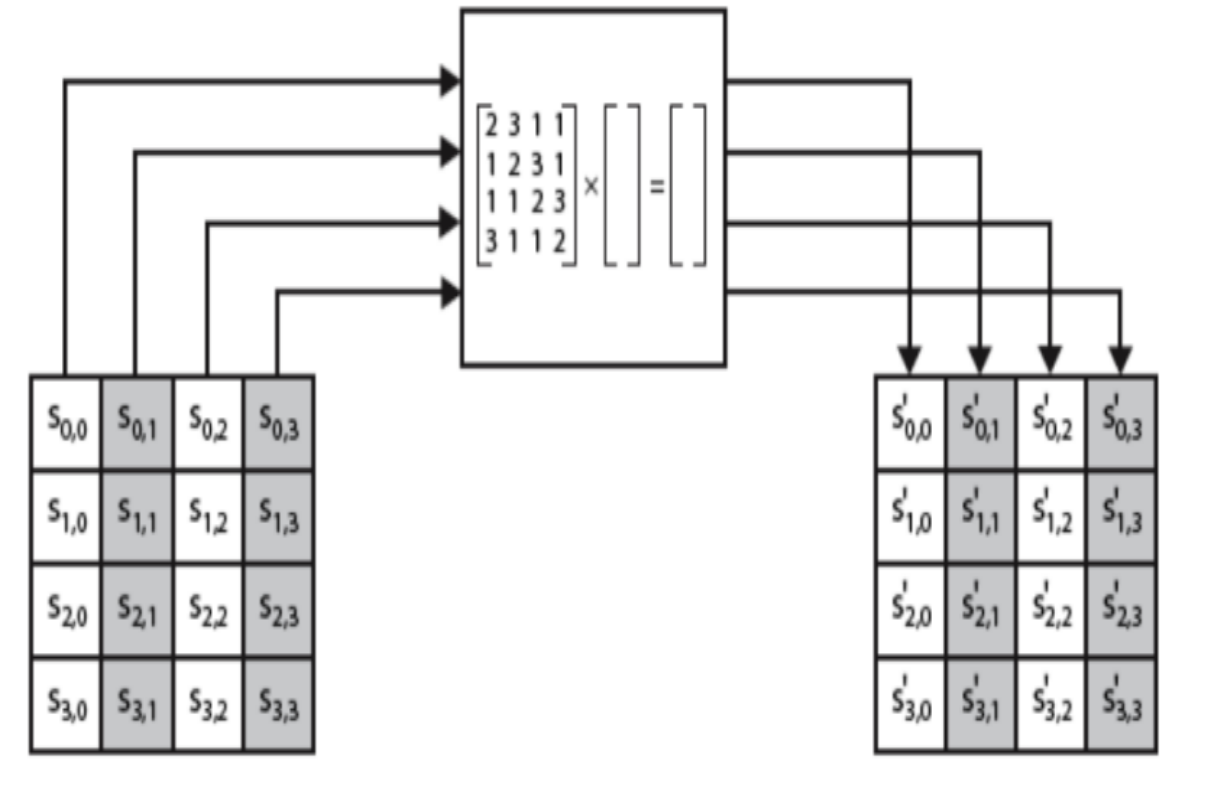
C’est une transformation linéaire. Elle correspond à une permutation cyclique des octets sur les lignes de l’état. Le décalage des octets correspond à l’indice **i** de la ligne considérée (***0<=i< 4***)



- La première ligne n'est pas décalée pour être écrite telle quelle.
- La deuxième ligne est décalée **d'un octet vers le côté gauche**.
- La troisième rangée est décalée de **deux positions vers la gauche**.
- La quatrième rangée est décalée **de trois positions vers la gauche**.

Transformation MixColumns() :

C'est une transformation linéaire,Cette transformation est appliquée à un état colonne après colonne. Il est réalisé par un produit matriciel utilisant les 4 octets d'une colonne. Les colonnes sont traitées comme des polynômes GF $(\mathbf{2^8})$et multipliées modulo $\boldsymbol{x^4} + \mathbf{1}$ avec les polynômes fixes donnés figure suivante :



$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix}$$

$$s'_{0,c} = (\{02\} \bullet s_{0,c}) \oplus (\{03\} \bullet s_{1,c}) \oplus s_{2,c} \oplus s_{3,c}$$

$$s'_{1,c} = s_{0,c} \oplus (\{02\} \bullet s_{1,c}) \oplus (\{03\} \bullet s_{2,c}) \oplus s_{3,c}$$

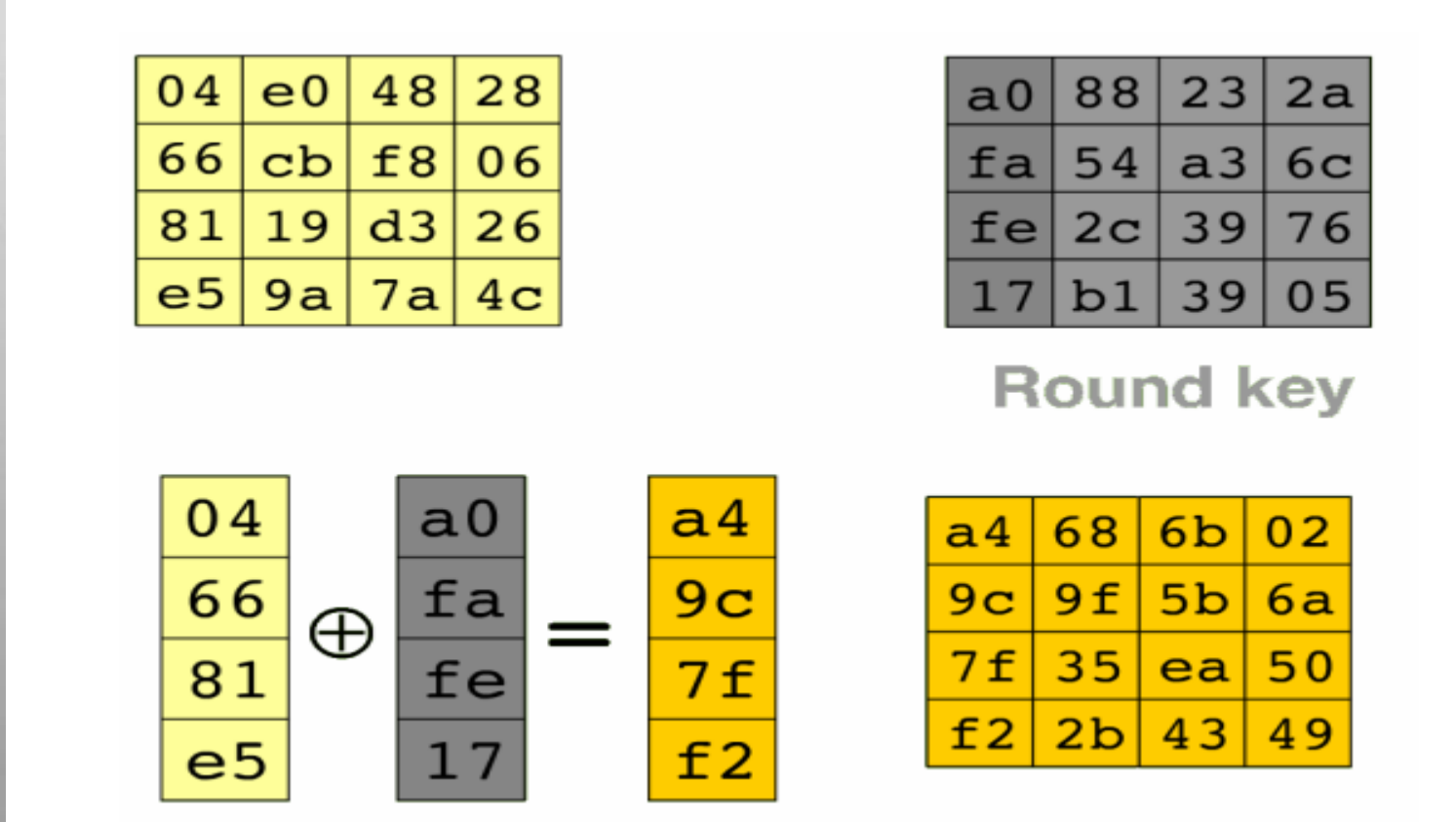
$$s'_{2,c} = s_{0,c} \oplus s_{1,c} \oplus (\{02\} \bullet s_{2,c}) \oplus (\{03\} \bullet s_{3,c})$$

$$s'_{3,c} = (\{03\} \bullet s_{0,c}) \oplus s_{1,c} \oplus s_{2,c} \oplus (\{02\} \bullet s_{3,c})$$

Transformation AddRoundKey():

C’est l’ajout de la clé de ronde (ou de la clé lors de la ronde initiale) à l’état considéré (l’addition est un ou exclusif).

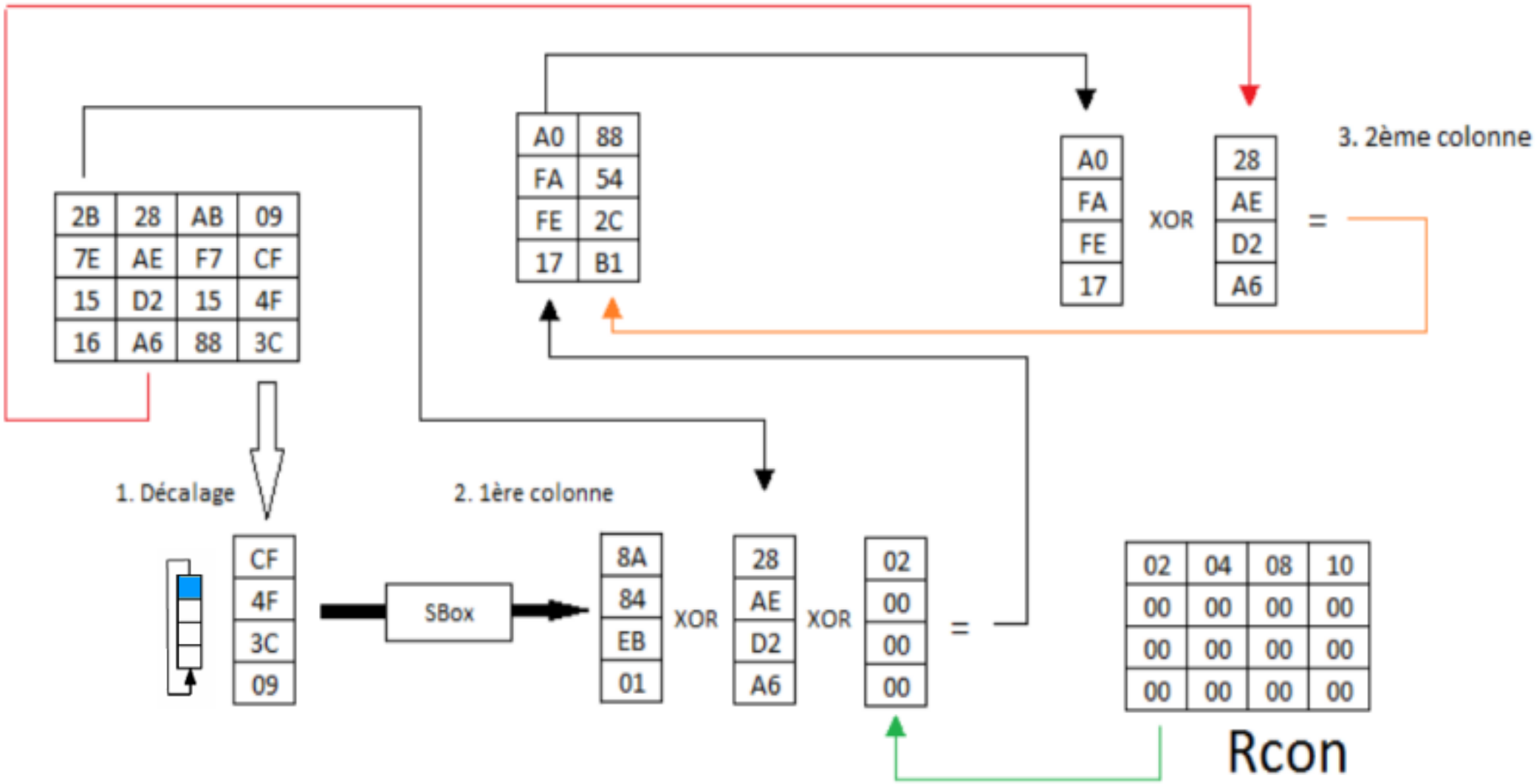
Un XOR (au niveau bit) est appliqué entre chacun des octets de l’état et de la clé de ronde.



Construction des sous-clés

La clé doit être **modifiée avant de passer à l'étape de la transformation AddRoundKey**.

- La première colonne de la clé une fois modifiée va correspondre à la dernière colonne, décalée du bas vers le haut.
- Ensuite, elle va être modifiée avec la S-BOX,
- une opération XOR va être réalisée entre le résultat obtenu, la première colonne de la clé d'origine et la colonne x (x correspond au numéro De la ronde) de la matrice Rcon (tableau fixe de constantes de tours).
- les trois autres colonnes restantes seront des opérations XOR entre la colonne de la clé d'origine et la dernière colonne ajoutée à la nouvelle clé.


2B 28 AB 09
7E AE F7 CF
15 D2 15 4F
16 A6 88 3C
1. Décalage
CF
4F
3C
09
SBox
2. 1ère colonne
8A
84
EB
01
XOR
28
AE
D2
A6
XOR
02
00
00
00
=
A0 88
FA 54
FE 2C
17 B1
A0
FA
FE
17
XOR
28
AE
D2
A6
=
3. 2ème colonne
02 04 08 10
00 00 00 00
00 00 00 00
00 00 00 00
Rcon

Déchiffrement AES

Les clefs de ronde sont utilisées dans l’ordre inverse de celui du chiffrement. On notera que l’ordre des transformations diffère de celui du chiffrement. Le déchiffrement consiste à appliquer dans l’ordre inverse du chiffrement les transformations inverses correspondantes.

L'algorithme AES - 128

Entrées : Le texte clair $m \in \{0,1\}^{128}$, la clé $k \in \{0,1\}$

Sorties : Le texte chiffré $c \in \{0,1\}^{128}$.

état $s \leftarrow m$
$s \leftarrow \text{AddRoundkey}(s, k)$
pour $i = 1 \cdots 9$ **faire**
  $s \leftarrow \text{SubBytes}(s)$
  $s \leftarrow \text{ShiftRows}(s)$
  $s \leftarrow \text{MixColumns}(s)$
  $s \leftarrow \text{AddRoundkey}(s, k_i)$
fin
$s \leftarrow \text{SubBytes}(s)$
$s \leftarrow \text{ShiftRows}(s)$
$s \leftarrow \text{AddRoundkey}(s, k_{10})$
$c \leftarrow s$
Retourner c

Rappelle:!!!!!!!

Exponentiation modulaire rapide

L'exponentiation modulaire est calcul de puissance (exponentiation) suivie d'un calcul de modulo, qui est souvent utilisé dans la cryptographie.

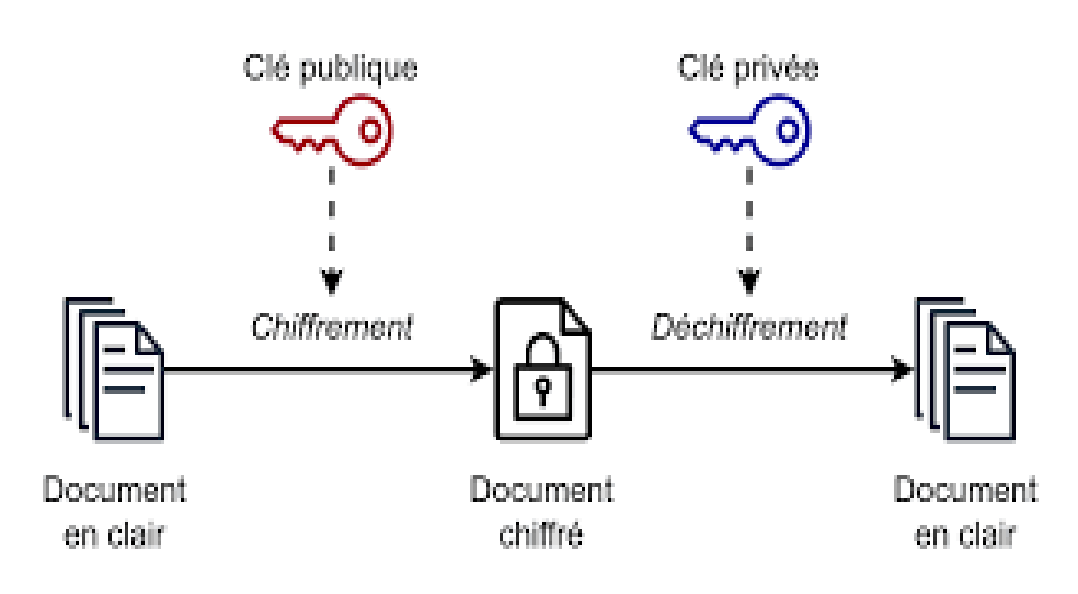
L'objectif : calculer: $\boldsymbol{c = a^e \bmod n}$ en réduisant le nombre de multiplications.

Par contre, l'algorithme d'exponentiation modulaire rapide se contente au maximum **de** $\boldsymbol{2 * log_2}$ **(e)** multiplications.
Par exemple, pour calculer $\boldsymbol{a^{1023}}$ a 1023 , avec la méthode directe, on doit effectuer 1023 multiplications, alors qu'avec l'exponentiation modulaire rapide, le nombre maximum de multiplications est
$\boldsymbol{2 * log_2}$ **(1023) = 20.**

4.2. Chiffrement Asymétrique

L’idée cruciale derrière le concept du système cryptographique asymétrique est que, chaque **utilisateur A possède une paire de clés, une clé privée n’étant connue que de A et une autre clé publique connue de tous.**

Donc, il n’est pas nécessaire de partager des secrets à l’avance.



4.2.1. Échange de clés : Protocole Diffie Hellman

Le protocole **Diffie-Hellman** est un algorithme cryptographique permettant à deux parties de générer une clé secrète partagée par un canal de communication non sécurisé. Cette clé secrète peut ensuite être utilisée pour chiffrer les communications entre elles.

L'idée principale du protocole est de permettre aux deux parties d'échanger une clé secrète sans jamais la transmettre directement.

La sécurité du protocole repose sur la difficulté du problème de **logarithme discret.**

Chez Alice	Publique (internet)	Chez Bob
	p = nombre premier arbitraire	
	g = nombre aléatoire inférieur à p	
Ax = nombre aléatoire (privé)		Bx = nombre aléatoire (privé)
Ay = g^{Ax} modulo p		By = g^{Bx} modulo p
Ay	→	Ay
By	←	By
s = By^{Ax} modulo p		s = Ay^{Bx} modulo p
	← échange de données chiffrées avec s →	

Dans ce protocole, **la clé publique est (g, p),** la clé privée de **Alice est Ay** et **la clé privée** de **B est By**.

B. Fonctionnement

Supposant que les entités **Alice et Bob** on veulent s'échanger une clé avec **le protocole Diffie Hellman**, les étapes du protocole sont comme suit :

• **<u>Etape 1 :</u>**

Alice et Bob choisissent ensemble un grand nombre premier **p** et un entier générateur **g** tel que $1 \leq g \leq p - 1$. Cet échange n'a pas besoin d'être s´sécurisé.

• **<u>Etape 2 :</u>**

Alice et Bob choisissent secrètement un entier chacun qui sont respectivement **Ax et Bx.**

• **<u>Etape 3 :</u>**

Alice calcule : $Ay = g^{Ax} \mod p$

et **Bob** calcul***:*** $\boldsymbol{By} = g^{Bx} \mod p$

• **<u>Etape 4 :</u>**

Alice et Bob s'échangent les valeurs de **Ay et *By*** . Cet échange n'a pas besoin d'être sécurisé.

Dans ce protocole, **la clé publique est (g, p),** la clé privée de **Alice est Ay** et **la clé privée** de **Bob est By**.

4.2.2. Chiffrement de RSA (Rivest, Shamir et Adleman)

L'algorithme **RSA** (Rivest–Shamir–Adleman) est l'un des algorithmes de cryptographie à clé publique les plus utilisés dans les systèmes **de sécurisation des échanges**, comme pour le **chiffrement des données** ou **la signature électronique**. Il repose sur des principes mathématiques liés **à la factorisation des grands nombres premiers.**

L'algorithme RSA assure :

- ✓ ***la confidentialité,***
- ✓ ***l'intégrité,***
- ✓ ***l'authenticité,***
- ✓ ***la non-répudiation des communications,***
- ✓ ***stockage de données électroniques***.

L'algorithme RSA repose sur deux clés : une clé **publique** et une clé **privée**.

•**Clé publique** : Utilisée pour **chiffrer** un message ou pour **vérifier une signature**.

•**Clé privée** : Utilisée pour **déchiffrer** un message ou pour **signer** un message.

A. Création des clés

Les étapes de génération des clés RSA sont les suivantes :

• Choisir ***p et q***, deux ***grands nombres premiers distincts*** ;

• Calculer leur produit: **n= p ∗ q**, appelée module de chiffrement ;

• Calculer la fonction d'Euler : $\phi(n) = (p - 1)(q - 1)$; (Cette fonction est utilisée pour choisir un exposant public) .

• **Choisir un exposant public *e*** : tell que $pgcd(e, \varphi(N))=1$,

• Calculer l'exposant privé ***d*** tel que : $e\ d \bmod \phi(n) = 1$.

Une fois ces calculs effectués, on obtient :

•**Clé publique** : (e, n)

•**Clé privée** : (d, n).

B. Chiffrement:

Le message **m** est transformé en texte chiffré **c** avec la clé publique (***e, n)*.**

$$c = m^e \bmod n.$$

C. Déchiffrement:

Pour déchiffrer **C**, on utilise la clé privée (**d, n**) comme suit :

$$M = C^d \bmod n.$$

4.2.3. Chiffrement d’El Gamal

Le **chiffrement d'ElGamal** est un algorithme de cryptographie asymétrique basé sur le problème du **logarithme discret**. Il a été conçu par **Taher ElGamal** en 1985 .

Il est utilisé pour assurer:

- la confidentialité des communications dans des systèmes de cryptographie à clé publique.
- **le chiffrement des données,**
- **la signature numérique**.

A. Génération de clés :

- **Choisir un groupe cyclique G** de générateur **g**, avec un grand nombre premier **p** (qui définit l'ordre du groupe).

NB: Un nombre premier **p** et un générateur **g** de ce groupe **G** sont choisi publiquement.

- Choisir une clé privée ***a*** **tel que :**

a < p ;
g < p ;

- Calcule de la clé publique A : $A = g^a \bmod p$

La clé publique est (A, g, p) et la clé privée est **a**.

B. Chiffrement

Pour chiffrer un message M, on choisit un nombre **aléatoire b**, tel que 1**< b < p-1** et **le pgcd(b, p−1) = 1**, puis on calcule :

$$C_1 = g^b \bmod p;$$
$$C_2 = M * A^b \bmod p;$$

Le message chiffré est alors (C_1, C_2).

C. Déchiffrement:

Pour déchiffrer un message avec l'algorithme d'El Gamal, on utilise la clé privée :

$$M = C_2 {C_1}^{-1} \bmod p$$

Comme le protocole Diffie-Hellman, El Gamal est sécurisé contre les attaques d'interception.
Le chiffrement d'El Gamal est **probabiliste**, vue que le chiffrement d'un même message donnera des résultats différents en raison du nombre aléatoire ***k***, ce qui augmente la sécurité en rendant les attaques par analyse de fréquence plus difficiles.

Limitations :

•**Efficacité** : Le chiffrement d'El Gamal est plus lent que d'autres systèmes.

•**Taille des messages chiffrés** : Les messages chiffrés dans El Gamal ont une taille double par rapport au message original, car le système génère deux parties pour chaque message.

•**Requiert un nombre aléatoire** : Le système de chiffrement dépend de la génération d'un nombre aléatoire k unique pour chaque chiffrement, ce qui peut compliquer la gestion de la sécurité dans des systèmes contraints en ressources.

4.2.4. Chiffrement de Rabin

Le chiffre de Rabin est un chiffrement asymétrique qui a été inventé en 1979 par Michael Rabin.

Il repose sur un problème de **racines carrées modulo un produit de deux grands nombres premiers**.

Le but est de sécuriser la communication en utilisant des clés publiques et privées, tout en restant basé sur la même idée que RSA : la clé publique est utilisée pour chiffrer, et la clé privée pour déchiffrer.

Comme dans le chiffrement d'El Gamal, le chiffrement de Rabin est un algorithme **probabiliste**, c'est-à-dire que chaque message donnera des résultats de chiffrement différents même s'il est chiffré plusieurs fois avec la même clé publique, ce qui améliore la sécurité contre certaines attaques par analyse de fréquence.

L'une des principales caractéristiques du chiffrement de Rabin est qu'il génère **quatre racines possibles** pour chaque message chiffré. Cela peut poser des problèmes dans certains contextes où l'on souhaite obtenir un message unique. **Toutefois, il est possible de lever cette ambiguïté en ajoutant des mécanismes de vérification.**

A. Génération des clés

Comme pour tous les algorithmes de chiffrement à clé publique, le chiffre de Rabin fait usage d'une clé publique et d'une clé privée. La génération de clés est comme suit :

• Choisir deux grands nombres premiers***, p et q***, tel que ***q mod 4 = 3* et q mod 4 = 3.** Cela rend l'extraction des racines carrées **modulo p et q** très rapide.

• **la clé publique: Calculer n = p ∗ q**,.

• Le couple (**p, q) constituent la clé privée.**

B. Chiffrement

Pour chiffrer un message M en utilisant la clé publique **n** dans l'algorithme de chiffrement Rabin, on calcule:

$$C = M^2 \bmod n;$$

Par conséquent, le cryptage implique une multiplication et un modulo, donc, il est beaucoup plus rapide que le cryptage RSA, même lorsque l'on choisit un petit exposant de cryptage RSA.

En effet, le cryptage dans le chiffrement Rabin est beaucoup plus rapide que presque n'importe quel autre schéma de clé publique.

C. Déchiffrement

Le déchiffrement est beaucoup plus compliqué ; on veut essentiellement calculer la valeur de :

$$\boldsymbol{M = \sqrt{c}\ \ mod\, n}$$

A première vue, cela n'utilise aucune information privée, mais cela révèle que l'on a besoin de la factorisation de **n** pour pouvoir trouver la racine carrée. Notamment pour calculer **M** le processus est selon les étapes suivantes :

<u>Etape 1 :</u>

on calcule m_p et m_q tel que :

- $\boldsymbol{m_p = C^{\frac{p+1}{4}}\ mod\ \ p}$
- $\boldsymbol{m_q = C^{\frac{q+1}{4}}\ mod\ \ q}$

Etape 2 :

on calcule Y_P et Y_q en utilisant l'algorithme d'Euclide étendu pour calculer l'équation diophantienne suivante :

- $\mathbf{p * Y_P + q * Y_q = 1}$

Etape 3 :

on calcule R et S tel que :

- $\mathbf{R = (p * m_q * Y_p + q * m_p * Y_p)}$
- $\mathbf{S = (p * m_q * Y_p - q * m_p * Y_p)}$

Etape 4 :

Dans cette étape, on calcule les solutions possibles qui sont comme suit : **R, S, n − R, n − S**

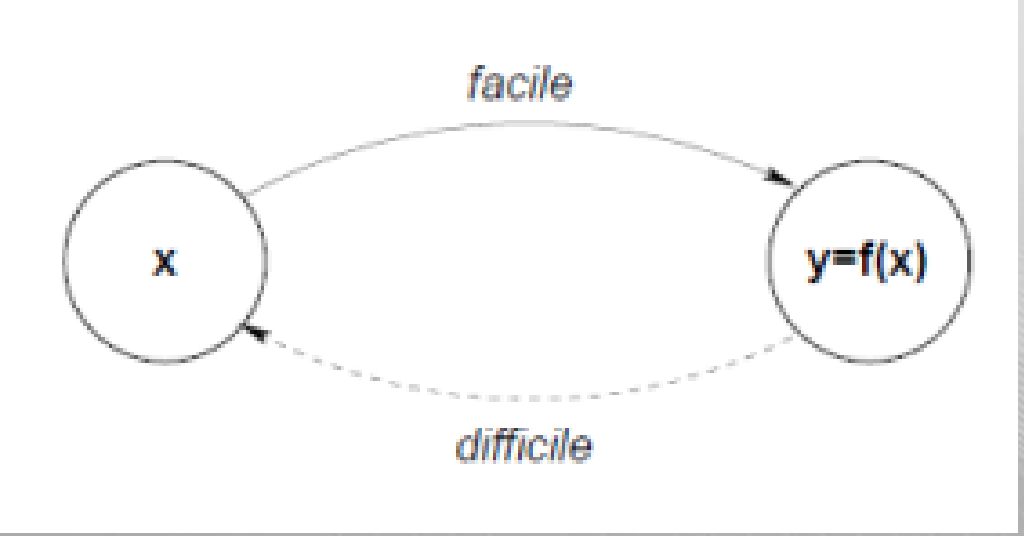
Parmi ces solutions, une seul représente le message en claire M.

5. Les fonctions à sens unique

Considérons deux ensembles arbitraires X et Y et une fonction $f: X \rightarrow Y$.

Soit $f(X)$ l'ensemble image de X par f. La fonction f est dite à sens unique :

- si pour tout x de X, il est facile de calculer $f(x)$ (autrement dit, f peu être calculée en temps polynomial)
- s'il est difficile de trouver, pour la plupart des $y \in f(X)$ un $x \in X$ tel que $f(x) = y$.



- Factorisation des nombres premiers
- Exponentiel modulaire
- Problème du logarithme discret
- Fonction de hachage

Caractéristiques clés d'une fonction à sens unique :

•**Facilité de calcul direct** : L'image d'un élément donné est facile à calculer.

•**Difficulté d'inversion** : Il est pratiquement impossible de retrouver l'élément original à partir de l'image sans informations supplémentaires.

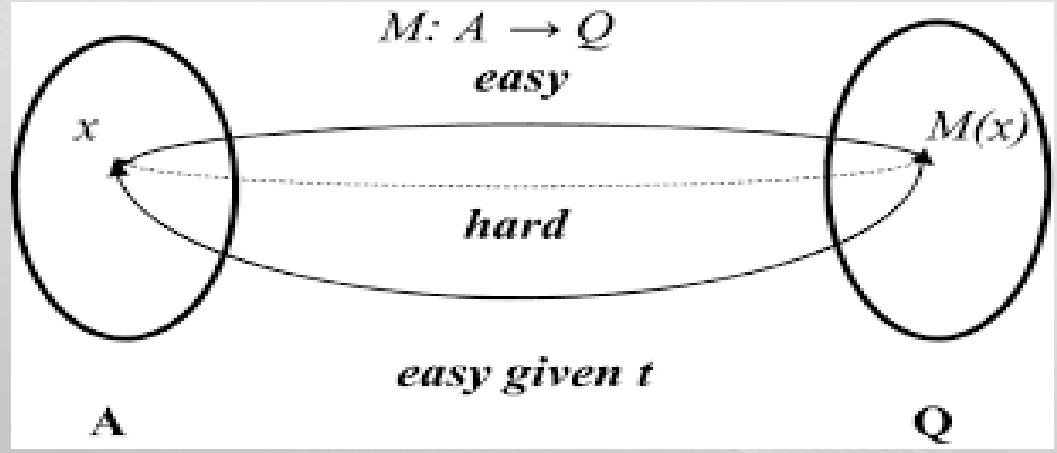
•**Résistance aux collisions** : Une bonne fonction à sens unique minimise les risques que deux entrées distinctes donnent la même image (collision).

Password Storage
1
2
Message Integrity
FasterCapital
Digital Signatures
3
4
Data Authentication

5.1. Fonction à sens unique à trappe

C’est une fonction à sens unique qui, en plus de ses caractéristiques habituelles (facile à calculer mais difficile à inverser), possède une **trappe secrète** qui permet de retrouver facilement l'inverse de la fonction **si l'on connaît un secret spécifique**, appelé la "clé secrète" ou "trappe".

- **Exemple:**



- Cryptographie asymétrique
- Signature numérique

Propriétés principales :

1. **Facile à calculer** dans un sens .
2. **Difficile à inverser sans la clé secrète.**
3. **Existent une trappe secrète** : Si vous avez accès à une "trappe secrète" (ou clé secrète), vous pouvez inverser la fonction facilement et retrouver l'entrée à partir de l'image. C'est ce qui distingue une fonction à sens unique à trappe d'une fonction à sens unique classique.

6. Les fonctions de Hachage

Les fonctions de hachage sont des primitive cryptographiques importantes et elles sont largement utilisées dans les protocoles.

Elles calculent un résume d'un message qui est une chaîne de bits courte et de longueur fixe. Pour un message particulier, le résume du message, ou la valeur de hachage, peut être considère comme l'empreinte digitale d'un message, c'est-`a-dire une représentation unique d'un message. Contrairement à tous les autres algorithmes de cryptographie présentés jusqu'à présent dans ce cours, ***les fonctions de hachage n'ont pas de clé***.

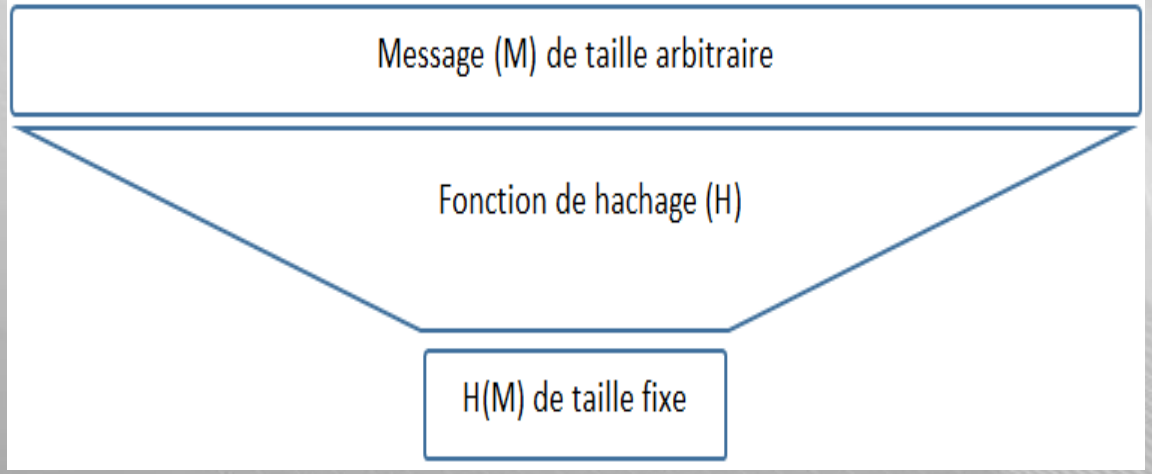
une chaîne de bits courte et de longueur fixe

6.1. Propriétés des fonctions de hachages

Soit H une fonction de hachage, on note ***h = H(x),*** alors x est appelé préimage de h.
Les propriétés à vérifier dans une fonction de hachage **H** à utiliser pour la cryptographie sont :

1. **Propriété de compression :**

Quelques soit **la taille du message x**, il est facile de calculer **le haché H(x)**, et sa la taille est fix et elle est généralement inferieur à celle du message ;

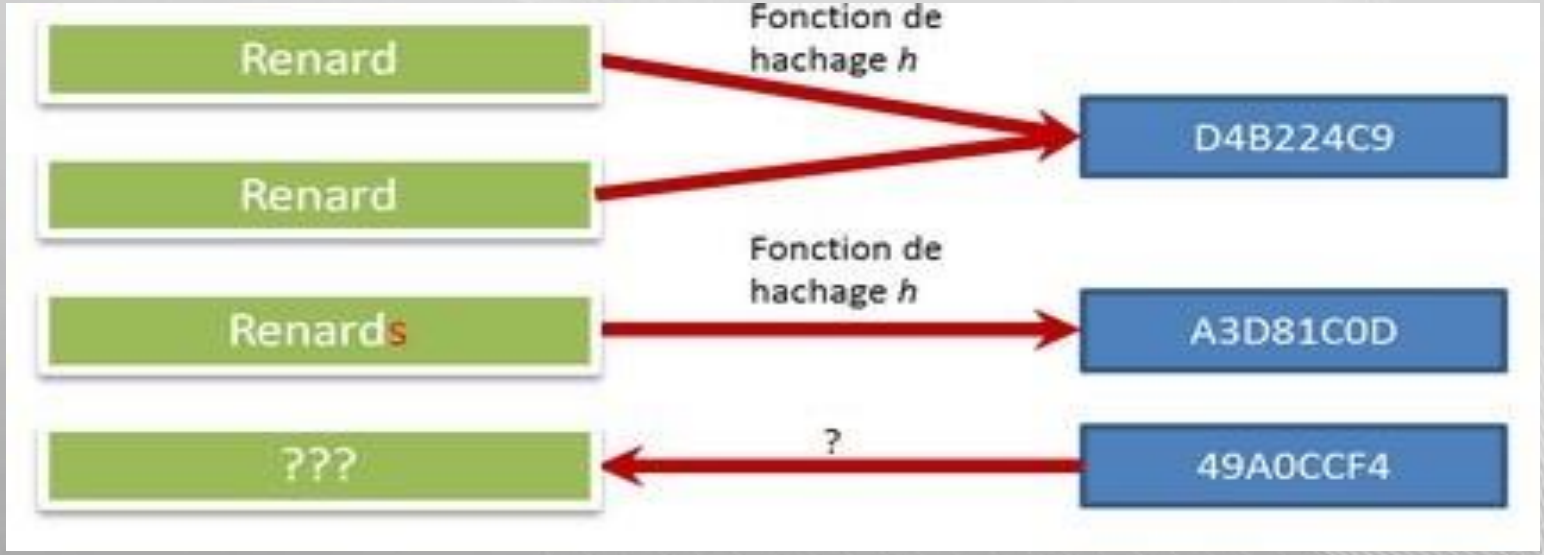


2. Résistance à la préimage :

Etant donné **y**, il est difficile de trouver **un x** tel que **y = H(x)** (**<u>fonction à sens unique</u>**) ;

3. Résistance à la collision :

Il est très difficile de trouver **x et x ′** tel que : **x ′≠ x et H(x ′) = H(x).**

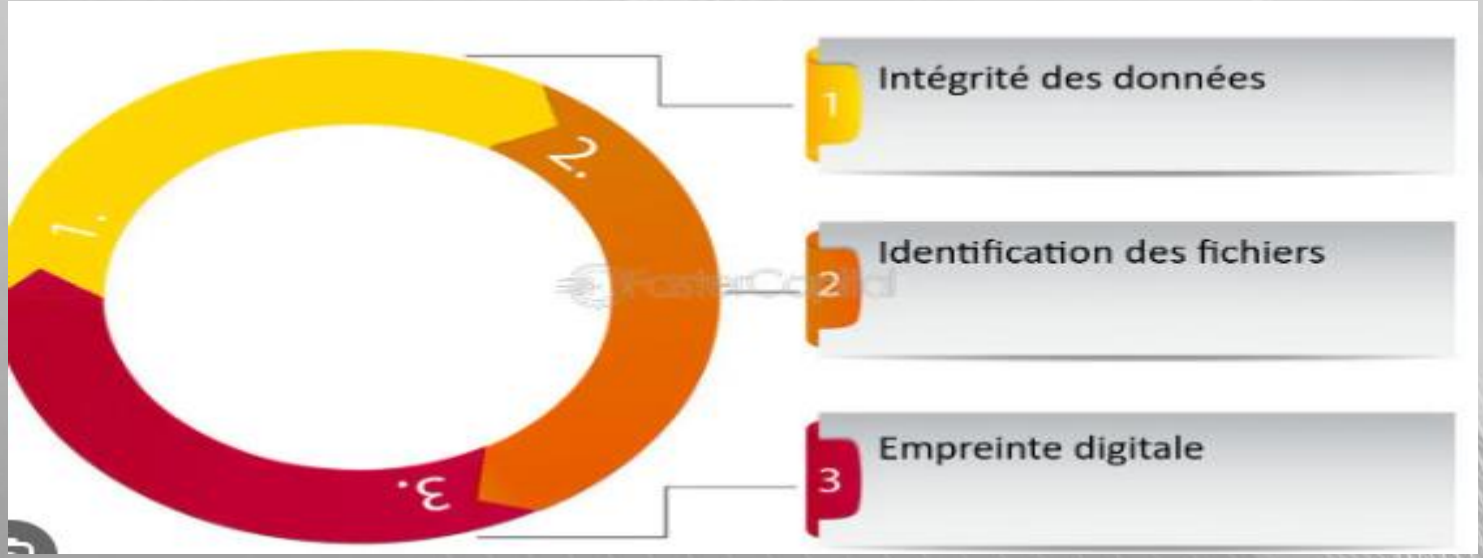


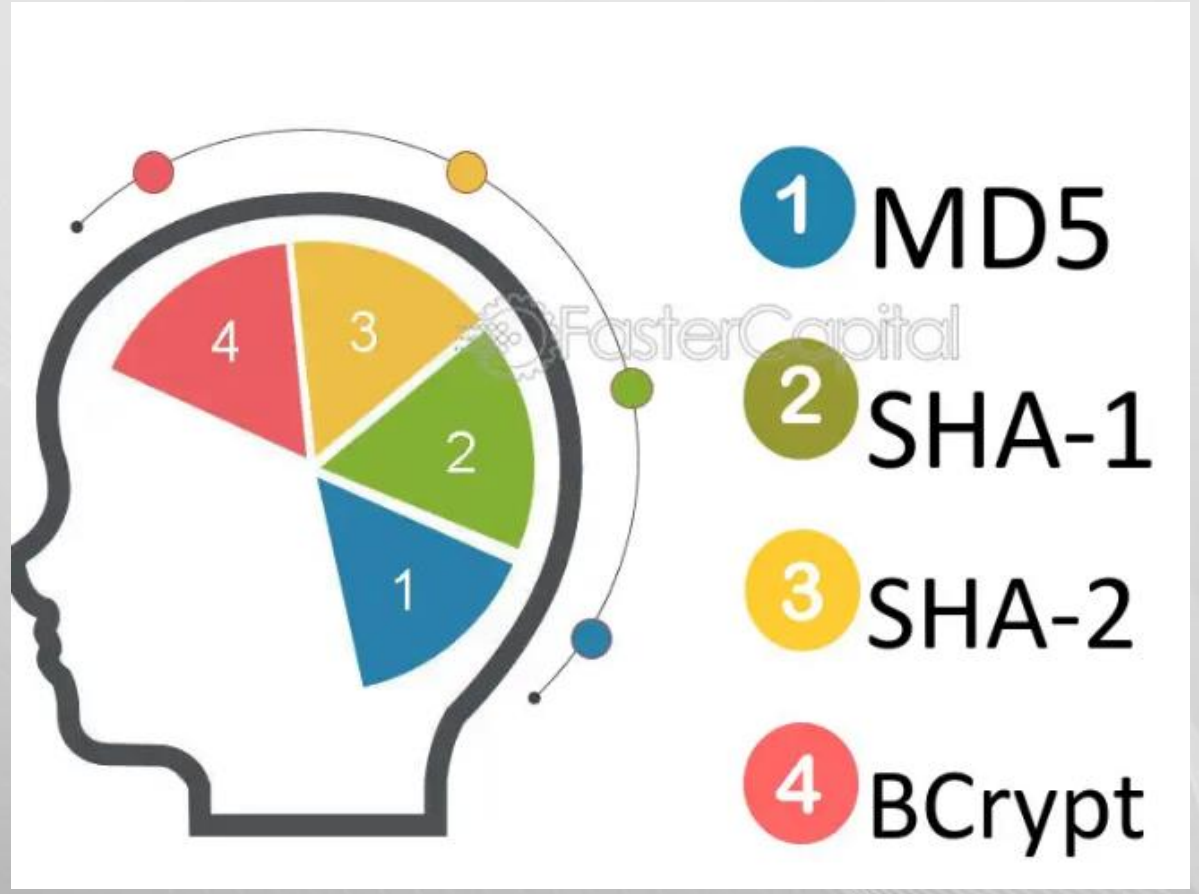
6.2. Utilisation des fonctions de hachages

Les fonctions de hachage sont utilisées dans:

• L'intégrité est assurée dans la mesure où les fonctions de hachage permettent d'obtenir facilement une empreinte (haché, condensé, résumé) d'une donnée (fichier, message...), cette empreinte est utilisée comme l'identification unique de cette donnée pour contrôler d'éventuelles altérations ou modifications lors de la transmission.

• la signature numérique, et pour des raisons d'efficacité , on signe l'empreinte de la donnée à transmettre et pas la donnée elle-même.




4
3
2
1
FasterCapital
1 MD5
2 SHA-1
3 SHA-2
4 BCrypt

Les algorithmes de Hachage:

1. **MD5:** L'algorithme MD5 est un algorithme de hachage largement utilisé qui produit une valeur de hachage **128 bits**. Cependant, il est considéré comme moins sûr en raison de sa sensibilité aux **collisions et aux attaques pré-image**.

2. **SHA-1:** L'algorithme SHA-1 produit une valeur de **hachage de 160 bits** et est plus sûr que MD5.Cependant, il est toujours vulnérable aux attaques de collision et est supprimé en raison de ses faiblesses de sécurité.

3. **SHA-2:** La famille des algorithmes SHA-2, y compris **SHA-256** et **SHA-512**, produit respectivement des valeurs **de hachage de 256 et 512 bits**. Ils sont plus sûrs que SHA-1 et sont couramment utilisés dans les signatures numériques et l'authentification des messages.

4. **BCrypt:** l'algorithme BCrypt est spécialement conçu pour **le hachage du mot de passe.** Il est connu pour sa sécurité et son évolutivité. Il est plus lent que les autres algorithmes, mais sa capacité à ajuster la charge de travail le rend plus résistant aux attaques par force brute.

MD5 (Message Digest 5)

La fonction de hachage MD5 est une version améliorée de MD4, les deux algorithmes ont été développé par Ron Rivest, un des créateurs de RSA.

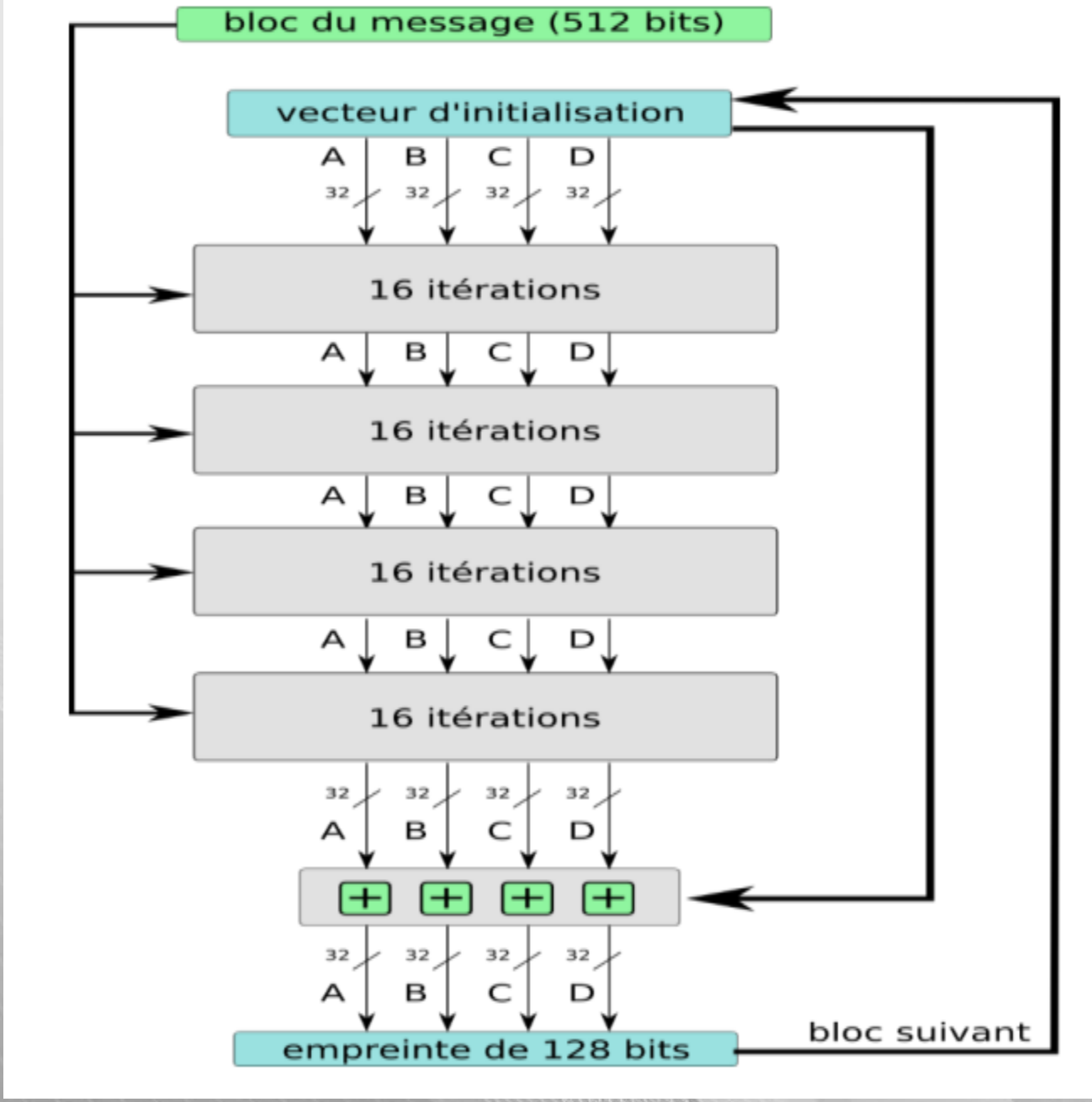
MD5 utilise des blocs de 512 bits et génère des **messages chiffrés de 128 bits**. Chaque bloc est découpé en **16 sous-blocs de 32 bits** (A, B, C et D).

Principe de fonctionnement

- MD5 prend 4 buffers de 32 bits en entrée (en hexadécimal):

A=01234567
B=89abcdef
C=fedcba98
D=76543210

- MD5 est composé de quatre rondes qui exécutent chacune 16 opérations.
- Pour chaque ronde, une seule fonction prenant 3 arguments codés sur 32 bits et renvoyant une valeur sur 32 bits est utilisée pour les 16 opérations.
- A l'issue des 4 rondes, les nouvelles valeurs de A, B, C, D sont ajoutés aux anciennes.
- L'empreinte finale de 128 bits est la concaténation du dernier jeu ABCD.



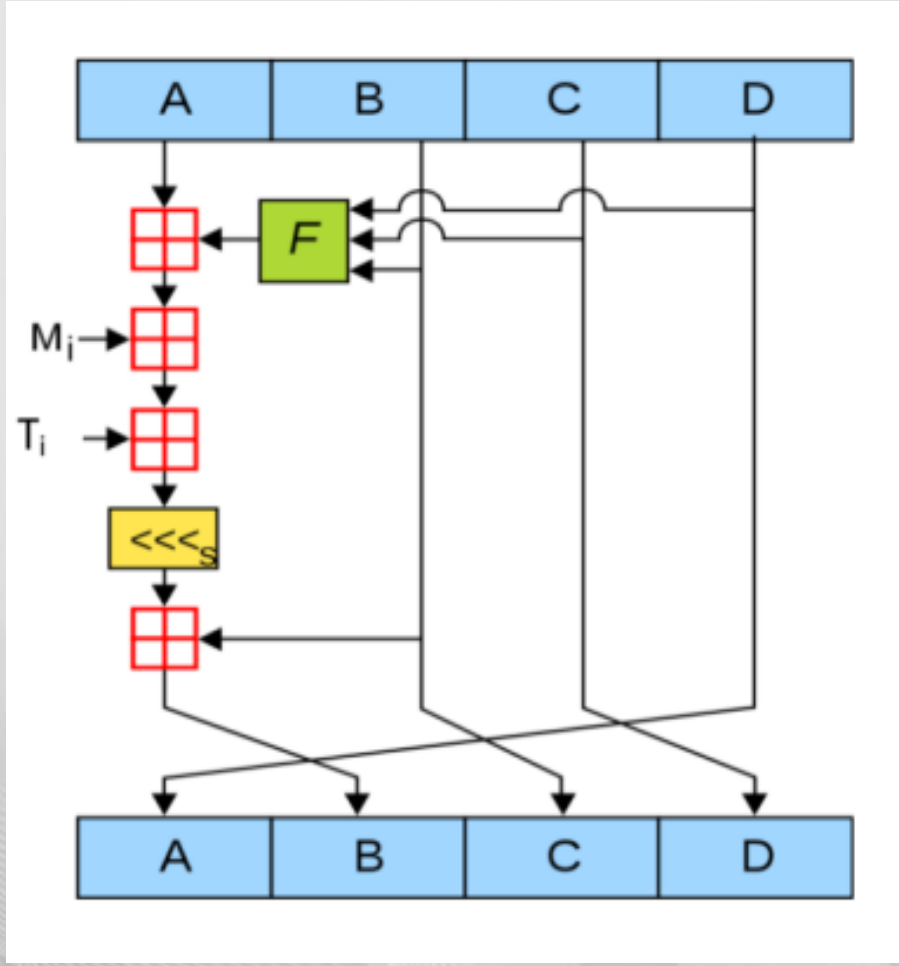
- A chaque étape de chaque ronde une fonction non linéaire est effectuée sur 3 des variables B,C,D, le résultat est ajouté à un des 16 mots du bloc de message et à un élément du tableau T.

  - Les 4 fonctions sont les suivantes :

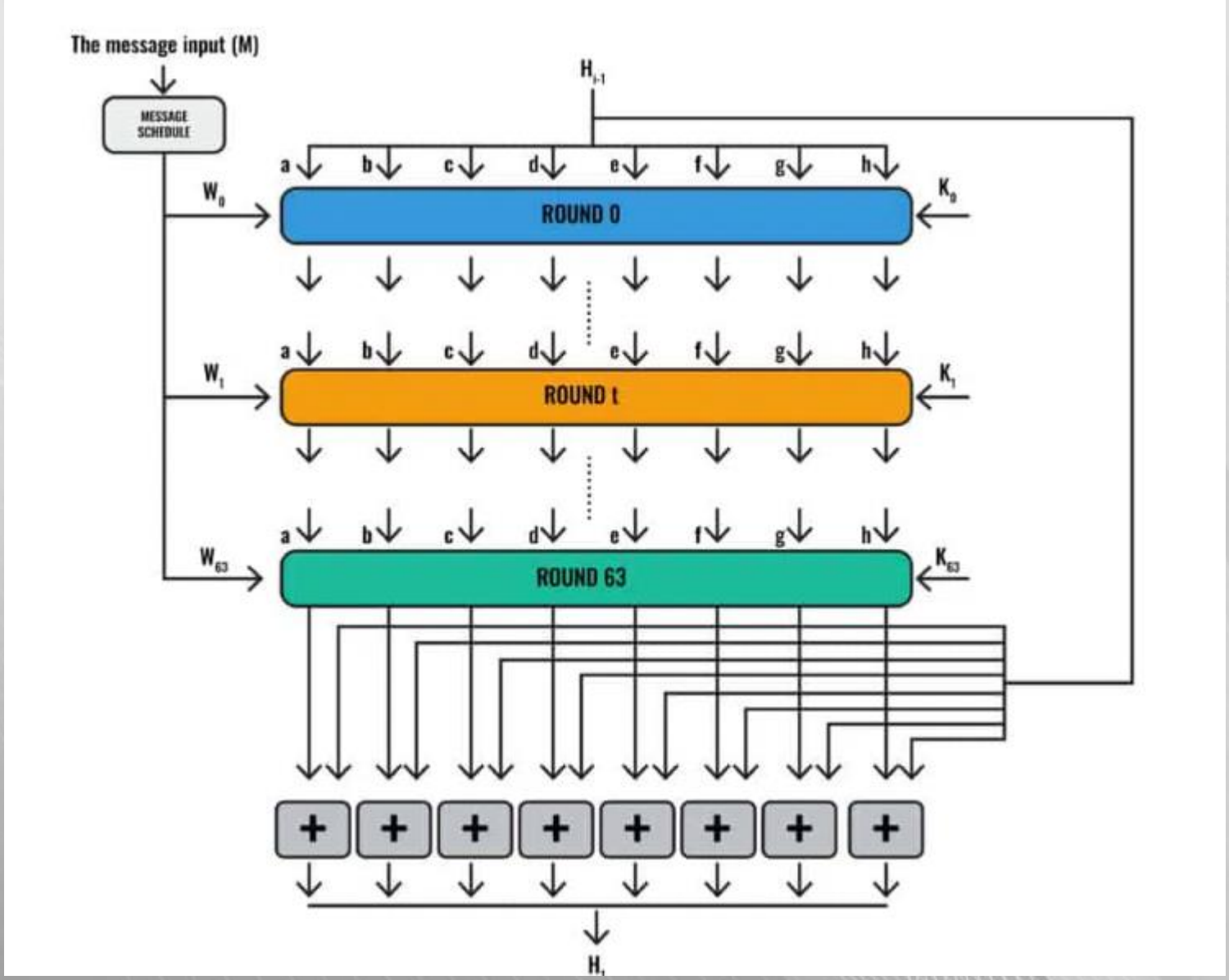
1. F= (B and C) or (not(B) and D)
2. F = (D and B) or (C and not(D))
3. F = B XOR C XOR D
4. F = C XOR (B OR NOT(D))

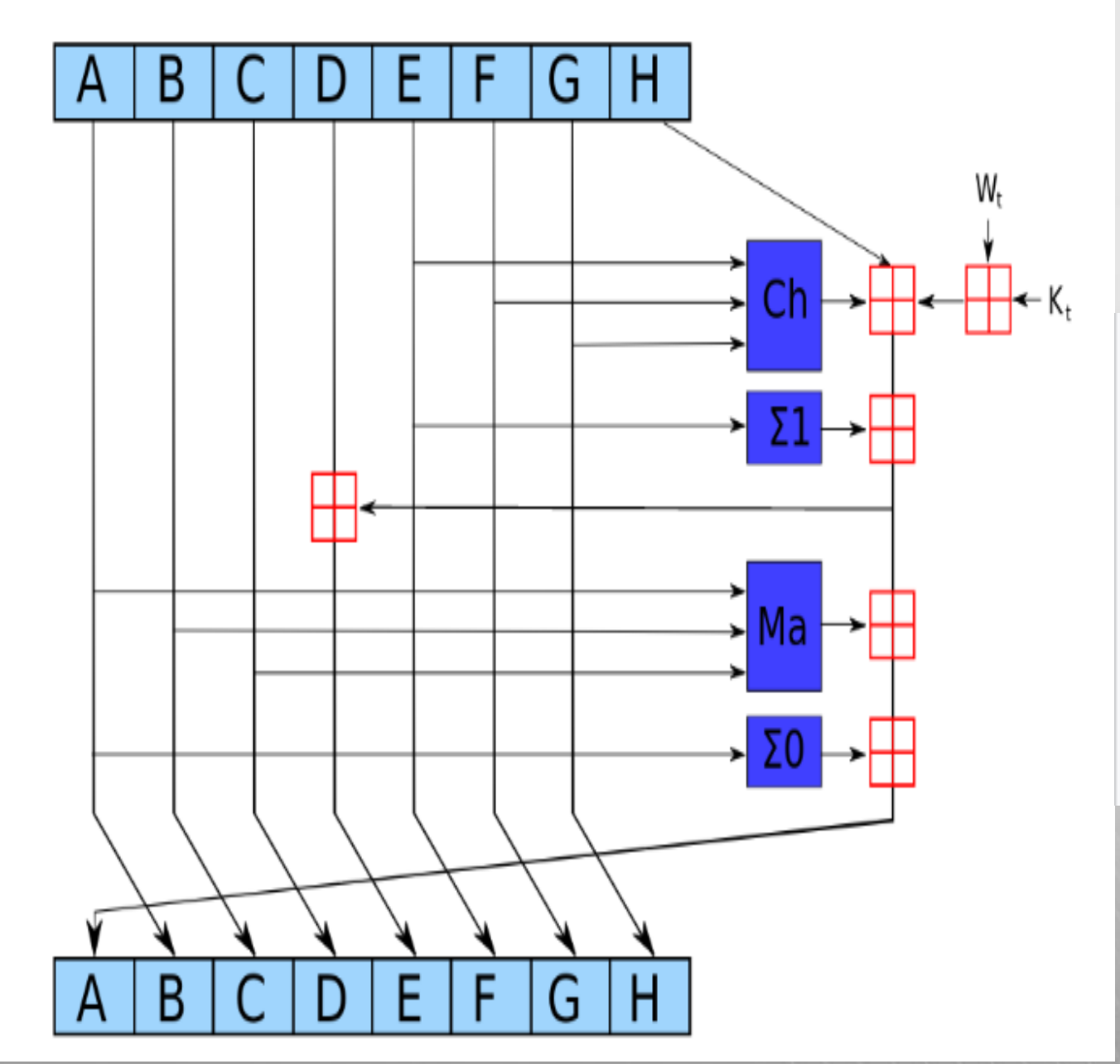
- **Mj** symbolise un bloc de 32 bits provenant du **message à hacher .**
- **Ti** est une constante de 32 bits, différentes pour chaque itération

- Le résultat est décalé de **s bits**.



L'algorithme SHA-2





$$\mathrm{Ch}(E,F,G) = (E \wedge F) \oplus (\neg E \wedge G)$$

$$\mathrm{Ma}(A,B,C) = (A \wedge B) \oplus (A \wedge C) \oplus (B \wedge C)$$

$$\Sigma_0(A) = (A \ggg 2) \oplus (A \ggg 13) \oplus (A \ggg 22)$$

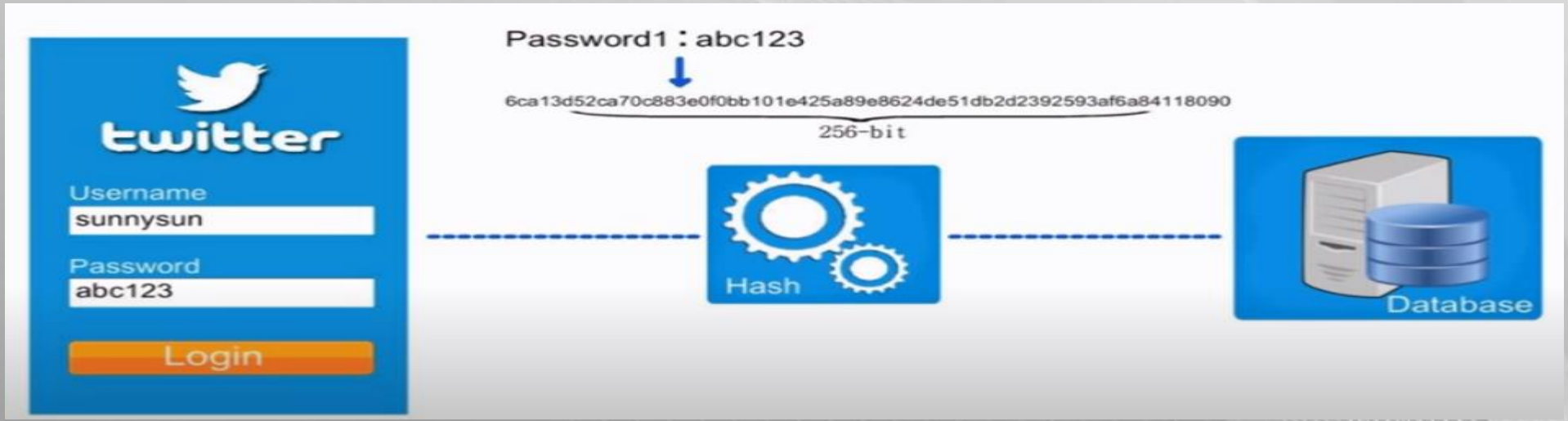
$$\Sigma_1(E) = (E \ggg 6) \oplus (E \ggg 11) \oplus (E \ggg 25)$$

Les fonctions de hachage ont de nombreuses **applications pratiques** dans divers domaines de la technologie, en particulier dans la **sécurité informatique**, la **gestion des données**, et les **cryptomonnaies**.

1. Vérification de l'intégrité des données

Exemple : Lorsqu'un fichier est téléchargé, son empreinte (hash) peut être calculée et comparée à celle fournie par le site de téléchargement. Si les empreintes correspondent, cela signifie que le fichier n'a pas été altéré durant le téléchargement.

2. Stockage sécurisé des mots de passe



7. La signature numérique

Une **signature numérique** est un mécanisme cryptographique qui **permet de garantir l'intégrité, l'authenticité et l'origine d'un message ou d'un document électronique**.

Elle joue un rôle fondamental dans les échanges sécurisés:

- la banque en ligne,
- les contrats électroniques,
- la transmission de données sensibles.

- **Authentification** : La signature numérique garantit que l'expéditeur est bien celui qu'il prétend être, car seule la personne possédant la clé privée associée à la clé publique peut signer un message valide.
- **Intégrité** : Toute modification du message après la signature entraînerait une discordance entre l'empreinte recalculée et celle déchiffrée, ce qui permettrait de détecter toute altération.
- **Non-répudiation** : Une fois qu'un message est signé, le signataire ne peut pas nier avoir signé ce message, car il est impossible de produire une signature valide sans disposer de la clé privée correspondante.

7.1. Le principe de la signature numérique

Etape 1: Génération de la signature :

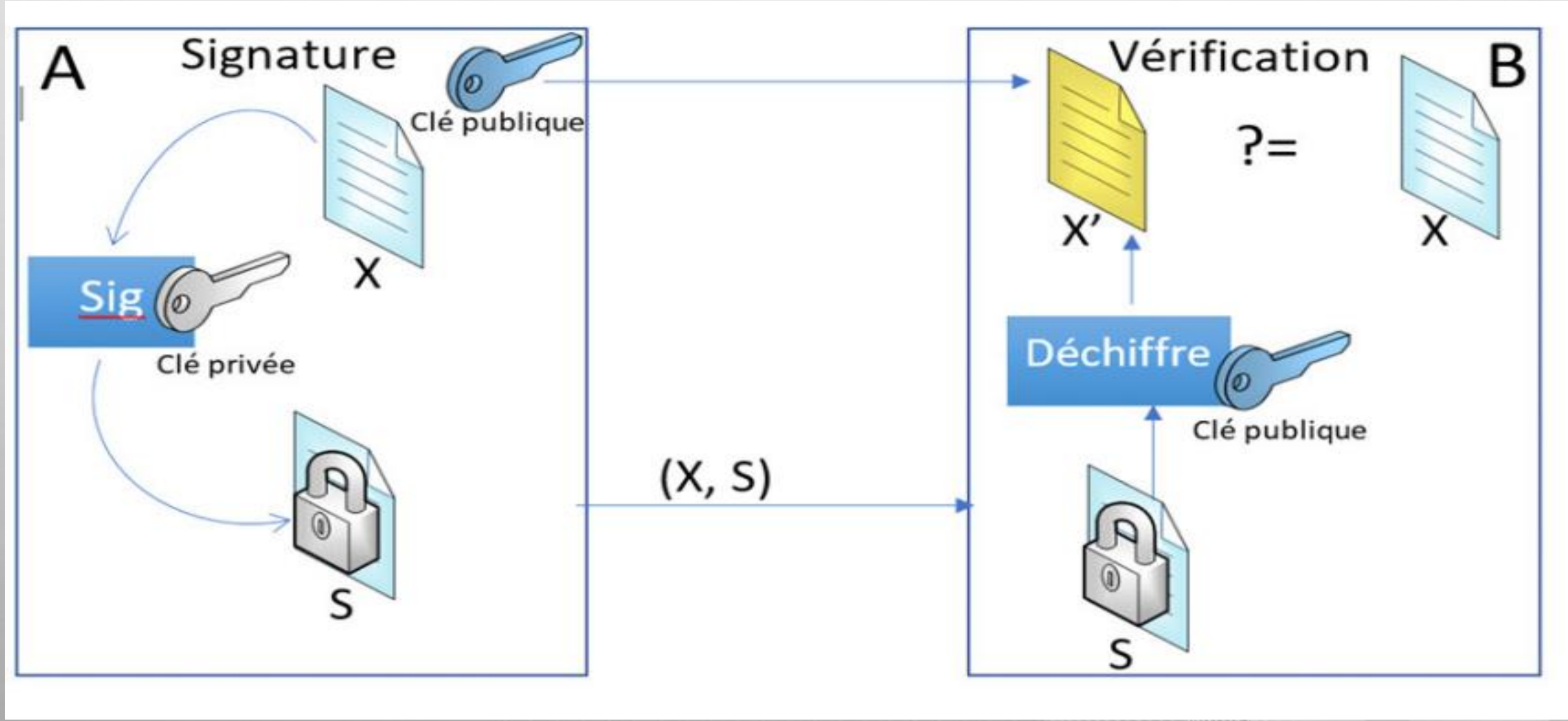
•**Hachage du message** : D'abord, le message (ou le fichier) à signer est passé par une fonction de hachage (comme SHA-256) pour produire un **empreinte** du message (une valeur de taille fixe). Cela permet de réduire la taille des données à signer.

•**Chiffrement de l'empreinte** : l'empreinte du message est chiffrée avec la **clé privée** du signataire. Cela crée la signature numérique. La clé privée est unique et est détenue exclusivement par la personne qui signe.

ETAPE 2: Vérification de la signature :

Afin de vérifier que la signature numérique est valide, la personne qui reçoit le message doit effectuer les étapes suivantes :

1. Calculer l'empreinte du message reçu avec la même fonction de hachage que celle utilisée pour créer la signature.
2. Déchiffrer la signature numérique en utilisant la **clé publique** du signataire. La clé publique est accessible à toute personne et permet de récupérer l'empreinte du message original.
3. Si l'empreinte calculée correspond à celle déchiffrée, cela signifie que le message n'a pas été modifié (intégrité) et qu'il a bien été signé par la personne possédant la clé privée associée à la clé publique (authenticité).


A
Signature
Clé publique
X
Sig
Clé privée
S
(X, S)
Vérification
B
?=
X'
X
Déchiffre
Clé publique
S

- Le processus commence lorsque **l'entité A signe le message X en utilisant une clé privé.**
- Supposant que seul A peut signer un message X en son nom. Afin d'associer une signature au message, X est également une entrée de l'algorithme de signature.
- La signature S est ajoutée **au message X et la paire (X, S)** est envoyée à l'entité B. Il est important de noter qu'une signature numérique en elle-même ne sert à rien sauf si elle est accompagnée du message.
- La signature numérique elle-même est simplement une (grande) valeur entière, par exemple, une chaîne de 2048 bits.
- **La signature n'est utile à B que s'il a les moyens de vérifier qu'elle est valide**. Pour cela, une **fonction de vérification est nécessaire qui prend à la fois X et la signature S comme entrées**. Afin de lier la signature à A, la fonction nécessite **également la clé publique de A**. Si X a effectivement été signé avec la clé privée qui appartient à la clé de vérification publique, la sortie est vraie, sinon elle est fausse.

7.2. La signature numérique avec RSA

Le procède de génération des paramètres est identique à celui utilisé pour l'algorithme de cryptage RSA, **l'expéditeur A doit disposer d'une clé publique (e, n) et d'une clé privée (d, n).**

Les étapes de la création de la signature numérique du message M avec RSA sont :

• Etape 1 :

Calculer le haché de M tel que : $\mathbf{h} = \boldsymbol{H}(\boldsymbol{M})$;

• Etape 2 :

Calculer $\boldsymbol{S} = \boldsymbol{h}^{\boldsymbol{d}}$ **mod n**;

• Etape 3 :

Envoyer la signature numérique **($\boldsymbol{M}$, S)**

Les étapes de vérification de l'entité B de la signature numérique transmise par A Sont :

• Etape 1 :

Calculer du haché h1, tel que $\mathbf{h_1} = \boldsymbol{H}(\boldsymbol{M})$;

• Etape 2 :

Calculer de $\mathbf{h_2} = \mathbf{S^e}$ **mod n**

• Etape 3 :

Comparer les deux signature $\mathbf{h_1} \mathbf{?} = \boldsymbol{h_2}$, si les deux signatures sont égales, alors le message M est authentique, sinon le message M est altéré.

7.3. La signature numérique avec Rabin

Le procédé de génération des paramètres est identique à celui utilisé pour l'algorithme de cryptage de Rabin, **l'expéditeur A** doit disposer **d'une clé publique n** et **d'une clé privée (p, q).**

Les étapes de la création de la signature numérique du message M avec Rabin sont :

• Etape 1 :

Calculer : ***h = H(M)*** ;

• Etape 2 :

Calculer $\alpha = h^{\frac{p+1}{4}} \bmod n$

• Etape 3 :

Choisir **S parmi les quatre solutions possible** ;

• Etape 4 :

Envoyer la signature numérique (***M*** **, S**).

Les étapes de vérification de l'entité B de la signature numérique transmise par A Sont :

• Etape 1 :

Calculer du haché h1, tel que ***h1 = H(M)*** ;

• Etape 2 :

Calculer de : $\mathbf{h_2} = \alpha^2 \mathbf{\bmod n}$;

• Etape 3 :

Comparer les deux signature $\mathbf{h_1 ? = h_2}$, si les deux signatures sont égales, alors le message M est authentique, sinon le message M est altéré.

7.4. La signature numérique avec EL Gamal

Le procédé de génération des paramètres est identique à celui utilisé pour l'algorithme de cryptage de Rabin***, l'expéditeur A doit disposer d'une clé publique (A, g, p), et d'une clé privée a***.

Les étapes de la création de la signature numérique du message M avec El Gamal sont :

• Etape 1 :

Calculer h = H(M) ;

• Etape 2 :

Choisir un nombre **aléatoire b(b < p et pgcd (b, p − 1) = 1);**

• Etape 3 :

Calculer les $B = g^b \bmod n$;

• Etape 4 :

Calculer S tel que $S = b^{-1}\ (H(M)-a\ .\ B) \bmod (p-1)$;

• Etape 5 : envoyer la signature numérique (M, S) tel que **s = (B, S).**

Les étapes de vérification de l'entité B de la signature numérique transmise par A Sont :

• Etape 1 :

Calculer $h1 = H(M)$;

• Etape 2 :

Si $h^B\ B^S \bmod p = g^h \bmod p$ alors la signature est valide.

8. Les certificats électroniques

Un certificat électronique est un fichier électronique infalsifiable, délivré par une autorité de certification, qui permet d’authentifier une entité dans le cyberespace. Cette autorité digne de confiance atteste après constat (par sa signature électronique), la véracité du contenu du certificat.

Autrement dit, Un certificat est une structure de donnée signée numériquement **qui atteste sur l’identité du possesseur de la clé privée correspondante à une clé publique.**

Une carte d’identité contient entre autres des informations sur :

•Le propriétaire: Noms, prénoms, adresse, emprunte et signature manuscrite ;

•La période de validité de la CNI ;

•Le tampon ou cachet de l’autorité reconnue et abilitée à le faire, ayant délivré la carte et une présentation (forme, couleur, papier) qui permettent de reconnaître qu’il n’est pas un faux.

Un certificat électronique

Le format le plus couramment utilisé actuellement est le format **X.509 V3**. Ce fichier ou structure de données contient au moins les informations suivantes :

- les noms et prénoms du propriétaire du Certificat, Son service, Son adresse électronique, **Sa clé publique** ;
- les dates de validité du certificat ;
- le nom de l'autorité ayant délivré le certificat et sa signature électronique.
- le numéro de version du certificat (v1, v2 ou v3) ;
- l'algorithme asymétrique et la fonction de hachage utilisés pour la signature ;

Général Détails

Ce certificat a été vérifié pour les utilisations suivantes :

Certificat client SSL

Certificat serveur SSL

Émis pour

Nom commun (CN)	*.malekal.com
Organisation (O)	<Ne fait pas partie du certificat>
Unité d'organisation (OU)	Domain Control Validated
Numéro de série	53:F4:C3:EA:AF:2C:72:FF:43:DC:09:FF:8D:26:CA:06

Émis par

Nom commun (CN)	Gandi Standard SSL CA 2
Organisation (O)	Gandi
Unité d'organisation (OU)	<Ne fait pas partie du certificat>

Période de validité

Débute le	26 mai 2018
Expire le	12 juillet 2020

Empreintes numériques

Empreinte numérique SHA-256	F6:79:94:A5:BD:FB:F2:6C:B6:CC:09:EC:03:61:13:1E: C3:DE:82:D5:22:37:16:F5:0B:C3:60:58:C2:03:6B:41
Empreinte numérique SHA1	3A:E2:F0:B5:05:10:25:D8:5A:AE:00:D6:6E:32:6C:76:79:DB:D2:A5

Prouver l'authenticité d'une clé publique

Prouver le lien entre une identité et une bi-clé

Techsmith
REALISEE AVEC L'EVALUATION GRATUITE DE CAMTASIA

Etablir un environnement de confiance entre deux entités ayant besoin de communiquer et de s'échanger des informations :

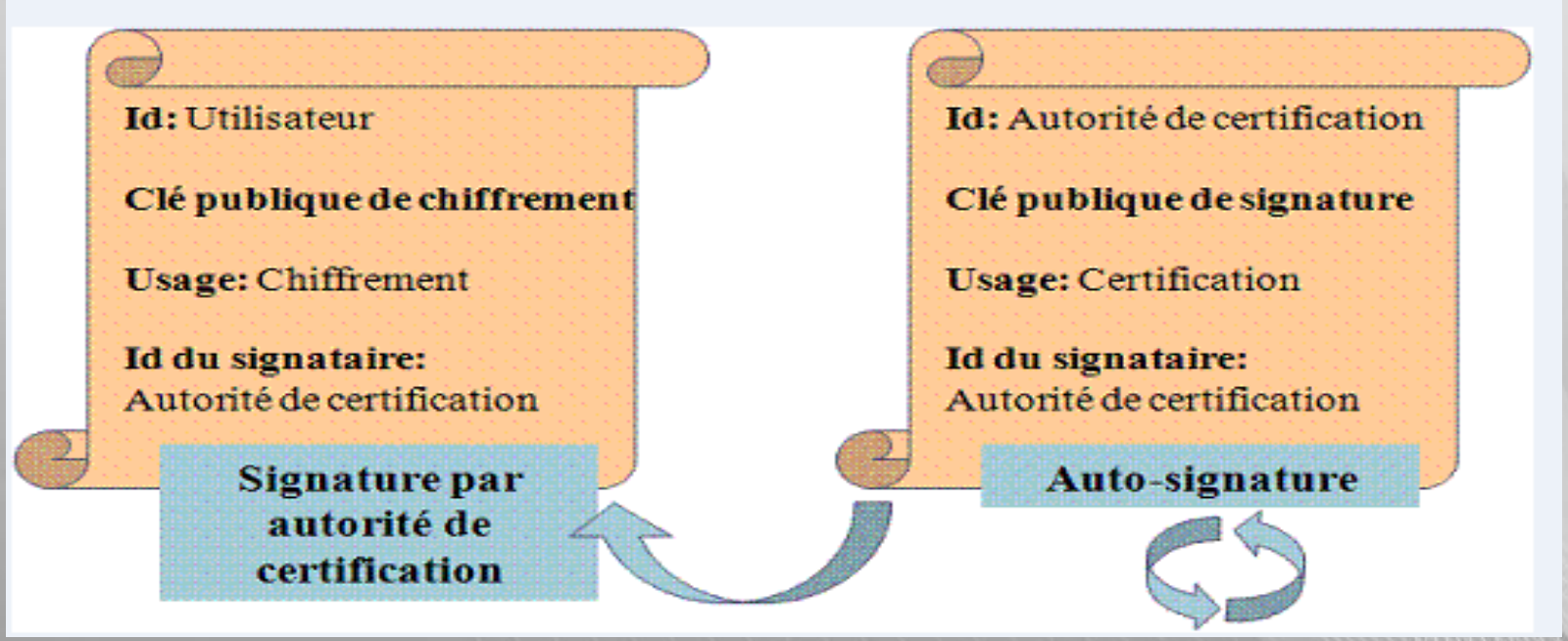
- non-répudiables
- confidentielles

Activer Windows

Il existe deux types de certificats selon le niveau de signature :

•Les **certificats auto-signés** sont des certificats à usage interne. Signés par un serveur local, ce type de certificat permet de garantir la confidentialité des échanges au sein d'une organisation.

•Les **certificats signés par un organisme de certification** sont nécessaires lorsqu'il s'agit d'assurer la sécurité des échanges avec des utilisateurs anonymes.

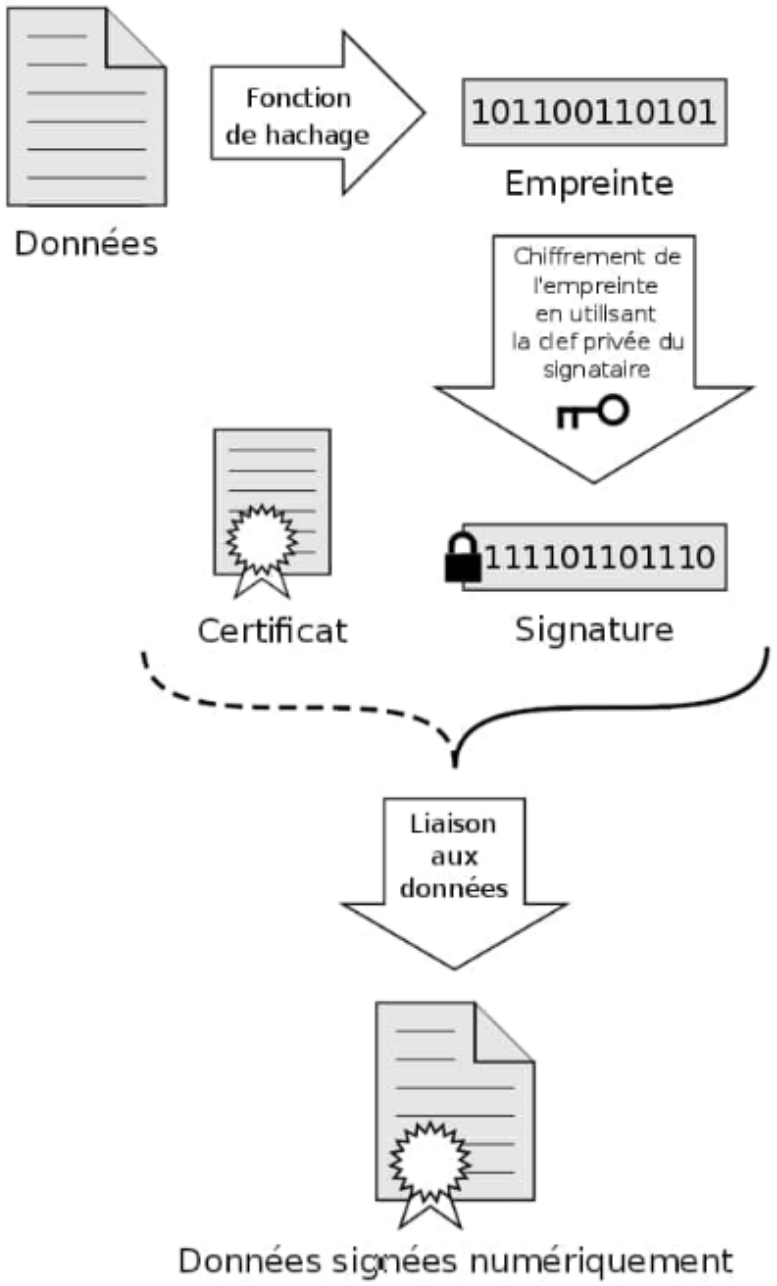


Un certificat numérique utilise la cryptographie asymétrique. Ce procédé utilise :

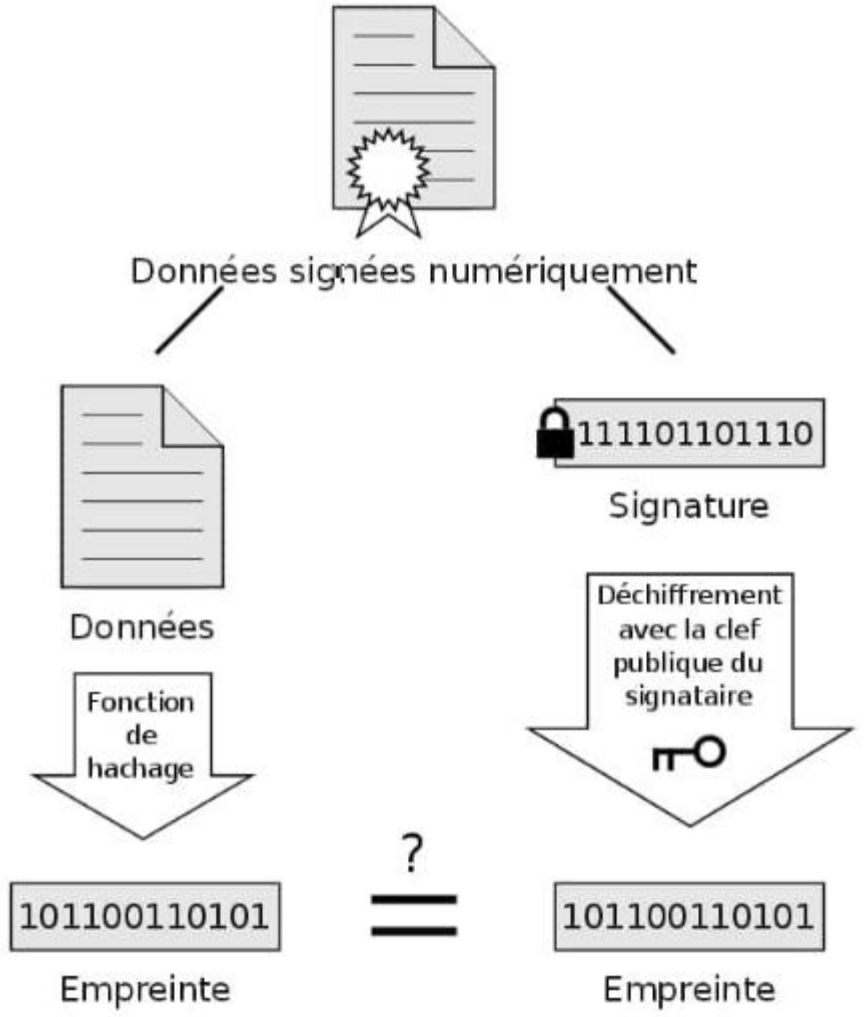
•Une clé privée : Cette clé, personnelle au signataire, chiffre l’empreinte unique du document (et non pas le document lui-même). Le résultat de cette opération est attaché au document, en tant que signature électronique.

•Une clé publique : Lors de la réception du document signé, le destinataire utilise la clé publique pour déchiffrer la signature, ce qui donne une empreinte. Si l’empreinte correspond à celle du document, cela confirme l’authenticité de la signature. Tout changement dans le document après sa signature se traduirait par une incohérence entre l’empreinte déduite de la signature et celle du document, indiquant ainsi son altération.

Signature



Vérification



Si les empreintes sont identiques, la signature est valide

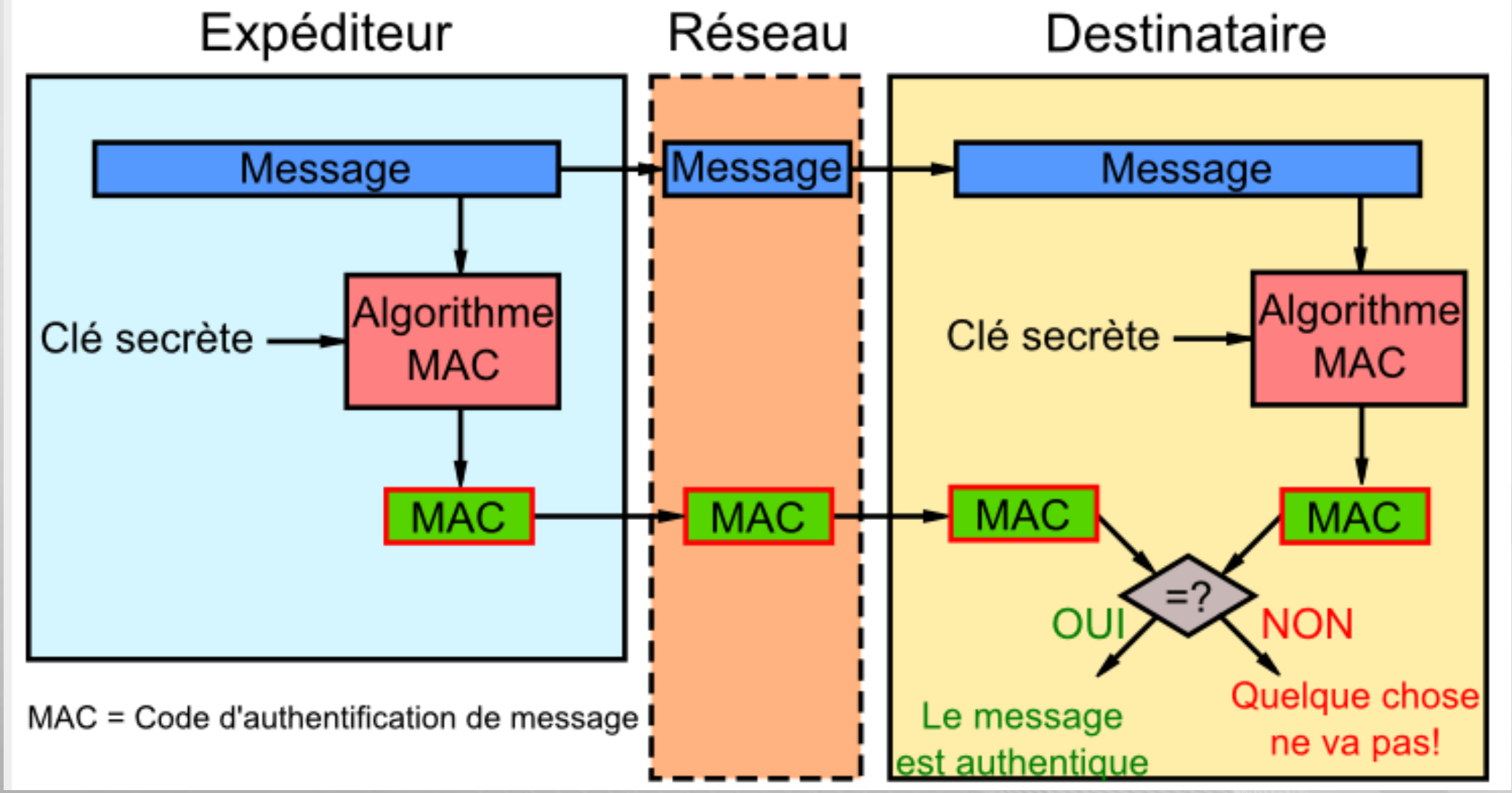
9. Les codes d'authentification des messages:

Le chiffrement permet d’empêcher un individu non autorisé de lire un message, mais il n’empêche pas cette personne de falsifier le message. Un message modifié, même si l’altération n’entraîne rien de non-sens, peut avoir des coûts réels. Un code d’authentification de message (MAC) permet d’empêcher la falsification des messages.

Les **codes d'authentification des messages** (MAC, pour *Message Authentication Code*) sont des valeurs de hachage utilisées pour vérifier l'intégrité d'un message et son authenticité. Ils permettent de garantir que le message n'a pas été altéré et qu'il provient bien de l'expéditeur authentifié.

Un **MAC** est un court code calculé à partir du message et d'une clé secrète partagée entre l'expéditeur et le destinataire. Ce code permet au destinataire de vérifier :

- Si le message n’a pas été modifié pendant la transmission (**intégrité**).
- Si le message provient bien de l'expéditeur attendu, à condition qu'il possède la clé secrète partagée (authenticité).


Expéditeur
Réseau
Destinataire
Message
Message
Message
Clé secrète
Algorithme MAC
Clé secrète
Algorithme MAC
MAC
MAC
MAC
MAC
=?
OUI
NON
Le message est authentique
Quelque chose ne va pas!
MAC = Code d'authentification de message

Algorithmes de MAC

Il existe plusieurs algorithmes pour générer des MACs. Les plus utilisés sont :

1. **HMAC (Hashed Message Authentication Code)**

Le HMAC utilise une fonction de hachage cryptographique (par exemple SHA-256 ou SHA-3) combinée avec une clé secrète. Il s’agit d’une des méthodes les plus courantes et les plus sûres pour la génération d'un MAC.

Le HMAC calcule le hachage du message en le combinant avec une clé secrète et applique le hachage à plusieurs reprises pour renforcer la sécurité.

Principe de Fonctionnement de HMAC

Le fonctionnement de HMAC repose sur deux éléments principaux :

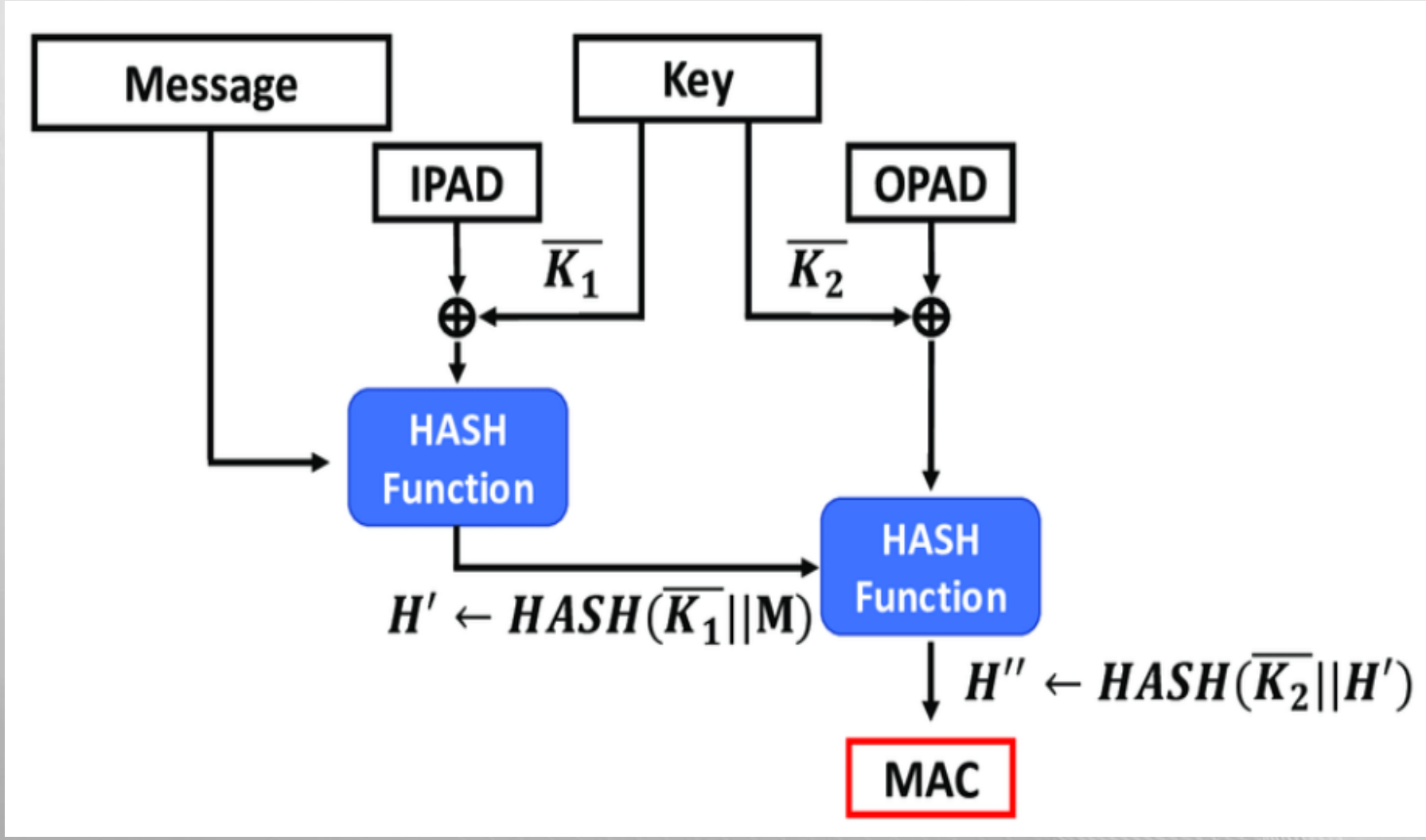
1. **Une clé secrète** : Partagée entre l'expéditeur et le destinataire, elle est utilisée pour générer le HMAC.
2. **Une fonction de hachage** : Comme SHA-256, SHA-1, ou d'autres algorithmes de hachage cryptographique.

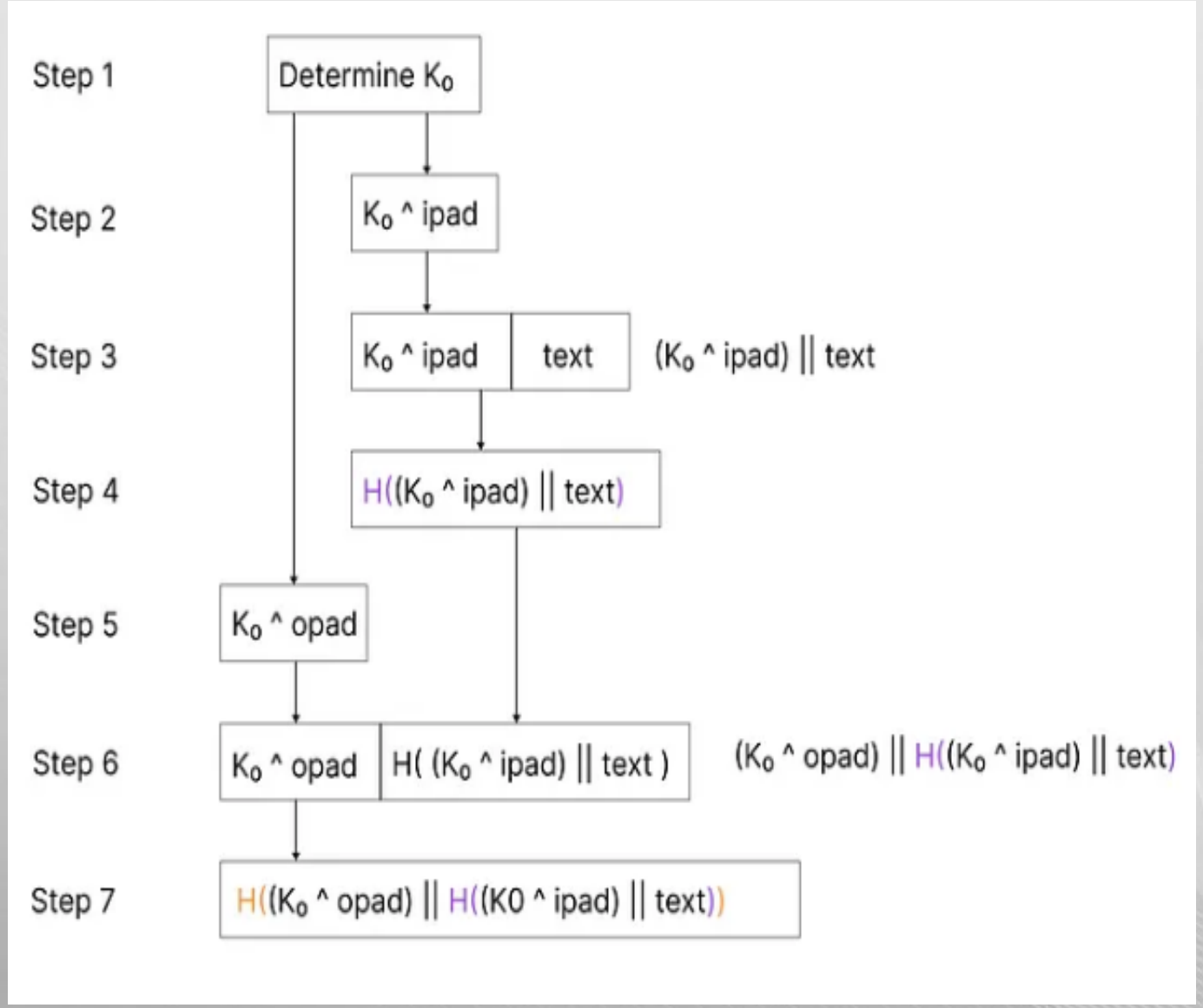
Le calcul du HMAC implique plusieurs étapes :

1.Préparation de la clé : Si la clé est plus longue que la taille du bloc de la fonction de hachage (par exemple 64 octets pour SHA-256), elle est hachée pour obtenir une clé de taille appropriée. Si la clé est plus courte, elle est complétée avec des zéros jusqu'à la taille du bloc.

2.Création de la clé interne et externe : La clé secrète est combinée avec des constantes (appelées ***ipad*** et ***opad***) pour créer deux versions de la clé : la clé interne et la clé externe.

•**ipad** : une séquence de 0x36 (binaire) qui est XORée avec la clé.
•**opad** : une séquence de 0x5c (binaire) qui est également XORée avec la clé.

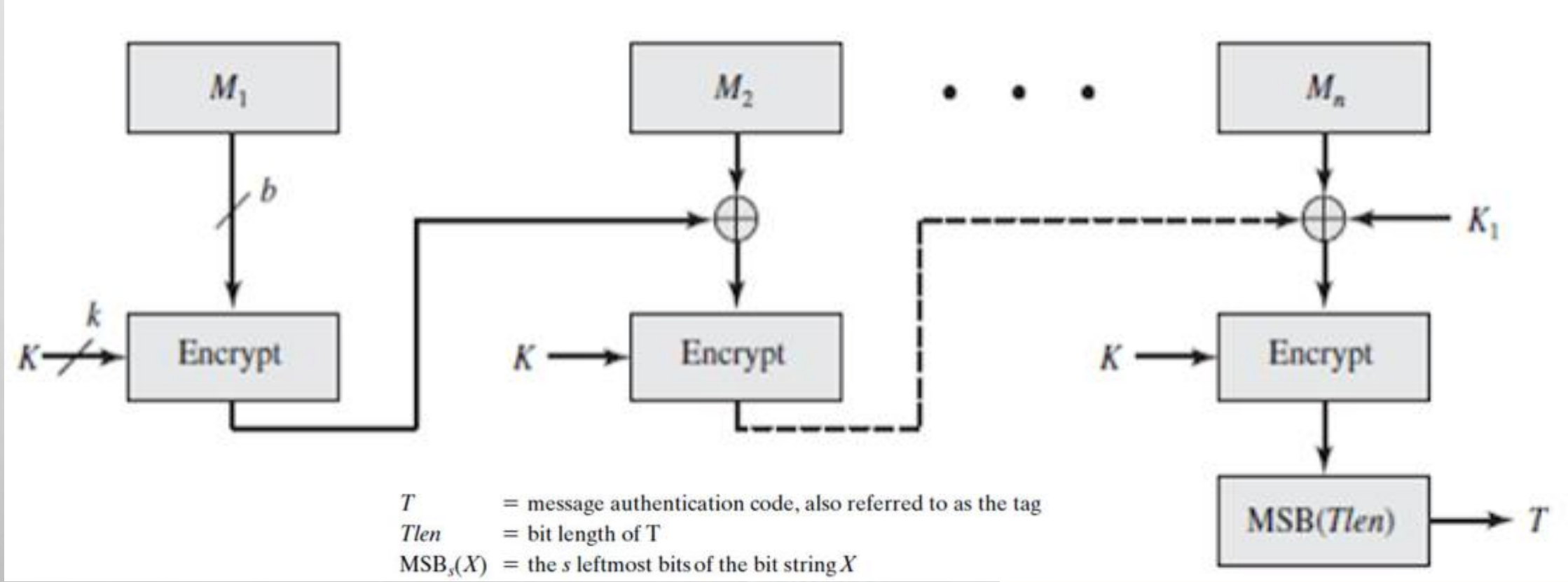

Message
Key
IPAD
OPAD
$\overline{K_1}$
$\overline{K_2}$
HASH Function
HASH Function
$H' \leftarrow HASH(\overline{K_1}||\mathrm{M})$
$H'' \leftarrow HASH(\overline{K_2}||H')$
MAC


Step 1
Determine K_0
Step 2
K_0 ^ ipad
Step 3
K_0 ^ ipad
text
(K_0 ^ ipad) || text
Step 4
H((K_0 ^ ipad) || text)
Step 5
K_0 ^ opad
Step 6
K_0 ^ opad
H((K_0 ^ ipad) || text)
(K_0 ^ opad) || H((K_0 ^ ipad) || text)
Step 7
H((K_0 ^ opad) || H((K0 ^ ipad) || text))

2. CMAC (Cipher-based Message Authentication Code)

Le **CMAC** est un mécanisme de validation de l'intégrité et de l'authenticité des messages.

Le **CMAC** repose sur un algorithme de chiffrement symétrique (comme **AES** ou **DES**) pour générer un code d'authentification. Le processus est similaire à celui de HMAC, mais au lieu d'utiliser une fonction de hachage, il repose sur un chiffrement pour obtenir le code d'authentification des messages.



CMAC avec AES

Pour AES, b = 128, et pour triple DES, b = 64. Le message est divisé en n blocs (M1 , M2 ,…, Mn). La taille de la clé k est de 128, 192 ou 256 bits ; pour triple DES, la taille de la clé est de 112 ou 168 bits.

$C1 = E(K, M1)$

$C2 = E(K, [M2 \oplus C1])$

$C3 = E(K, [M3 \oplus C2])$

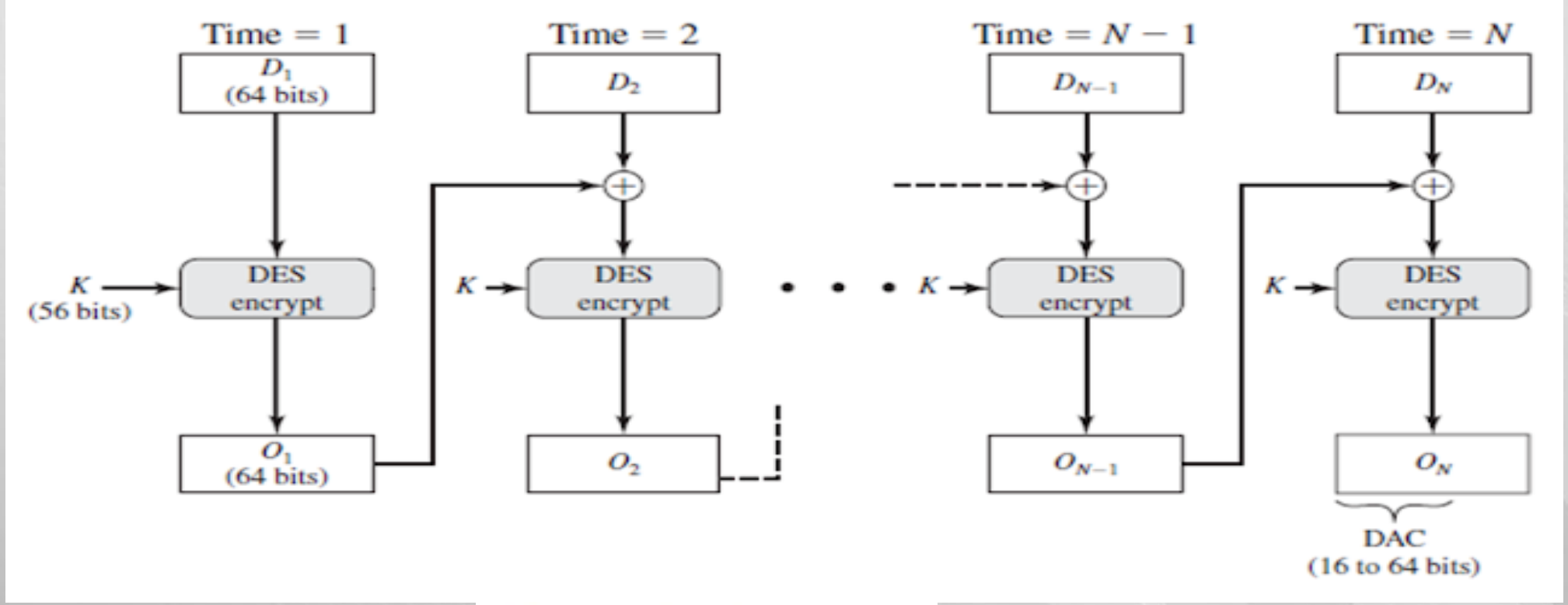
.

.

.

$Cn = E(K, [Mn \oplus Cn\text{-}1 \oplus K1])$

CMAC avec DES



$$
\begin{aligned}
O_1 &= E(K, D) \\
O_2 &= E(K, [D_2 \oplus O_1]) \\
O_3 &= E(K, [D_3 \oplus O_2]) \\
&\cdot \\
&\cdot \\
&\cdot \\
O_N &= E(K, [D_N \oplus O_{N-1}])
\end{aligned}
$$

c) UMAC (Universal Message Authentication Code)

UMAC est un algorithme qui utilise des fonctions de hachage rapide, comme **Poly1305**, pour générer des codes d'authentification de messages avec une performance optimisée; et des techniques de cryptographie symétrique pour produire un MAC de manière rapide et efficace.

UMAC utilise une combinaison de deux éléments essentiels pour générer un code d'authentification des messages :

1.Fonction de hachage : Utilisation de fonctions de hachage cryptographiques rapides, souvent adaptées à des architectures modernes comme celles des processeurs multi-cœurs ou des plateformes hautes performances.

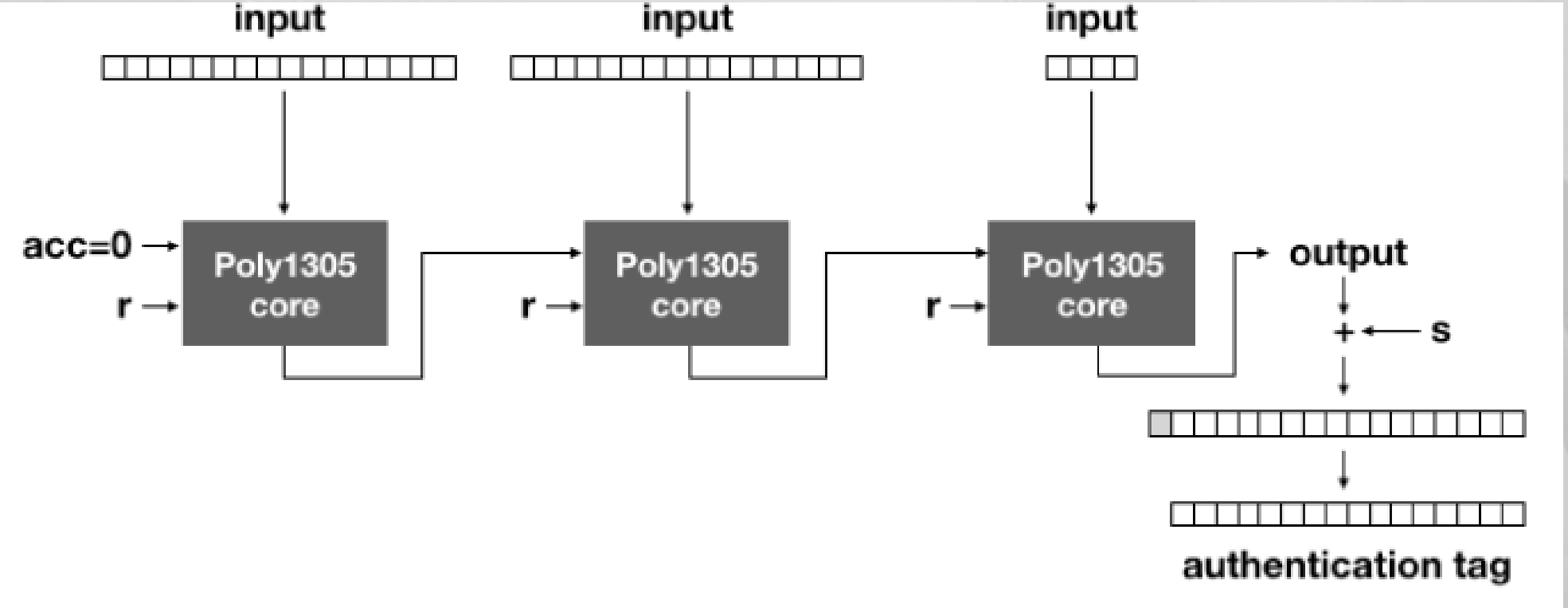
2.Clé secrète : Une clé secrète partagée entre l'expéditeur et le destinataire qui sert à produire le code d'authentification.

1.Clé secrète (K) : La clé secrète est divisée en deux parties :

- ✓ **K1** (128 bits) : Utilisée pour les calculs sur les blocs de message(les calculs d'addition et de multiplication avec les blocs du message).
- ✓ **K2** (128 bits) : Utilisée pour la manipulation des blocs pendant le processus de hachage.

2. Message (M) : Le message est divisé en blocs de taille fixe, et chaque bloc est "haché" avec l'algorithme Poly1305 en effectuant une série de multiplications et d'additions modulo un nombre premier$2^{130} - 5$ pour produire un résultat de taille fixe (128 bits).

3. Combinaison des résultats : Les résultats des différentes étapes sont combinés pour produire le **UMAC** final.



Applications des MAC

Les codes d'authentification des messages sont utilisés dans une variété de scénarios pour sécuriser les échanges d'informations :

- **Protocoles de communication sécurisés** : Les MACs sont utilisés pour garantir l'intégrité des messages dans des protocoles comme **TLS/SSL** (utilisé pour sécuriser les connexions Internet) ou **IPSec** (utilisé pour sécuriser les réseaux).
- **Authentification des messages** : Dans des systèmes de messagerie sécurisée, les MACs peuvent être utilisés pour garantir que les messages envoyés n'ont pas été falsifiés.
- **Authentification des API** : Les API (interfaces de programmation) peuvent utiliser des MACs pour valider les requêtes entrantes et s'assurer qu'elles proviennent d'une source authentique, en vérifiant l'intégrité et l'authenticité des messages échangés.
- **Stockage sécurisé des mots de passe** : Les MACs peuvent être utilisés pour sécuriser les mots de passe, notamment dans les systèmes qui emploient un **salt** et une clé secrète partagée.

Sécurisation de la clé secrète

L'un des aspects cruciaux de l'utilisation des MACs est la gestion de la clé secrète. Si un attaquant connaît la clé, il peut créer un MAC valide pour un message falsifié. Par conséquent :

•Il est essentiel de protéger la clé secrète en la stockant de manière sécurisée.

•Utiliser des mécanismes comme la gestion des clés dans des modules matériels de sécurité (HSM) ou des coffres-forts logiciels.

Limitations des MAC

•**Clé secrète partagée** : L'une des limitations est que la clé secrète doit être partagée entre l'expéditeur et le destinataire, ce qui peut entraîner des problèmes de distribution sécurisée.

•**Pas de confidentialité** : Un MAC garantit l'intégrité et l'authenticité, mais il ne fournit pas de confidentialité. Il ne chiffre pas le message.

9. Les générateurs de nombre pseudo aléatoire

La génération de nombres aléatoires et les fonctions à sens unique sont deux éléments fondamentaux dans la conception des systèmes cryptographiques et dans l'analyse de leurs sécurités.

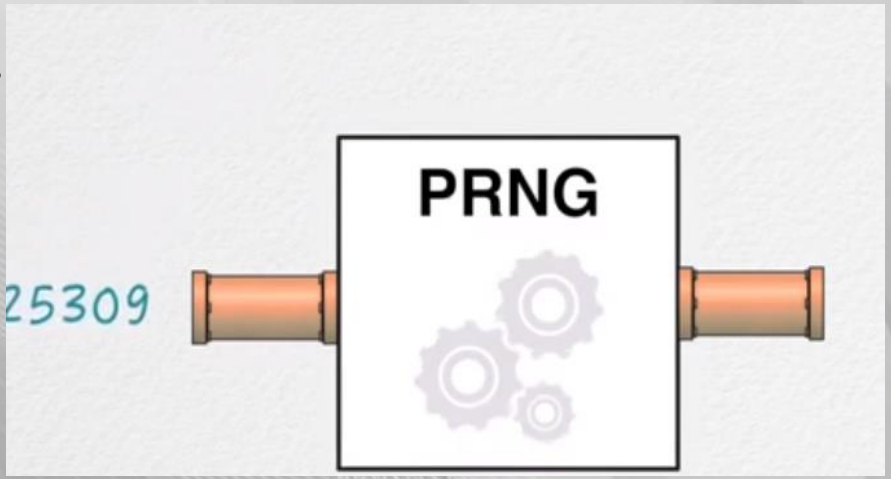
Il existe plusieurs **sources de l'aléa dans la nature comme les phénomènes physiques (quantique, radioactivité, bruit thermique, etc.)** qui présentent de vrais aléas. Cependant, dans les applications informatiques il est possible de générer du pseudo-aléa en utilisant des générateurs pseudo-aléatoires.

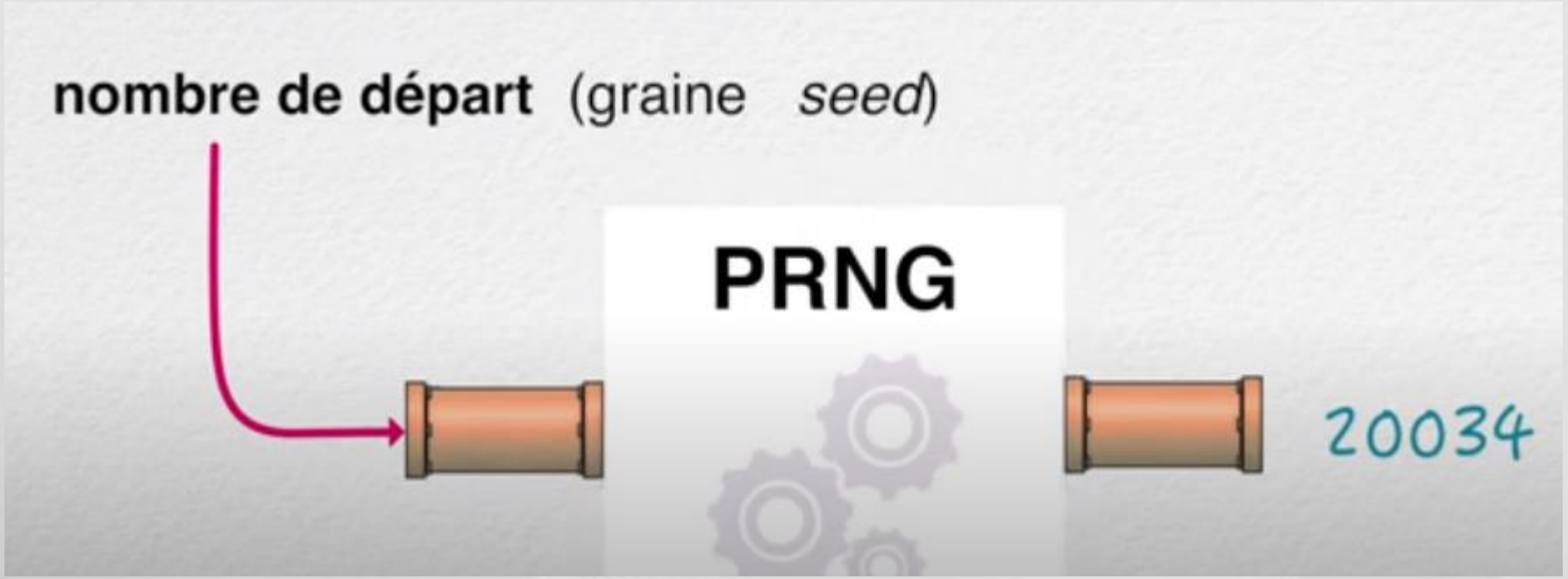
Un générateur pseudo-aléatoire est en général construit à **partir d'une fonction mathématique déterministe**, qui **calcule par récurrence une suite d'états**, dont est tirée la suite pseudo-aléatoire.

- La suite d'états est-elle même une suite de valeurs internes cachées
- La valeur initiale de l'état est appelée le **germe ou la graine**.

À partir du germe X_0 on calcule une suite cachée d'états :

$$X_k = f(X_k - 1)$$





La graine (*seed*)

- L'heure POSIX

1552249206

Caractéristiques d'un bon PRNG :

1. **Période longue** : La séquence de nombres doit être longue avant de se répéter.
2. **Uniformité** : Les nombres générés doivent être distribués de manière uniforme sur l'intervalle choisi (par exemple entre 0 et 1).
3. **Indépendance** : Les valeurs successives doivent être indépendantes.
4. **Prédictibilité** : Si quelqu'un connaît l'état interne du PRNG (ou la graine), il peut prédire la séquence de nombres futurs. Par conséquent, pour des applications sensibles comme la cryptographie, des générateurs spécifiquement conçus pour être imprédictibles sont utilisés, appelés **générateurs de nombres aléatoires cryptographiquement sûrs**.

Notez

- Si nous générons une séquence de nombres avec une graine , puis nous générons une autre séquence en utilisant la même graine, les deux séquences seront identiques. Puisque l'espace d'état est fini, *Sn* doit éventuellement revenir à un état dans lequel il se trouvait quelque temps auparavant.

- Le plus petit entier *p* tel que pour un état donné la fonction *f* retourne à cet état après ***P*** itérations est appelé ***la période du générateur***. De toute évidence, des périodes plus longues valent mieux que des périodes courtes. Mais une longue période à elle seule garantit certainement un bon résultat aléatoire.

Un exemple de générateur pseudo-aléatoire est la fonction **rand () ou random ()** que l'on trouve dans la plupart des langages des programmation.

Les types des générateurs pseudo aléatoire:

1. Générateur Congruentiels linéaire

Un **générateur congruentiel linéaire** est un générateur de nombres pseudo-aléatoires dont l'algorithme, introduit en 1948 par D.H Lehmer, sous une forme réduite, pour produire des nombres aléatoires, est basé sur des congruences et une **<u>fonction linéaire</u>.**

On définit une suite$(X_n)_n$:

$$(X_n)_n = (a\, X_{n-1} + b) mod\ m$$

•Un germe x_0 qui doit être un nombre entier positif,
•Un multiplicateur a
•Un incrément b

choix des paramètres *a*, *b* et *m*

Dans ce type d'algorithme, la qualité du générateur va entièrement dépendre du choix de *a*, *b* et *m* car on ne peut pas se satisfaire d'un générateur qui produit des suites plus ou moins aléatoires, sans aucune raison apparente, selon le choix de X_0.

Choisir *a*, *c* et *m* " au hasard " n'est pas une bonne idée, prenons un exemple :

Soit le générateur congruentiel linéaire (*a*=25, *c*=16, *m*=256), nous obtenons

- avec X_0 = *10*, la suite : 10, 10, 10, 10, 10, ...
- avec X_0 = *11*, la suite : 11, 35, 123, 19, 235, 3, 91, 243, 203, 227, 59, 211, 171, 195, 27, 179, 139, 163, 251, 147, 107, 131, ...
- avec X_0 = *12*, la suite : 12, 60, 236, 28, 204, 252, 172, 220, 140, 188, 108, 156, 76, 124, 44, 92, 12, 60, 236, 28, 204, 252, 172, ...

Le choix de a, b est primordial.

Afin de pouvoir choisir une graine X_0 sans contraintes entre 0 et m-1, il faut chercher à maximiser la période du générateur. Or il se trouve que les valeurs de a et b sont connues, ce qui permet d'obtenir une période maximale (égale à m).

Pour maximiser la période du générateur, afin qu'elle soit égale à M, on observera les conditions suivantes :

- b doit être premier avec M, c'est à dire $PGCD(b,M)=1$.
- Pour chaque nombre premier p divisant M, $(a-1)$ est un multiple de p.
- $(a-1)$ doit être un multiple de 4 si M en est un.

Dans le cas où b est nul, la période est maximale si :

- M est premier,
- a^{M-1} est un multiple de M,
- a^{j-1} n'est pas divisible par M pour $j=1, j=2..., j=m-2$

Obtenir des nombres uniformes dans [0,1] Comme $0 \leq Xk \leq m-1$, il y a trois possibilités pour construire un nombre uniforme Uk entre 0 et 1 :

$$U_K = \frac{X_K}{m} \text{ ou } U_K = \frac{X_K}{m-1} = \frac{X_K}{m} + \frac{1}{2m}$$

Algorithme	a	B	M
RANDU	$65539\ (2^{16}+3)$	0	2^{31}
Générateur du Turbo Pascal	129	907633385	2^{32}
Standard minimal*	16807	0	$2^{31}-1$
Générateur de Marsaglia*	69069	0	2^{32}
Générateur de Knuth&Lewis*	1664525	1013904223	2^{32}
Générateur de Haynes*	6364136223846793005	0	2^{64}

** Générateur de bonne qualité*

2. Extension matricielle

Il existe une extension matricielle des générateurs à congruence linéaire. On parle alors de générateurs à congruence matricielle :

$$X_{n+1} = (A \times X_n + B) \; mod \; m$$

X_n et X_{n+1} sont des vecteurs de dimension **K** tandis que A et B sont des matrices de taille Kx K .

Ces générateurs sont souvent utilisés sous une forme réduite c'est-à-dire sans le membre B . Nous les retrouverons un peu plus loin lorsque nous traiterons les générateurs GFSR.

3. Générateurs congruentiels multiplicatifs

La plupart des logiciels de calcul ou de développement disposent d'un générateur de nombres aléatoires. Pour des raisons principalement historiques, celui-là est souvent de type linéaire congruentiel.

Définition et propriété

Lorsque b = 0 , on parle de générateur congruentiel multiplicatif (MLCG) et la récurrence devient :

$$(X_n)_n = (a\, X_{n-1}) mod\ m \quad ; \forall\, k \in N^*$$

Dans ce cas, l'état 0 est absorbant : si Xk = 0 , alors les termes suivants dans la suite seront tous nuls. Le générateur doit donc prendre ses valeurs dans Nm={0} , de cardinal m −1 .

4. Générateur de Tausworthe (ou LFSR : Linear Feedback Shift Register)

Le générateur de Tausworthe est une extension du générateur congruentiel linéaire qui consiste à ne plus utiliser seulement X_{n-1}pour fabriquer un nouvel élément X_n mais plutôt un ensemble de valeurs précédentes :

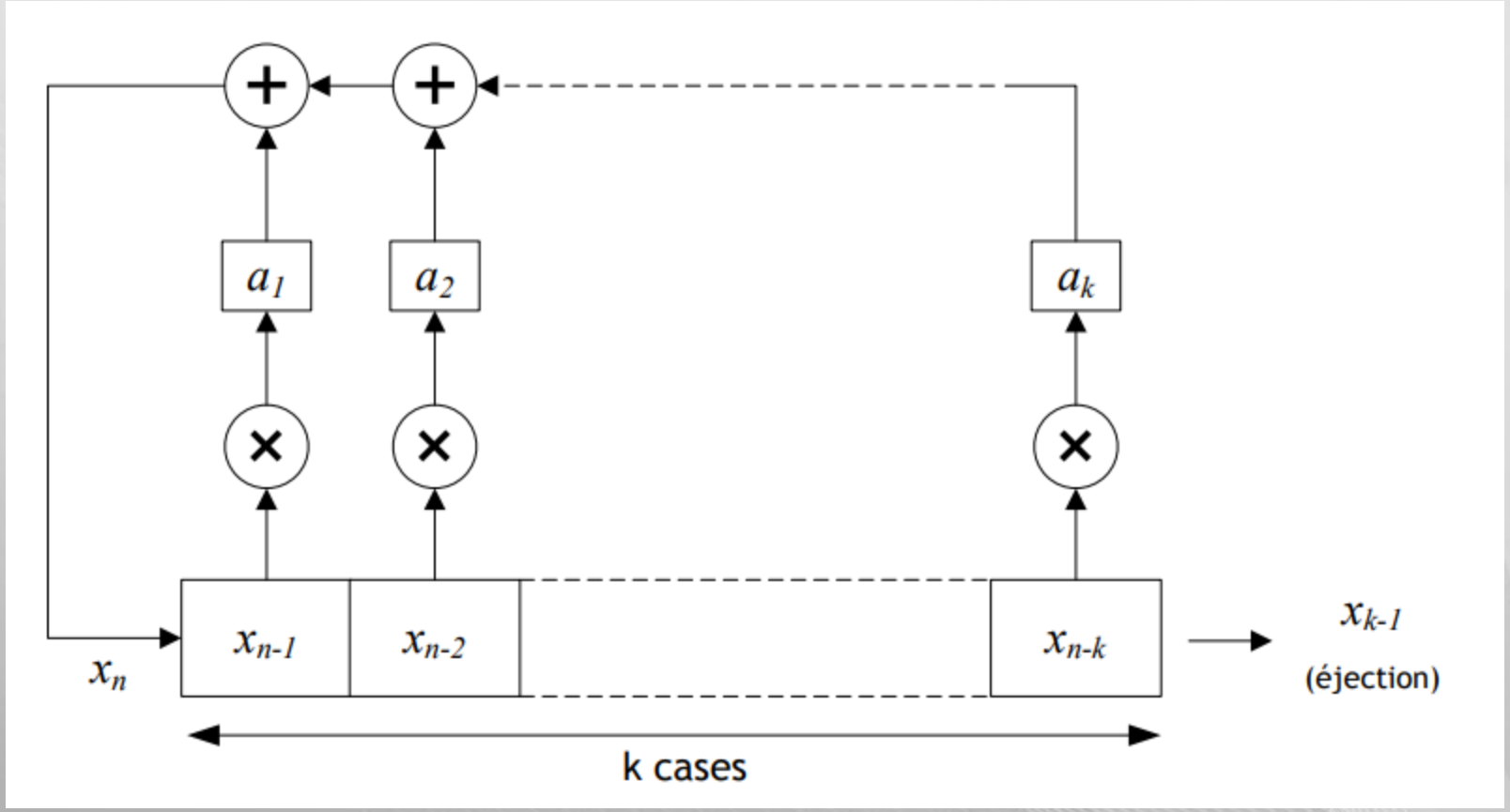
$$x_n = (a_1 \times x_{n-1} + \cdots + a_k \times x_{n-k})\ mod\ m \text{ avec } n \geq k$$

Le LFSR est simplement un générateur de Tausworthe où m=2, c'est-à-dire adapté à une utilisation informatique (les deviennent des bits) car les opérations modulo 2 peuvent se traduire directement par un opérateur XOR (ou exclusif) :

$$X_n = (a_1 X_n + \cdots + a_K X_{n-k})\ mod\ 2$$

Où:

$a_1, a_2 \ldots \ldots, a_k \quad \in \{0, 1\}$ et $a_k \quad \neq 0$


a_1
a_2
a_k
x_n
x_{n-1}
x_{n-2}
x_{n-k}
x_{k-1}
(éjection)
k cases

5. Générateur de Fibonacci

La méthode de Fibonacci est une méthode dérivée des générateurs à congruence additive, basée sur la suite de Fibonnacci :

$$\mathcal{F}_n = (\mathcal{F}_{n-1} + \mathcal{F}_{n-2})\ mod\ m \text{ avec } \mathcal{F}_0, \mathcal{F}_1 \text{ fixés (graines)}$$

$\mathcal{F}_0$	$\mathcal{F}_1$	$\mathcal{F}_2$	$\mathcal{F}_3$	$\mathcal{F}_4$	$\mathcal{F}_5$	$\mathcal{F}_6$	$\mathcal{F}_7$	$\mathcal{F}_8$	$\mathcal{F}_9$	$\mathcal{F}_{10}$	$\mathcal{F}_{11}$	$\mathcal{F}_{12}$	$\mathcal{F}_{13}$	$\mathcal{F}_{14}$
0	1	1	2	3	5	8	13	21	34	55	89	144	233	377

$\mathcal{F}_{15}$	$\mathcal{F}_{16}$	$\mathcal{F}_{17}$	$\mathcal{F}_{18}$	$\mathcal{F}_{19}$	$\mathcal{F}_{20}$	$\mathcal{F}_{21}$	$\mathcal{F}_{22}$	$\mathcal{F}_{23}$	$\mathcal{F}_{24}$	$\mathcal{F}_{25}$	...
610	987	1597	2584	4181	6765	10946	17711	28657	46368	75025	...

La suite de Fibonacci originale.

$$\mathcal{F}_n = \frac{1}{\sqrt{5}}\left(\varphi^n - \left(-\frac{1}{\varphi}\right)^n\right) \text{ où } \varphi \text{ est le nombre d'or}$$

Expression fonctionnelle pour $\mathcal{F}_0 = 0, \mathcal{F}_1 = 1$

Evidemment, la qualité de la génération est médiocre car les nombres consécutifs sont fortement corrélés.

5. GFSR (Registre à décalage à rétroaction linéaire de Galois)

Il est possible de représenter le fonctionnement du générateur de Tausworthe sous forme matricielle :

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ x_5 & x_4 & x_3 & x_2 & x_1 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} mod\ m \text{ avec } k = 5$$

$$X_n = (A_1 \times X_{n-1} + \cdots + A_k \times X_{n-k})\ mod\ m \text{ avec } n \geq k$$

6. TGFSR

En 1992, Makoto Matsumoto et Yoshiharu Kurita ont apportés des modifications au GFSR pour en augmenter significativement la période. Le terme , qui était auparavant une matrice identité LxL est remplacé par une matrice particulière de même taille qui va « **twister** » la récurrence.

$$X_n = \left(X_{n-(k-r)} + X_{n-k}\right) mod\ 2$$

r est une constante et A est une matrice .

$$\begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & \dots & 0 \\ 0 & 0 & 0 & 0 & 1 \\ a_L & a_{L-1} & a_{L-2} & \dots & a_1 \end{bmatrix}$$

Le générateur atteint une période maximale de $2^{KL} - 1$.

7. Générateurs cryptographiques

Les générateurs cryptographiques sont décrits comme sûr pour des applications cryptographiques. Il est généralement très difficile d’en prédire la sortie car ceux ci se basent souvent sur **les algorithmes à clé publique** existant en cryptographie. Ces algorithmes se basent sur des fonctions dîtes "**à sens unique**". Ces fonctions sont conçues de telles sorte que :

- L'obtention de l'image de à partir de soit très simple.
- La recherche de à partir de l'image de est un problème mathématique réputé difficile, ou informatiquement trop complexe (= trop lent).

Générateur Blum-Blum-Shub

Blum Blum Shub (BBS) est un algorithme de génération de nombres pseudo aléatoires proposé en 1986 par Lenore Blum, Manuel Blum et Michael Shub, d'où son nom. Le générateur n'est pas approprié pour toutes les applications, car il est relativement lent. Cependant, **son niveau de sécurité élevé** en fait un bon candidat pour des applications cryptographiques.

On donne le produit: n= p . q

Avec p et q sont deux grand nombre premier de la forme (4K+ 3), ils sont gardé secret.

Calculer le premier germe X_0 : pgcd(X_0 ,n)=1).

Les états récursif X_k $sont$ donnés par :

$$X_{k+1} = X_k^2 \bmod n$$

De l'état S_K qui est la valeur interne secréte ; on tire le k^{ime} $bit\ X_K$ de **m** la suite pseudo aléatoire en prenant le bit de poids faible de X_K .

$$U_K = LSB\ (X_K) = X_K\ mod\ 2$$

MERCI

POUR VOTRE ATTENTION